KU LEUVEN

FACULTY OF SCIENCE

Connectivity Bounds in Graphs and Beyond

Erdős Problem 915, from Proof to Search

Louis Vandenbruwaene

Supervisor: Prof. Stijn Cambie
KU Leuven

Thesis presented in
fulfillment of the requirements
for the degree of Master of Science
in Mathematics

Academic year 2026–2027

Open Access Disclaimer

This master's thesis is a student work submitted as part of an academic programme at KU Leuven. The work is made available for examination and archiving in accordance with the applicable KU Leuven regulations and open-access policy for master's theses.

Foreword

This thesis studies how many edges or arcs a network can contain when no pair of vertices is allowed to have too many independent routes between them. The starting point is Erdős Problem 915, an extremal question about edge-disjoint paths in simple graphs. The work is told as a journey. It begins with the variants whose answers can be proved cleanly by hand, watches those proofs grow longer and more delicate as the model changes, and at the point where a single argument no longer suffices it turns to a unified program. That program stores all twelve variants in one common data structure, so a single piece of code serves them all. It measures connectivity exactly with a maximum-flow routine, the standard way to count the independent routes between two points. It runs a prover that sweeps every graph of a small fixed size and, when none beats a target, reports that sweep as a genuine upper-bound proof. And it runs a search that, like a hot metal cooling slowly into shape, settles random graphs toward the densest ones the rules allow.

I am grateful to my supervisor, Prof. Stijn Cambie, and my daily advisor for their guidance, corrections, and mathematical feedback throughout the project. I thank my promotor, Prof. Jan Goedgebeur, whose suggestion of the `nauty` generation pipeline sharply accelerated the directed enumeration. I also thank the people who discussed early versions of the arguments, checked examples, or helped clarify the presentation. Above all I thank my girlfriend Sara, whose patience and encouragement carried me through the long stretches of this work. Any remaining mistakes are my own.

Contribution Statement

This thesis is my own work. I carried out the literature study, recast the twelve variants of the problem in one common language, and designed and wrote the single program that runs through all of them. I built every example, by hand or by computer search, checked each one with that program, wrote the proofs that appear in the thesis, and typeset the whole document. Where a result comes from the literature I say so and cite it. The theorems I lean on but do not claim as mine are those of Mader, Leonard, Sørensen and Thomassen, Menger, Gomory and Hu, and Baranyai, the hypergraph connectivity results of Bérczi, Chandrasekaran, Király and Kulkarni, and the standard probabilistic tools used for the random graphs.

The program has three jobs, and it keeps them separate. It *measures*: given any graph, it computes exactly, through maximum flow, how many independent routes the busiest pair of vertices has. It *proves*: for a fixed number of vertices it sweeps a finite space of graphs, and when nothing in that space beats a target, that sweep is a genuine upper-bound proof rather than a guess. And it *discovers*: a search that cools like a heated metal settling into shape wanders the space of graphs and finds dense ones, which give lower bounds and suggest conjectures but never prove an upper bound by themselves. Every general statement in the thesis rests on a proof by hand or a cited theorem. The computer searches are evidence that points the way, never a replacement for the argument. I used AI language models, including Claude Code, ChatGPT, and Gemini, as tools for literature search, LaTeX editing, proofreading, and trying out proof drafts. I checked every mathematical statement and proof in the thesis myself and take full responsibility for the content.

The genuinely new results are the following. For hyperedge networks I settle the vertex-disjoint problem completely at the two strictest route limits, $m = 2$ and $m = 3$, for every hyperedge size r and every number of vertices n (Theorems A.36 and A.39): the vertex-disjoint and edge-disjoint answers turn out to be equal, both $\lfloor (m-1)(n-1)/(r-1) \rfloor$. The $m = 3$ case rests on a bound for incidence graphs that I prove using Tutte's decomposition of a graph into its three-connected pieces (Lemma A.38). For one-way multigraphs at $m = 3$ I prove an unconditional attachment lemma (Lemma A.27) and an odd-step theorem (Theorem A.29, Remark A.30) that together shrink the whole problem, for every n, down to two finite statements about multigraphs on just seven vertices that a computer can check, and they do so without assuming the unproved minimum-degree conjecture the earlier route needed.

Short Summary

Erdős Problem 915, posed by Bollobás and Erdős [1], asks for the maximum number of edges in a graph on n vertices such that no two vertices are connected by m edge-disjoint paths. In modern notation this asks for

$$\ell_m(n) = \max\{|E(G)| : |V(G)| = n,\ \lambda_G^{\max} \leq m-1\}.$$

Mader proved that, for simple graphs and $n \geq m \geq 2$,

$$\ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

This thesis revisits this theorem and studies the corresponding extremal problem under three independent changes: the graph model (simple graphs, multigraphs, and hypergraphs), the path-separation notion (edge-disjoint or internally vertex-disjoint), and the presence or absence of directions. Sampled random graphs $G(n,p)$ run alongside throughout as a lens on the typical case rather than as a separate model.

The thesis is organised as a journey from hand proofs to computation. The early variants are settled rigorously: the multigraph value $L_m(n) = (m-1)(n-1)$, the random-graph threshold $p^* = m/n$, the equality of the edge and vertex problems for $m \leq 4$, and the first divergence at $m = 5$, where Sørensen and Thomassen prove $k_5(n) = \lfloor 8n/3 \rfloor - 3$. The directed case forces the turn: the value $\ell_2^{\mathrm{dir}}(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$ is proved by a long induction, and for $m \geq 3$ the hand proof gives way to a conjecture, after an explicit ten-vertex graph rules out the naive guess. The centerpiece is then a unified program. One multiplicity-matrix representation models almost all twelve variants, an exact maximum-flow checker measures local connectivity, a cut-counting optimisation certifies upper bounds for small sizes, and a search guided by edge sensitivity, run as both simulated annealing and tabu search, discovers extremal graphs. The program rediscovers, each in its own variant, the double star (undirected multigraphs at $m = 2$), the directed hub, the one-directional bipartite digraph, and the augmented-bipartite construction (the directed cases), certifies the directed multigraph values $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$ for $n \leq 6$, and frames the remaining open problems on the directed frontier.

The interplay then pays back in the other direction. The hypergraph vertex problem is settled completely at $m = 2$ and $m = 3$: $k_m^{(r)}(n) = \lfloor (m-1)(n-1)/(r-1) \rfloor$ for every uniformity r, the $m = 3$ case through a new rank bound on incidence graphs proved by way of Tutte's triconnected decomposition. On the directed side, an unconditional attachment lemma and a conditional odd-step theorem reduce the entire directed multigraph problem at $m = 3$, value and extremal characterisation for all n, to two finite, machine-checkable statements about multigraphs on seven vertices, and directed hypergraphs receive their first bounds, quadratic in n.

Abbreviations and Symbols

GH	Gomory-Hu
w.h.p.	with high probability

$G = (V, E)$	graph with vertex set V and edge set E
$D = (V, A)$	directed graph with vertex set V and arc set A
n	number of vertices
m	forbidden-connectivity parameter, with all local connectivities at most $m - 1$
$\lambda_G(u, v)$	maximum number of edge-disjoint u-v paths
$\lambda_G^{\max}$	maximum local edge-connectivity over all vertex pairs
$\kappa_G(u, v)$	maximum number of internally vertex-disjoint u-v paths
$\kappa_G^{\max}$	maximum local vertex-connectivity over all vertex pairs
$\ell_m(n)$	simple undirected edge-disjoint extremal function
$L_m(n)$	undirected multigraph edge-disjoint extremal function
$k_m(n)$	simple undirected vertex-disjoint extremal function
$\ell_m^{\mathrm{dir}}(n)$	simple directed arc-disjoint extremal function
$L_m^{\mathrm{dir}}(n)$	directed multigraph arc-disjoint extremal function
$k_m^{\mathrm{dir}}(n)$	simple directed vertex-disjoint extremal function
$\mu(u, v)$	multiplicity of an edge or arc between u and v
$\sigma(e)$	edge sensitivity $\lambda^{\max}(G) - \lambda^{\max}(G - e)$
$G(n, p)$	binomial random graph

Contents

Chapter 1

The Problem and Its Base Cases

A network is useful not only because it connects, but because it connects in more than one way. A single road between two towns is a liability, and a second independent road is insurance. The mathematical content of that intuition is the number of routes between two points that share no segment, and the question this thesis circles is how many connections a network can hold before some pair of points is forced to have too many such routes. This chapter builds the question from nothing, settles the variants whose proofs are short enough to read in an afternoon, and then follows the same question along three axes until the proofs stretch past breaking. By the end the case for a machine has made itself.

1.1 Graphs and routes

To state the question precisely we need only a few ideas. A *graph* G is a finite set of points, called *vertices*, together with a set of links, called *edges*, each joining two of the vertices. We write $V(G)$ for the set of vertices and $E(G)$ for the set of edges, so that $|E(G)|$ is the number of edges, the very quantity this thesis tries to make as large as possible. One pictures the vertices as dots and the edges as lines between them.

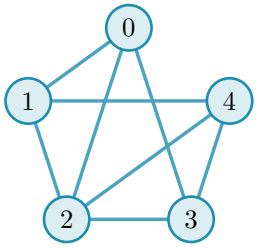


Figure 1.1. A simple graph on five vertices. It is K_5, the *complete* graph in which every one of the $\binom{5}{2} = 10$ pairs is joined by an edge, with the two edges $\{0,4\}$ and $\{1,3\}$ removed, so its edge set $E(G)$ holds eight edges. The five dots are the vertex set $V(G)$ and the lines are the edges. Because at most one line joins any given pair and no edge loops back to its own endpoint, this is a simple graph. The same graph returns in Figure 1.2, once we can count the independent routes between a pair.

A graph is the bare skeleton of a network: the vertices are the towns or the devices, and the edges are the roads or the links between them. We write n for the number of vertices, and the *degree* of a vertex is the number of edges meeting it. Unless we say otherwise our graphs are *simple*, meaning that at most one edge joins any given pair of vertices and its endpoints are distinct.

A *path* from a vertex u to a vertex v is a sequence of distinct vertices that starts at u, ends at v, and joins each consecutive pair by an edge. It is the formal version of a route through the network. Two paths between the same endpoints are *edge-disjoint* if no edge belongs to both, so that the loss of a single edge can break at most one of them. The largest number of pairwise edge-disjoint paths between u and v is their *local edge-connectivity* $\lambda_G(u, v)$, the count of genuinely independent routes the network offers to that pair. The pair with the most independent routes sets the ceiling for the whole graph:

$$\lambda_G^{\max} = \max_{u \neq v} \lambda_G(u, v),$$

the largest local edge-connectivity anywhere in G. A theorem of Menger [2], recalled in Appendix A, gives this quantity a second face: $\lambda_G(u, v)$ also equals the least number of edges one must remove to cut every route from u to v (Figure 1.2). Many independent routes between a pair is exactly the same thing as a costly cut between them.

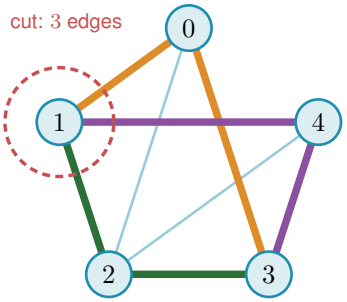


Figure 1.2. Menger's theorem on the graph of Figure 1.1. The pair $1, 3$ carries no edge of its own, yet three routes still join them with no shared edge, 1-0-3 (orange), 1-2-3 (green), and 1-4-3 (purple), each through a different middle vertex, so $\lambda(1, 3) = 3$. The three edges meeting vertex 1 (inside the red dashed ring) are a smallest set whose removal cuts every 1-3 route, and no two edges can, so the cheapest cut also has size three. The most independent routes a pair can muster always equals the cheapest cut between them, the fact every bound in this thesis leans on.

1.2 Erdős Problem 915

The constraint at the heart of this thesis is a ceiling on connectivity. Requiring $\lambda_G^{\max} \leq m - 1$ says that no pair of vertices is joined by as many as m edge-disjoint paths, or equivalently that every pair can be severed by removing fewer than m edges. The question is how many edges a graph can hold while staying under that ceiling.

Question 1.1 (Erdős Problem 915 [3]). How many edges can a graph on n vertices have if no two of its vertices are joined by m edge-disjoint paths?

Writing the answer as an extremal function,

$$\ell_m(n) = \max\{\, |E(G)| : |V(G)| = n,\ \lambda_G^{\max} \leq m - 1 \,\},$$

the problem asks for $\ell_m(n)$, the most edges possible under the ceiling. Mader settled the simple-graph case completely, proving both halves at once: no graph under the ceiling has more than

$\lfloor m(n-1)/2 \rfloor$ edges (the upper bound), and some graph reaches that count (the lower bound).

Theorem 1.2 (Mader [4])**.** *For all* $n \geq m \geq 2$*,*

$$\ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

Mader's own proof is a direct structural induction. The proof given in full in Appendix A takes a different road, by way of the Gomory–Hu tree, found independently of Mader, together with a characterisation of the graphs that attain equality (Theorem A.11). This is the road the thesis adopts throughout, because the very same tree returns for the multigraph and hypergraph variants (Theorem 1.3, Proposition 1.14), so a single argument serves all of them rather than each needing its own. Every graph has such a tree (Theorem A.2), and we single out the one identity that makes it work, because the same idea reappears when the program of Chapter 2 stores connectivities as a tree (a graph without cycles): a Gomory–Hu tree of G is a weighted tree on the same vertex set in which $\lambda_G(u, v)$ equals the lightest edge on the tree path from u to v. All $\binom{n}{2}$ local connectivities are therefore encoded by $n-1$ numbers, and $\lambda_G^{\max}$ is simply the heaviest edge of the tree.

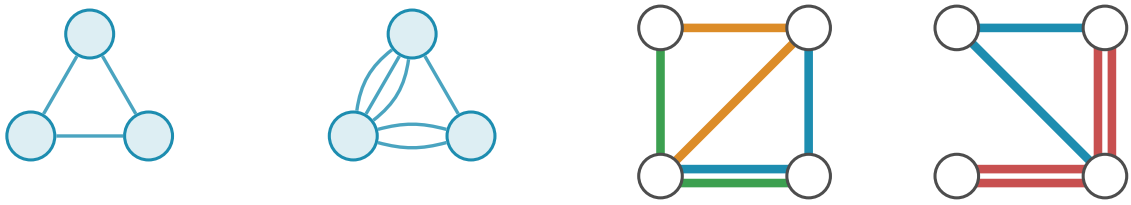
The whole argument (Appendix A) is one act of double counting on that tree. The word *charged* below is just accounting: to charge an edge to a tree edge is to assign it there for the count, and the proof works by charging every edge of G to tree edges and then adding the charges up. Each of the $n-1$ tree edges stands for a minimum cut of G, and an original edge of G is charged once for every tree edge lying between its endpoints, so summing the tree-edge weights counts every edge of G at least once and, crucially, counts most of them at least twice. The only edges charged a single time are those joining tree-adjacent vertices, and a *simple* graph has at most $n-1$ of those, one per tree edge. Every other edge is charged twice. Since each tree weight is a local connectivity and so at most $m-1$, the two counts meet at $2|E(G)| - (n-1) \leq (m-1)(n-1)$, which rearranges to the floor. The factor of two, and with it the gap between Mader's $\lfloor m(n-1)/2 \rfloor$ and the larger value $(m-1)(n-1)$ that parallel edges will buy in Theorem 1.3, is exactly the dividend of simplicity, the one place the argument uses that no pair carries a parallel edge.

One example fixes the scale. The complete graph K_n joins every pair, so every pair has $n-1$ routes that share no edge, $\lambda(u, v) = n-1$, while it carries all $\binom{n}{2}$ edges. It therefore meets the ceiling $\lambda^{\max} \leq m-1$ only once $m \geq n$. For smaller m the extremal graph must be strictly sparser than complete, and pinning down exactly how much sparser is the whole problem. For m strictly larger than n there are no extremal simple graphs since no edges can be added to the complete graph which is extremal for smaller m, justifying the assumption that $m \leq n$.

1.3 Three axes of variation

Mader's theorem is the floor we build on, and everything afterwards is a variation on it. There are three independent ways to change the rules, and they generate the landscape this thesis traverses.

The first axis is the *model*, drawn in Figure 1.3. Two independent choices generate it. The first is *arity*: an edge may join exactly two vertices, or a *hyperedge* may join any $r \geq 2$ of them, a route then passing through it between any two of its members. At $r = 2$ a hyperedge is an ordinary edge, so graphs are the $r = 2$ special case and the genuinely new setting begins at $r \geq 3$. The second is *multiplicity*: a *simple* object carries at most one copy of each edge, while a *multi* object allows parallel copies, recorded by a multiplicity $\mu(u, v)$, so a single pair can already carry several independent routes. Because the two choices are independent they form a small lattice. At the top sits the most general object, the *multi-hypergraph*, and lowering one toggle at a time restricts it downward: forbidding repeats gives the simple hypergraph, dropping the arity to two gives the multigraph, and doing both gives the simple graph at the bottom. Every theorem in this thesis is a statement about one node of this lattice, and the three rows of every comparison figure are its simple-graph, multigraph, and hypergraph levels.



simple graph — one edge per pair

multigraph — parallel edges $\mu(u, v) \geq 1$

hypergraph — hyperedges span $r \geq 3$

multi-hypergraph — a hyperedge repeated

Figure 1.3. The first axis of variation, the *model*, shown for the four object classes this thesis uses. In a *simple graph* each pair of vertices is joined by at most one edge. A *multigraph* permits parallel edges, here a triple and a double, so the multiplicity $\mu(u, v)$ counts how many copies join a pair. A *hypergraph* replaces pairwise edges by hyperedges, each a set of $r \geq 2$ vertices ($r = 2$ gives back ordinary edges, and the depicted examples use $r = 3$), drawn the way a metro map draws a line, threading all of its stations, one colour per hyperedge. Where two hyperedges share a stretch between stations the two lines run side by side, one colour each, exactly as a metro map separates lines along a shared stretch of track. A *multi-hypergraph* lets a hyperedge be repeated, drawn as two parallel lines through the same stations, the hypergraph counterpart of parallel edges. When all hyperedges have the same size r the hypergraph is r-uniform. The same connectivity question, how many independent routes a pair can be forced to share, is asked of all four.

The second axis is the *direction*, a third toggle independent of the other two. Setting it at the top of the model lattice gives the single most general object in the thesis, the *directed multi-hypergraph*, of which every other class is a restriction. Figure 1.4 redraws the four models of Figure 1.3 with arcs in place of edges. An undirected object is the symmetric special case of a directed one, each edge standing for a matched pair of opposite arcs $u \to v$ and $v \to u$, so direction strictly enlarges the family of objects. What it does *not* do is let the directed and undirected problems merge into one. The two opposite arcs offer a round trip the single edge

uv does not, the extremal digraphs are deliberately *lopsided*, with out-degree and in-degree pulled apart in a way no symmetric object can imitate, and the Gomory-Hu tree that settles every undirected case has no directed analogue (Appendix A), so the one-way variants stay distinct in both difficulty and method. A digraph carries three degree counts at each vertex. The *out-degree* $d^+(v)$ counts the arcs leaving v, and the vertices they reach are its *out-neighbours*. The *in-degree* $d^-(v)$ counts the arcs entering v, and the vertices they come from are its *in-neighbours*. The *total degree* $d(v) = d^+(v) + d^-(v)$ adds the two. A vertex with high out-degree fans arcs outward, one with high in-degree gathers them, and keeping the two balanced is the structural idea behind several of the directed constructions to come. The directed measure itself is easiest to meet on one concrete digraph. Figure 1.5 shows a network with three arc-disjoint routes between the two chosen vertices, which equals the smallest arc-cut that separates them, and the degree bound that limits both, which is exactly how Menger reads in a one-way network.

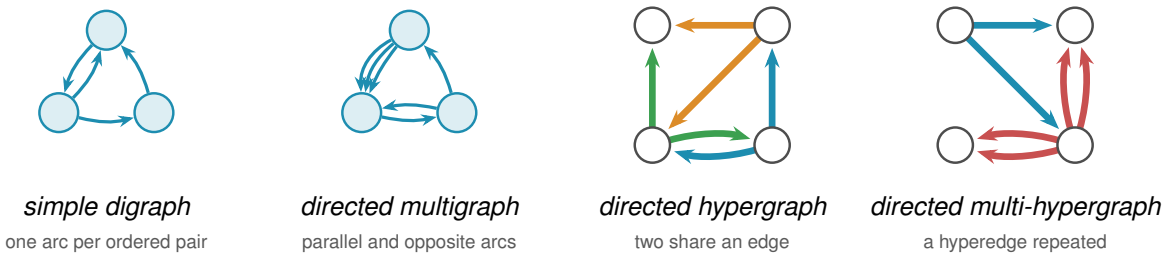


Figure 1.4. The four models of Figure 1.3 redrawn *directed*, the second axis applied to the first. Every edge becomes an arc, and a pair may now carry arcs both ways, the round trip an undirected edge cannot offer. The *simple digraph* still allows at most one arc per ordered pair, so the two-cycle on the top pair is simple. The *directed multigraph* adds parallel arcs, here a triple one way and a bidirected pair. The *directed hypergraph* draws the same three hyperedges as Figure 1.3, in the same colours, each now a metro-styled star, the tail sending one thick coloured arc to each of its heads, the star convention of Figure A.8. Taking each star's tail to be the middle station of the undirected line keeps every arc on the same edge as before. The blue and green hyperedges still share the bottom edge, now one arc running each way, the directed counterpart of two metro lines sharing a stretch of track. The *directed multi-hypergraph* repeats the red hyperedge, so each of its arcs becomes a parallel pair. This last panel is the directed multi-hypergraph at the top of the model lattice, the most general object in the thesis.

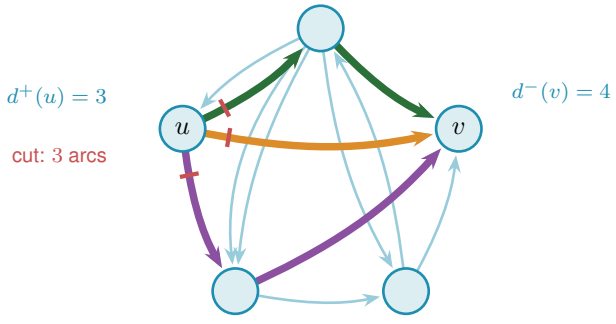


Figure 1.5. A directed version of the K_5 example with multiple arcs. Three arc-disjoint directed routes connect u to v (orange, green, purple). The three red marks cut the arcs leaving u, the first arc of each route, so $\lambda(u, v) = 3$ by Menger's theorem. A directed cut counts only the arcs leaving u's side, so the lone arc entering u from the top is not part of it. The out-degree $d^+(u) = 3$ and in-degree $d^-(v) = 4$ bound the value from above, and $\lambda(u, v) = \min(3, 4) = 3$ meets the smaller one, at u.

The third axis is the *separation*: routes may be required to avoid sharing an edge, as above, or the stricter *internally vertex-disjoint*, sharing no intermediate vertex, which gives the local vertex-connectivity $\kappa_G(u, v)$ and its maximum $\kappa_G^{\max}$. Whitney's inequality $\kappa_G(u, v) \leq \lambda_G(u, v)$ [5] records that avoiding shared vertices is at least as demanding as avoiding shared edges, and Figure 1.6 shows the gap on a graph where two routes are edge-disjoint yet forced through one common vertex.

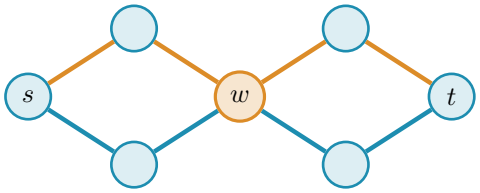


Figure 1.6. The third axis, the *separation*. The two s–t routes drawn here, one orange and one blue, share no edge, so they count as two edge-disjoint paths and $\lambda(s, t) = 2$. Both pass through the single vertex w, however, so they are *not* internally vertex-disjoint, and since every s–t route must cross w we have $\kappa(s, t) = 1$. This is Whitney's inequality $\kappa \leq \lambda$ made concrete: the gap between the two separations is exactly what the vertex-disjoint problem measures, and the rest of the chapter shows it stays shut until $m = 5$.

Three models, two directions, two separations: twelve versions of one question. The thesis is the attempt to answer as many of them as honest mathematics allows, and to build a machine for the rest. Some answers are short and we prove them here. Some answers are long, and their length is the reason Chapter 2 exists. The rest of this chapter collects the variants whose proofs are short enough to belong in the body, so that the reader meets the easy slope before the climb.

1.4 Cases with short proofs

Allowing parallel edges removes the parity subtlety that produces the floor in Mader's theorem, and the answer becomes exactly linear. Write $L_m(n)$ for the multigraph analogue of $\ell_m(n)$.

Theorem 1.3. *For undirected multigraphs on n vertices, $L_m(n) = (m - 1)(n - 1)$.*

Proof. For the lower bound, take a spanning tree of K_n, that is, a connected cycle-free subgraph that touches all n vertices and so has exactly $n - 1$ edges, and give every tree edge multiplicity $m - 1$, meaning $m - 1$ parallel copies. The result has $(m - 1)(n - 1)$ edges. Any two vertices are joined by a unique path in the tree, of some length $k \geq 1$. To separate them one must cut some edge of that path, and the cheapest choice is the lightest tree edge on it, which still carries $m - 1$ copies, so $\lambda(u, v) = m - 1$ for every pair and $\lambda^{\max} = m - 1$. The graph sits right at the ceiling.

For the upper bound, suppose $\lambda_G^{\max} \leq m - 1$ and build a Gomory–Hu tree T of G (Theorem A.2), the same $(n - 1)$-edge summary of all connectivities used for Mader's theorem. Each tree edge e stands for a minimum cut of G, of capacity $c(e) = \lambda_G(u, v) \leq m - 1$ for the pair (u, v) it represents. Now charge each edge of G to a tree edge: an edge $\{x, y\}$ crosses the cut of the lightest tree edge on the tree path from x to y, so it is counted inside that tree edge's capacity. Summing the $n - 1$ capacities therefore counts every edge of G at least once, which already

gives $|E(G)| \le \sum_{e\in T} c(e) \le (m-1)(n-1)$. Unlike Mader's simple-graph count there is no second charge to halve, because parallel edges are now allowed and nothing forces an edge to be counted twice. The tree-of-multiplicity-$(m-1)$ construction above meets this bound exactly, which is why the answer is the clean product $(m-1)(n-1)$ and not a floor. □

The vertex-disjoint question is just as quick at the first nontrivial value. Forbidding two internally vertex-disjoint paths is a strong constraint: it forbids cycles.

Proposition 1.4. *For simple graphs, $k_2(n) = n-1$, attained exactly by trees.*

Proof. If G contains a cycle, then any two vertices on that cycle are joined by the two paths of the cycle, which are internally vertex-disjoint, so $\kappa_G^{\max} \ge 2$. Hence $\kappa_G^{\max} \le 1$ forces G to be acyclic, and an acyclic graph on n vertices has at most $n-1$ edges, with equality for trees. A tree indeed has $\kappa^{\max} = 1$, since removing the single intermediate vertex on any path disconnects its endpoints. □

Here $k_m(n)$ denotes the vertex-disjoint extremal function, the counterpart of $\ell_m(n)$ for $\kappa^{\max}$. We have just seen $k_2(n) = n-1 = \ell_2(n)$. The agreement of the edge and vertex problems is no accident at small m, and the moment it breaks is the first place the proofs begin to strain.

1.5 The first divergence: vertex-disjoint paths

Replacing edge-disjoint by internally vertex-disjoint routes changes nothing at first. Leonard proved that the edge and vertex problems give the same answer while m stays small.

Theorem 1.5 (Leonard [6])**.** *For simple graphs and $m \le 4$,*

$$k_m(n) = \ell_m(n) = \left\lfloor \frac{m(n-1)}{2} \right\rfloor.$$

For $m \le 4$ the extremal graphs for the two problems coincide: Whitney's inequality $\kappa \le \lambda$ is tight where it matters and the harder vertex constraint costs nothing (Figure 1.8, the three overlapping blue curves). One is tempted to expect this forever. It fails at the very next value, and the failure is sharp.

Theorem 1.6 (Sørensen-Thomassen [7])**.** *For simple graphs and all $n \ge 10$,*

$$k_5(n) = \left\lfloor \frac{8n}{3} \right\rfloor - 3 \;>\; \left\lfloor \frac{5(n-1)}{2} \right\rfloor = \ell_5(n).$$

The restriction $n \ge 10$ is the range over which Sørensen and Thomassen certify that $k_5(n)$ equals the formula. The formula itself already exceeds $\ell_5(n)$ for $n = 6$ (by one), for $n = 8$ (by one), and for all $n \ge 9$: at $n = 10$ the vertex formula gives 23 and the edge formula gives 22. At $n = 7$ both formulas coincidentally tie at 15. For $n \le 9$ the true $k_5(n)$ must be determined by

structural analysis rather than the formula, which is why the theorem's stated domain begins at $n = 10$.

At $m = 5$ the vertex problem genuinely diverges from the edge problem, visible in Figure 1.8 as the shaded gap that opens between the dashed blue and solid orange curves. The reason is structural. Leonard's method leans on the extremal graphs being assembled from cliques glued along shared cliques. A *k-tree* is built up one vertex at a time: start from a complete graph on $k + 1$ vertices, then again and again add a single new vertex and join it to all k vertices of some clique of size k that is already there. Figure 1.7 grows a 2-tree, hanging each new vertex on an edge already present. These chordal scaffolds are exactly the building blocks the method uses. Through $m = 4$ those scaffolds are forced, and counting their edges gives the bound.

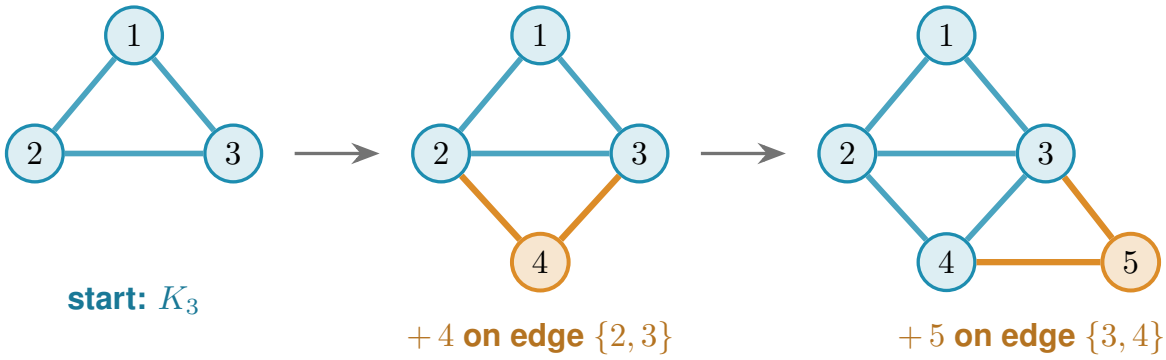


Figure 1.7. A 2-tree grown one vertex at a time, read left to right. The blue triangle on $\{1, 2, 3\}$ is the starting K_3. Vertex 4 is then joined to the two ends of the edge $\{2, 3\}$, and vertex 5 to the two ends of the edge $\{3, 4\}$ (the new vertex and its two orange edges at each step), each new vertex hung on an edge already present. A general k-tree follows the same recipe with K_{k+1} in place of the triangle and a k-clique in place of the edge. These are the chordal scaffolds Leonard's count rests on.

At $m = 5$ they are no longer the unique extremal shape. Additional configurations appear, and the clean count breaks. Sørensen and Thomassen recover the exact value by a delicate structural induction, which we cite rather than reproduce. For $m \geq 6$ even that much is unknown: the vertex-disjoint problem has stood open since 1974, and the extremal structure is genuinely not understood. It is the one direction this thesis does not attempt to force, and Chapter 4 explains why.

1.6 The directed case and the quadratic phenomenon

Direction breaks a symmetry that undirected graphs never had. In an undirected graph the number of edges is tied to the connectivity through every vertex's degree. In a digraph a vertex can have many arcs in and none out, and a whole graph can lean entirely one way. Send every arc from one half of the vertices to the other and nothing comes back: between any two vertices there is at most one route, yet the number of arcs is quadratic.

Construction 1.7 (One-directional bipartite digraph)**.** Split V into parts A and B with $|A| = \lfloor n/2 \rfloor$ and $|B| = \lceil n/2 \rceil$, and include every arc from A to B. All arcs run from A to B, and none within

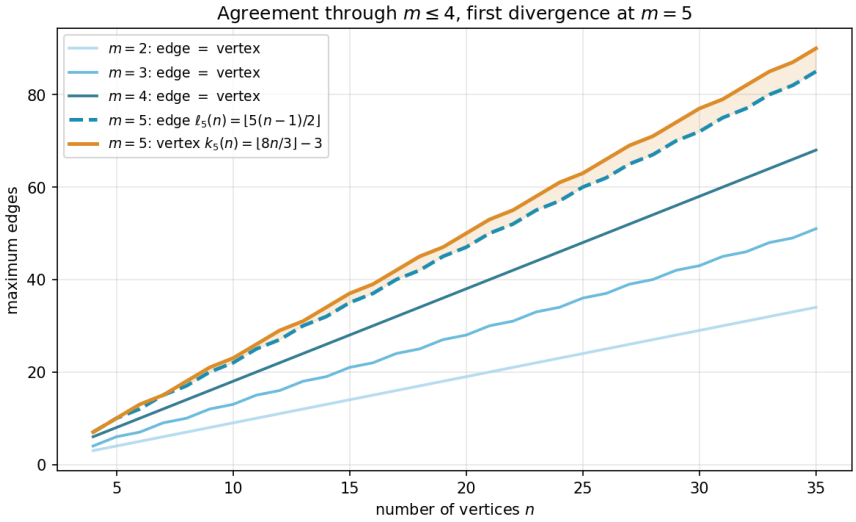


Figure 1.8. The edge and vertex extremal functions for $m = 2, 3, 4$ (blue shades, single curve per m since they agree) and $m = 5$ (dashed blue for edge, solid orange for vertex). Through $m = 4$ the two problems give identical answers, and at $m = 5$ the vertex formula $k_5(n) = \lfloor 8n/3 \rfloor - 3$ separates above the edge formula $\ell_5(n) = \lfloor 5(n-1)/2 \rfloor$, with the gap shaded.

A, within B, or from B back to A. Since B has no outgoing arcs, a vertex in A cannot reach any other vertex of A, and two vertices of B cannot communicate at all. For any $a \in A$ and $b \in B$ the only directed path from a to b is the single arc $a \to b$, so $\lambda(a, b) = 1$ and $\lambda^{\max} = 1$. The arc count is $|A| \cdot |B| = \lfloor n^2/4 \rfloor$.

Figure 1.9 draws the construction. Every arrowhead lands in B and none leaves it, so the picture is a wall of one-way traffic: from any a on the left there is exactly one way to reach any b on the right, the single arc between them, and there is no way at all to travel between two vertices on the same side. The arc count $|A| \cdot |B|$ is quadratic, yet no pair carries more than one route. This is the lopsidedness direction buys, and it has no undirected analogue. It competes with a second construction built around a single centre.

That second construction is the directed hub. It keeps each vertex's in-degree and out-degree balanced, the directed measure already met in Figure 1.5, and spends its arcs on a central vertex rather than a one-way wall.

Construction 1.8 (Directed hub)**.** For $n \ge m \ge 2$, take a hub vertex h and label the remaining $N = n - 1$ vertices $0, 1, \ldots, N - 1$. Add the $2N$ hub arcs $h \to i$ and $i \to h$ for every i, and on the non-hub vertices add the circulant arcs $i \to i + j \pmod N$ for $j = 1, \ldots, m - 2$. Counting these gives $2N$ hub arcs (each of the N spokes joined to h in both directions) plus $(m - 2)N$ circulant arcs ($m - 2$ layers, one arc per spoke), so the total is $2N + (m - 2)N = mN = m(n - 1)$ arcs. Every non-hub vertex has $d^+ = d^- = 1 + (m - 2) = m - 1$, and any ordered pair of vertices contains a non-hub endpoint, so the directed degree bound (Lemma A.3, which says $\lambda(u, v) \le \min(d^+(u), d^-(v))$, a route out of u must use one of u's out-arcs and one of v's in-arcs) gives $\lambda^{\max} \le m - 1$.

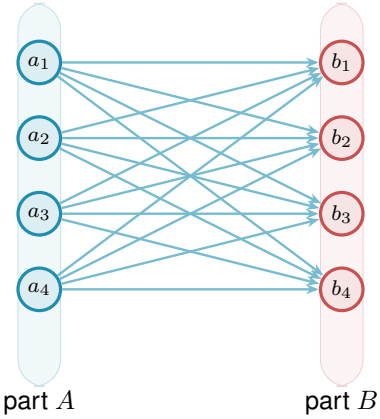


Figure 1.9. The one-directional complete bipartite digraph $B_{k,k}$ of Construction 1.7, drawn here at $n = 8$ with $|A| = |B| = 4$. Every one of the $|A| \cdot |B| = \lfloor n^2/4 \rfloor$ arcs runs left to right, from A to B. There are no arcs inside A, inside B, or back from B to A. Because B has no outgoing arcs and A no incoming ones, the only route from any a_i to any b_j is the single arc $a_i \to b_j$, so every pair has local edge-connectivity 1 while the arc count grows quadratically.

At $m = 2$ this construction is easiest to picture: the circulant layer is empty and what remains is the *bidirected star* of Figure 1.10, a hub joined to each spoke by a matched pair of opposite arcs. Every spoke can reach every other only by going in to the hub and back out, two hops, so no pair is ever joined by two independent routes, and the arc total is exactly $2(n-1)$. For $m \geq 3$ not all $m(n-1)$ arcs can touch the hub, since a simple digraph has at most $2(n-1)$ arcs incident to one vertex: the surplus lives in the circulant layers among the spokes, tuned so that no degree exceeds the budget. The two constructions compete on different scales, linear against quadratic, and which one wins depends on n. At $m = 2$ the competition resolves exactly.

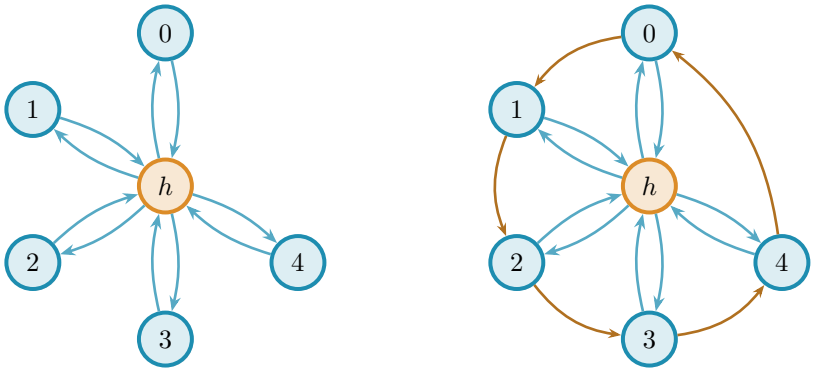


Figure 1.10. The directed hub at $n = 6$ for two values of m. *Left ($m = 2$):* the bidirected star. The hub h is joined to each spoke by one arc in each direction, giving $2(n-1) = 10$ arcs. No circulant layer is added, so all spoke communication routes through h, so $\lambda^{\max} = 1$. *Right ($m = 3$):* a single circulant layer is added among the five spokes. The blue arcs are the same hub-spoke pairs as on the left. The orange arcs are that one circulant layer, the cycle $0 \to 1 \to 2 \to 3 \to 4 \to 0$, so each spoke has $d^+ = d^- = m - 1 = 2$. The total is $m(n-1) = 15$ arcs and $\lambda^{\max} = 2$. For $m \geq 3$ the spokes are no longer isolated from one another, and tracing all the routes becomes the argument that fills the rest of the chapter.

Theorem 1.9 (Directed arc value at $m = 2$)**.** *For simple digraphs,*

$$\ell_2^{\mathrm{dir}}(n) = \max\left(2(n-1),\ \left\lfloor \frac{n^2}{4} \right\rfloor\right).$$

The hub construction leads for $n < 7$, the one-directional bipartite construction leads for $n > 7$, and the two tie at $n = 7$, where both reach 12 arcs.

The lower bound is the better of the two constructions. The upper bound is where the trouble starts.

Idea of the upper bound at $m = 2$. The value $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$ is a contest between a linear term and a quadratic one, and which leads is pure arithmetic: $2(n-1) \geq \lfloor n^2/4 \rfloor$ precisely for $n \leq 7$, with equality at $n = 7$, where both give 12. So M follows the bidirected star up to the crossover and the bipartite digraph past it, and $n = 7$ is the single value where the two constructions tie. The proof that no digraph beats $M(n)$ is an induction on n (Appendix A). Delete a vertex of *minimum* total degree: since the degrees sum to twice the arc count, an averaging argument bounds that degree by $2a/n$, where a is the number of arcs, and the remaining digraph on $n-1$ vertices still respects the ceiling, so the inductive bound applies to it. Feeding the two facts together gives $a \leq \frac{n}{n-2} \lfloor (n-1)^2/4 \rfloor$, and a short parity check shows the right-hand side never exceeds $\lfloor n^2/4 \rfloor$ once $n \geq 8$: for even n it lands exactly on the bipartite value, and for odd n a sub-unit slack is swallowed by the fact that a is an integer. The averaging closes the step with no separate hub case, which is why the difficulty is not depth but bulk. Nothing in the argument is deep. It is a careful walk through two parity regimes, with the base cases $n \leq 7$ settled by exhaustive computer search rather than by hand. That experience of a correct but long and almost entirely mechanical case-check is exactly what motivates the chapters that follow. One genuinely wants to hand the case-checking to a machine.

The vertex-disjoint version of the same question needs no separate machinery at $m = 2$. Vertex-disjoint routes are at most as numerous as arc-disjoint ones, and the two constructions above keep $\kappa^{\max} = 1$ as readily as $\lambda^{\max} = 1$, so the value transfers exactly.

Theorem 1.10 (Directed vertex value at $m = 2$)**.** *For simple digraphs,*

$$k_2^{\mathrm{dir}}(n) = \max\left(2(n-1),\ \left\lfloor \frac{n^2}{4} \right\rfloor\right) = \ell_2^{\mathrm{dir}}(n).$$

1.7 The failure of direct proof for $m \geq 3$

For $m \geq 3$ the induction no longer closes, and the natural guess is wrong. The natural initial conjecture was that the two constructions above continued to be the whole story, giving $\max(m(n-1), \lfloor n^2/4 \rfloor)$. A single graph on ten vertices refutes it.

Remark 1.11 (A thirty-arc counterexample). Take $A = \{a_1, \ldots, a_5\}$ and $B = \{b_1, \ldots, b_5\}$ with all 25 arcs from A to B, and add a directed five-cycle $b_1 \to b_2 \to b_3 \to b_4 \to b_5 \to b_1$ inside B.

This digraph has 30 arcs. Every vertex of B has exactly one extra in-arc from inside B, so two arc-disjoint routes from a_i to b_j exist: the direct arc $a_i \to b_j$, and the detour $a_i \to b_{j-1} \to b_j$ using the 5-cycle predecessor of b_j (indices mod 5). For the matching upper bound, the only in-arcs of b_j reachable on a path from a_i are the direct arc $a_i \to b_j$ and the cycle arc $b_{j-1} \to b_j$. Removing both disconnects a_i from b_j, while neither alone does, so the minimum cut has size 2. Hence $\lambda(a_i, b_j) = 2$. Routes between two vertices of B never leave B and give $\lambda(b_i, b_j) = 1$. Hence $\lambda^{\max} = 2$, so the graph is feasible at $m = 3$, yet $30 > \max(3 \cdot 9,\ 25) = 27$.

Figure 1.11 draws this thirty-arc digraph and traces the mechanism. The twenty-five faint arcs are the complete one-directional bipartite layer from A to B, the construction we already understand. On top of them the orange five-cycle gives each vertex of B a single extra in-arc from inside B. That one extra arc is exactly what doubles the connectivity. For the marked pair a_1, b_2 the two heavy routes show how: the blue arc is the direct hop $a_1 \to b_2$, and the green path $a_1 \to b_1 \to b_2$ borrows the cycle arc $b_1 \to b_2$ for its second step. The two share no arc, so $\lambda(a_1, b_2) = 2$, and the same pair of routes exists for every a_i, b_j. Five extra arcs lift the whole digraph from one independent route per pair to two, and the thirty arcs overshoot the old prediction of twenty-seven.

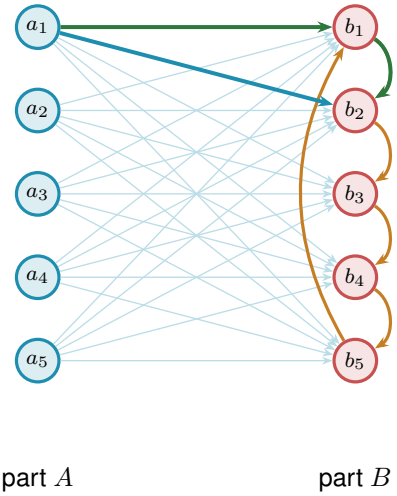


Figure 1.11. The thirty-arc counterexample of Remark 1.11: the augmented bipartite digraph (Construction 1.12) at $m = 3$, $n = 10$, $|A| = |B| = 5$. The 25 faint arcs are the complete one-directional layer from A to B, and the 5 orange arcs form the directed cycle $b_1 \to b_2 \to b_3 \to b_4 \to b_5 \to b_1$ inside B, one extra in-arc per vertex of B. The heavy blue arc and the heavy green detour show the pair a_1, b_2 carrying 2 arc-disjoint routes: the direct arc $a_1 \to b_2$ (blue) and the two-step detour $a_1 \to b_1 \to b_2$ (green) through the cycle predecessor b_1. Hence $\lambda^{\max} = 2$, feasible at $m = 3$, while $30 > \max(3 \cdot 9,\ 25) = 27$.

The counterexample generalises: the five-cycle gave every vertex of B exactly one extra in-arc from inside B, and the budget for such arcs is $m - 2$ per vertex, not one. Figure 1.11 is the instance $m = 3$, where that budget is one and a single cycle suffices. For larger m each b_j collects $m - 2$ cyclic predecessors and the same picture acquires more concentric chords inside B.

Construction 1.12 (Augmented bipartite digraph)**.** For $m \geq 2$ and $n \geq m$, set $|B| =$

$\lceil (n+m-2)/2 \rceil$ and $|A| = n - |B| = \lfloor (n-m+2)/2 \rfloor$, include every arc from A to B, and inside B give each vertex b_j the $m-2$ in-arcs from its cyclic predecessors $b_{j-1}, \ldots, b_{j-(m-2)}$ (indices mod $|B|$). The total is $\lfloor (n+m-2)^2/4 \rfloor$ arcs. For the connectivity, fix $a \in A$ and $b \in B$: the direct arc and the $m-2$ two-step detours $a \to b' \to b$ through the in-neighbours b' of b inside B are $m-1$ arc-disjoint routes, and they are all there are, because the only in-arcs of b reachable from a are exactly those $m-1$ arcs, which therefore form a cut. Pairs inside B are limited by the in-degree $m-2$, pairs inside A and pairs from B to A have no routes at all, so $\lambda^{\max} = m-1$. At $m = 2$ the inside of B is empty and the construction reduces to Construction 1.7, with the balanced partition $(\lfloor n/2 \rfloor, \lceil n/2 \rceil)$. At $m = 3$ the formula gives $|B| = \lceil (n+1)/2 \rceil$, which coincides with $\lceil n/2 \rceil$ at odd n but gives $|B| = n/2 + 1$ at even n. Both the balanced and the shifted partition achieve the same arc count $\lfloor n^2/4 \rfloor + \lceil n/2 \rceil$ in this case, so the two are interchangeable at $m = 3$. The shift to a strictly larger B is structurally significant only at $m \geq 4$, where the balanced partition is suboptimal. At $m = 3$, $n = 10$ it is the thirty-arc counterexample of Remark 1.11, drawn in Figure 1.11 (using the equivalent balanced partition $|A| = |B| = 5$).

The two rival shapes invite a political picture. The hub is a single king through whom all traffic must pass, cheap to build but limited by the one throne. The bipartite family is a flat cabinet of ministers, each handling its own dossier with no central seat, and it scales quadratically. Which arrangement packs more arcs under the ceiling depends on n, and for $m \geq 3$ the cabinet overtakes the king once n is large. This is the corrected shape, and it leads to the standing conjecture for the directed arc problem.

Conjecture 1.13. *For simple digraphs and $m \geq 2$,*

$$\ell_m^{\mathrm{dir}}(n) = \max\left(m(n-1),\ \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor \right).$$

For $m = 2$ this is Theorem 1.9. For $m = 3$ the formula $\lfloor (n+1)^2/4 \rfloor$ equals $\lfloor n^2/4 \rfloor + \lceil n/2 \rceil$, so the conjecture reduces to the previous formula for $m = 2$ and $m = 3$. For $m = 3$, $n = 10$ the second branch gives $\lfloor 121/4 \rfloor = 30$, matching Remark 1.11. For $m \geq 4$ the new formula and the old balanced-partition count $\lfloor n^2/4 \rfloor + (m-2)\lceil n/2 \rceil$ no longer agree: the quadratic branches already differ (for $m = 4$ the new one is larger by exactly one at every even n), but the hub term $m(n-1)$ dominates both at small n, so the full conjectured value first overtakes the old one at $n = 12$ for $m = 4$. The construction of Construction 1.12 with the shifted partition witnesses the difference. The lower bound holds for all m, witnessed by Construction 1.8 on the first branch and Construction 1.12 on the second. The upper bound is open, and Chapter 4 returns to it with what structural progress the machine and the hand can jointly supply.

1.8 The hypergraph variant

The third model deserves its own short treatment, not because its base case is hard but because it is the one variant whose objects are stored and manipulated differently from all the rest, and

it is worth meeting that difference now rather than at the moment of computation. A hypergraph keeps no multiplicity matrix, only the list of its hyperedges, each a set of r vertices. We fix the edge size r, following the extremal literature, for a structural reason: under a connectivity ceiling a small edge is always at least as efficient per route as a large one, so a hypergraph free to mix sizes would maximise its edge count by using pairs only, and the question would collapse back to the graph case. Fixing r is what makes the hypergraph question a new question. The route between two vertices is a *Berge path*, which alternates vertices and hyperedges so that each consecutive pair of vertices lies in the hyperedge listed between them, and two such routes are edge-disjoint when they share no hyperedge. Concretely, in a three-uniform hypergraph a route from u to w might be the single step $u, \{u, x, w\}, w$ that crosses one hyperedge, or a longer alternation $u, \{u, x, y\}, y, \{y, z, w\}, w$ that switches hyperedge at the intermediate vertex y. With routes redefined this way the same Gomory–Hu machinery that proved the multigraph value carries over, and the base case has the same linear shape. The picture to keep in mind is a metro map: each hyperedge is a line, and a route from one vertex to another rides a line and changes to another line at a station the two share.

A single hyperedge may serve only one route, so two Berge routes that share no hyperedge are edge-disjoint, the hypergraph counterpart of two edge-disjoint paths in a graph. Reading routes off a metro map is a matter of seeing which lines share track: two hyperedges that share several vertices run side by side along the stretch between them, as in Figure 1.3, and two routes are hyperedge-disjoint exactly when the lines they ride share no track at all.

Proposition 1.14 (Hypergraph edge bound)**.** *An r-uniform hypergraph on n vertices with Berge edge-connectivity at most $m-1$ has at most $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ hyperedges. A simple hypergraph attains the bound whenever $m-1 \le \binom{n-2}{r-2}$, and a multihypergraph attains it whenever $(r-1) \mid n-1$.*

The proof, given in Appendix A, runs the multigraph argument through the hypergraph Gomory–Hu theorem of Bérczi, Chandrasekaran, Király and Kulkarni [8], charging each hyperedge to the at least $r-1$ tree cuts it crosses. The star hypertree (Figure 1.12) attains the $m = 2$ case, a hub joined to disjoint blocks of $r-1$ vertices, with Berge connectivity one everywhere and $\lfloor (n-1)/(r-1) \rfloor$ hyperedges, exactly as a tree attained the multigraph bound. One genuine surprise is worth recording here: for $r \ge 3$ the bound is attained by *simple* hypergraphs, with no repeated hyperedge (Theorem A.33). In the graph case simplicity is exactly what separates Mader's $\lfloor m(n-1)/2 \rfloor$ from the multigraph value $(m-1)(n-1)$. One edge size higher, it costs nothing.

The proposition counts hyperedges, but it does not say how to *measure* the Berge connectivity of a hypergraph we are handed, and that measurement is the genuinely new part. The metro picture shows why. An ordinary edge is a one-stop link between a single pair, so policing edge-disjointness means watching each pair on its own. A hyperedge is a whole line: it stops at all r of its vertices and so touches $\binom{r}{2}$ pairs at once, yet like a line that can run only one train at a time it may serve only one route. When several routes want to board the same line at different stations,

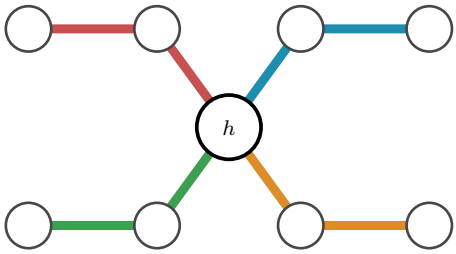


Figure 1.12. The star hypertree that attains the $m = 2$ hypergraph bound, here $r = 3$ and $n = 9$, drawn as a metro map with the hub h centred between two rows of stations. The single hub station lies on every line, and each of the $\lfloor (n-1)/(r-1) \rfloor = 4$ hyperedges is one metro line of $r = 3$ stations: it leaves the hub diagonally to an inner station and then runs horizontally to its outer station, hanging a fresh block of $r-1 = 2$ vertices off the hub. Because every line meets every other only at h, no pair has two hyperedge-disjoint routes, so the Berge connectivity is one everywhere, exactly as a spanning tree attained the multigraph bound.

the checker must wave exactly one through and turn the rest away, and it must do this for every line simultaneously. That bookkeeping is the one new piece of machinery the next chapter builds, and it is why the hypergraph rejoins the family only once the checker is in place.

1.9 Random graphs as intuition

The short proofs above hinted that the edge and vertex problems travel together for a while, agreeing through $m = 4$ before the split at $m = 5$. That separation is a statement about extremal graphs, the densest ones the ceiling allows, and it is worth asking whether the same split can be seen in ordinary graphs drawn at random, with no extremal tuning at all. The random model is the natural place to build intuition, because it shows the qualitative behaviour of the connectivity notions before any construction is pushed to its limit, and we will return to it whenever a picture of the typical graph clarifies a result. It is not a detour from the program to come, either: a sampled $G(n, p)$ is an ordinary graph, measured by the very same connectivity checker the next chapter builds for every other variant, so the random model is simply the one pipeline turned to *observe* the typical case rather than to prove or to discover the extremal one.

Each of the $\binom{n}{2}$ possible edges of $G(n, p)$ is included independently with probability p, and on the resulting graph we read off both binding connectivities, the edge version $\lambda_G^{\max}$ and the vertex version $\kappa_G^{\max}$, from the very same sample. Whitney's inequality $\kappa_G^{\max} \le \lambda_G^{\max}$ guarantees the vertex value never exceeds the edge one, but it is silent on how often the two actually come apart. Figure 1.13 looks directly at the distributions, fixing $n = 16$ and histogramming both connectivities over many samples at three densities. Two things are visible at once. As the density rises the whole distribution marches rightward, from a binding connectivity around five at $p = \frac{1}{4}$ to around thirteen at $p = \frac{3}{4}$, so by $n = 16$ the typical graph already sits well past the $m = 4$ at which the proofs agreed. And comparing the edge row to the vertex row, the vertex distribution sits at or just to the left of the edge one, never to its right, which is Whitney's inequality seen across a whole distribution rather than a single graph. The two pull apart on a fraction of samples that shrinks as the density grows, from about a fifth of the samples at $p = \frac{1}{4}$ to only a

handful by $p = \frac{3}{4}$. The reason is structural. For $\kappa^{\max}$ to fall below $\lambda^{\max}$ some pair must carry more edge-disjoint routes than internally vertex-disjoint ones, which happens only when several of those routes are forced through one shared intermediate vertex. In a sparse graph routes are scarce and a single low-degree vertex can be exactly such a bottleneck, but as p grows every vertex gains degree and the graph offers many alternative intermediate vertices, so independent routes almost never need to reuse one. In the dense limit the graph approaches the complete graph, where $\kappa^{\max} = \lambda^{\max} = n - 1$ for every pair and the gap closes entirely. The divergence the thesis records at $m = 5$ is therefore not an artefact of the extremal construction. It is already there, in miniature, in the random model.

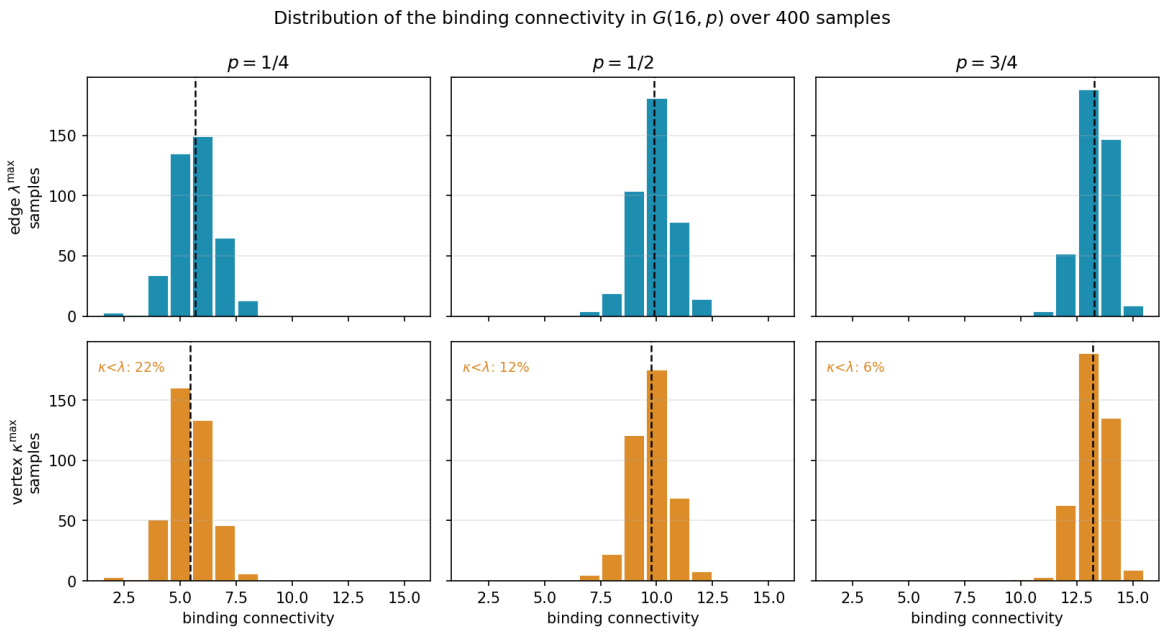


Figure 1.13. Distributions of the binding connectivity in $G(16, p)$ over 400 samples, at densities $p = \frac{1}{4}, \frac{1}{2}, \frac{3}{4}$. There are $2^{\binom{16}{2}} = 2^{120}$ labelled graphs on 16 vertices, far too many to enumerate, so these are Monte Carlo samples rather than the full population, and 400 is plenty to show the shape of each distribution. The top row histograms the edge-connectivity $\lambda^{\max}$, the bottom row the vertex-connectivity $\kappa^{\max}$, measured on the same samples, with the mean marked by a dashed line and the fraction of samples on which $\kappa^{\max} < \lambda^{\max}$ printed on each vertex panel. Reading a column top to bottom, the vertex distribution sits at or to the left of the edge one, which is Whitney's inequality, and the gap is largest at the sparsest density (22% of samples) and closes as p grows (6%). All six panels share one axis range.

The random model also supplies a landmark of a different flavour. Below a sharp density the forbidden configuration is almost surely absent, and above it almost surely present, and the location of that threshold is the same for every one of the connectivity notions above.

Theorem 1.15 (Appearance threshold). *Let $m = m(n) \geq 2$ grow with n so that $m/\ln n \to \infty$ and $m = o(n)$, and let $c > 0$ be a constant. In the binomial random graph $G(n, p)$ with $p = c \cdot m/n$,*

$$\lim_{n\to\infty} \Pr\big[\lambda^{\max} \geq m\big] = \begin{cases} 1 & \text{if } c > 1, \\ 0 & \text{if } c < 1, \end{cases}$$

so the threshold for the appearance of a pair with local edge-connectivity at least m is $p^* = m/n$. The same threshold governs local vertex-connectivity and the directed analogues (Remark A.46).

The proof is a concentration argument with two halves (Appendix A). Above the threshold the edge count concentrates past Mader's bound, so Theorem 1.2 itself forces a high pair into existence. Below it the maximum degree stays under m, and the degree bound caps every local connectivity. The two hypotheses earn their keep: $m/\ln n \to \infty$ is what lets a union bound over the n vertices close, and $m = o(n)$ keeps the density $p = cm/n$ below one and in the sparse regime where the threshold lives. One caveat on scope: the upper half quoted here uses Mader's bound, which is the simple undirected edge case, so it is that variant the proof settles directly. The vertex-disjoint and directed analogues sit at the same threshold $p^* = m/n$ but reach it through their own extremal bounds rather than Mader's, possibly with a different leading constant, as Remark A.46 records. The collapse of complexity is what deserves emphasis: in the probabilistic limit the distinctions between edges and vertices, directions and models all wash out, and the single number m/n decides the matter, a universality the synthesis of Chapter 4 records again.

One limitation must be stated plainly, lest the threshold be read as evidence about the problem this thesis actually studies. The growth hypothesis $m/\ln n \to \infty$ forces m to climb with n, so the theorem says nothing whatever about a fixed small m, the regime of every exact value above, where m is 2, 3, or 4 and n runs to infinity around it. There the union bound is vacuous and the appearance of a dense pair is governed by rare low-degree configurations that need finer, Poisson-type tools. The random model therefore enters as contrast and not as evidence: it shows how cleanly the question collapses in one limit, which throws into relief how stubbornly it resists in the fixed-m limit the rest of the thesis must attack by hand and by machine.

1.10 Where this work sits

It is worth pausing, before the machine, to place the question and the method against the two traditions they descend from: the extremal graph theory that posed the problem and the computational mathematics that will help settle it.

The mathematical lineage is the older of the two. The problem belongs to the family of extremal connectivity questions opened by Bollobás and Erdős, who asked how dense a graph can be while every pair of vertices stays below a fixed connectivity [1], logged today as Erdős Problem 915 [3]. Mader's theorem [4] closed the simple undirected edge case with the clean floor $\lfloor m(n-1)/2 \rfloor$ recalled in Theorem 1.2, and it is the floor every variant in this thesis is measured against. The vertex-disjoint branch has a longer and rougher history: Leonard [6] showed the edge and vertex problems coincide through $m \leq 4$, and Sørensen and Thomassen [7] found the first divergence at $m = 5$ with $k_5(n) = \lfloor 8n/3 \rfloor - 3$, after which the exact values $k_m(n)$ remain unknown. That gap between a settled edge case and a stalled vertex case is exactly the terrain Chapter 1 has been mapping, and the directed and hypergraph axes extend it further. The thesis adds to this lineage not a single new closed form but a uniform way of pushing each axis as far

as exact computation allows.

The computational tradition this thesis's pipeline belongs to is the practice of settling combinatorial questions by machine in a way a human can audit, and it splits into a few schools worth contrasting with the certify-then-prove loop built next. The oldest is *exhaustive generation*: enumerate every object of a given size up to isomorphism and check each one. McKay and Piperno's `nauty`/`geng` toolkit [9] is the standard engine for this, and its flagship results include exact small Ramsey numbers such as $R(4,5) = 25$ [10]. The enumeration of Chapter 2 is the same idea, run on the twelve variants of Problem 915. The second school is *SAT-assisted proof*, in which a question is encoded in propositional logic and a solver emits a machine-checkable certificate: Heule, Kullmann and Marek's resolution of the Boolean Pythagorean triples problem [11], whose verified proof runs to some two hundred terabytes, and Heule's determination of Schur number five [12] are the canonical examples of certified solver output at scale. The third is *flag algebras*, Razborov's framework [13] that converts asymptotic extremal bounds into semidefinite-programming certificates, the dominant tool for density problems in the large-n limit. Closest of all to the method here is the fourth, *LP/ILP-based certification*, where a feasibility or optimality argument over the rationals or integers is exhibited as a linear program whose optimal dual is itself the proof. This is precisely the form the cut-counting certifier of Chapter 2 takes.

Against that backdrop, what is distinctive here is consolidation rather than a new solver. A single checker measures connectivity exactly across all twelve problem variants on one data model, so search and certification share one codebase rather than living in separate tools. The certificates it emits are exact rational or integral objects, not floating-point estimates, so an upper bound it reports is a proof and not evidence. And because the same program both hunts for dense graphs and certifies the bounds they suggest, the discover step and the prove step are two modes of one loop, which is the arrangement the next chapter builds. What makes the consolidation honest is that it never blurs three kinds of knowledge: a value can be *proved* (by hand, or by exhausting every graph of a size), *certified* (by the same optimisation, reaching a vertex or two further), or only *conjectured* (witnessed by a dense graph the search exhibits, with no matching upper bound). This trichotomy, named in Section 3.6 and obeyed by the summary of Chapter 4, is the through-line of everything that follows.

Computer note

We are now stuck in three different ways: a fifty-year-old open problem at $m \geq 6$ for vertices, a long proof we would rather not check by hand at $m = 2$ for arcs, and a conjecture with no proof at all at $m \geq 3$. The base cases of the long induction were verified by exhaustive search, and the thirty-arc counterexample was found the same way. The next chapter builds a single program that models all of these cases at once, measures their connectivity exactly, and proves the small-case upper bounds outright.

Chapter 2

Certifying Bounds by Machine

The last chapter ended in three different kinds of stuckness, and all three share a shape. The objects are graphs of one flavour or another, the constraint is always a ceiling on connectivity, and the work is always either measuring a given graph or checking many small ones. That shared shape is what a single program can exploit. This chapter builds it, and then uses it for one purpose only: to *prove*. It gives every variant one representation, measures connectivity exactly through a max-flow checker, checks small cases by honest enumeration, and turns the finite checks the proofs of Chapter 1 relied on into a single auditable certifier. The hunt for dense examples, which proves nothing on its own, waits until Chapter 3.

A word on what the program is for. It is a microscope, not a source of final answers. It measures connectivity exactly, so what it reports about a given graph is a fact. It enumerates graphs of a fixed size, so for that size it can prove a theorem. It will later discover dense graphs, which are only lower bounds, but not here. Each of these has a different epistemic weight, and the program is built so that the three never get confused. Figure 2.1 draws this spine: a single *model* holds every variant, one exact *measure* reads connectivity off it, and from that measurement a *prove* step bounds a fixed size from above, with the *discover* step of the next chapter and a final *observe* step for the random model added later. Every routine introduced from here on is shown as a small card carrying its name, its inputs and output, a one-line account of how it works, and a coloured badge naming its place in the spine. The function bodies are not reproduced, because the point is the interface, not the code.

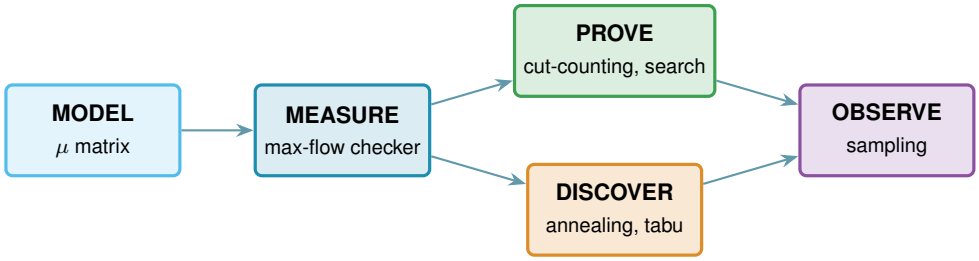


Figure 2.1. The architecture as one pipeline. A single data model, the multiplicity matrix μ, feeds an exact measurement, the max-flow checker. That measurement feeds the two bound-producing roles: a prover for upper bounds (the cut-counting optimisation and the exhaustive search), built in this chapter, and a discoverer for lower bounds (simulated annealing, or tabu search), built in Chapter 3. A final sampling role studies the random model. The badge colours on the code cards below match the boxes here.

2.1 The solver

Everything in this chapter is a single function, `solve`, and it helps to hold its most basic version in mind from the start, because every later idea is a refinement of it and nothing is ever a separate program for a separate case. Given n and m, the basic strategy is as plain as it sounds. Walk through every graph on n vertices, measure the largest local connectivity $\lambda^{\max}$ of each with a max-flow computation, and keep the densest graph that stays at or below the ceiling $m - 1$. Because it has looked at every graph, the count it returns is the exact answer, not an estimate.

That one loop already settles the smallest cases, and the rest of the chapter is built on top of it without ever leaving `solve`. We first make the measurement step trustworthy and quick, via the connectivity checker and a Gomory–Hu shortcut for undirected graphs. We then make the enumeration step skip whole families of hopeless graphs, via a pruned search. Finally, for the directed multigraph case, we replace enumeration altogether with one small optimisation that proves every m at once, which is the certifier. Each of these is the same `solve` doing less work for the same guarantee, and the choice among them is made automatically from the booleans it receives.

`solve(n, m, directed, simple, separation, exhaustive, max_seconds)` **DRIVER**

in the variant booleans `directed` and `simple` (or `hypergraph` with uniformity `r`), the `separation` (edge or vertex), n, m, the `method`, and a time budget `max_seconds`

out a `SolveResult` carrying a value with an honest label: exact, lower bound, or upper bound

how picks the method the case allows: brute-force enumeration, the pruned search, or the cut-counting certifier when proving, or the temperature search or tabu search when discovering (Chapter 3). It always respects the budget, and it is the one function the rest of the thesis ever calls.

2.2 A single representation for all variants

Two of the three axes from Chapter 1, the model and the direction, are properties of the object, and a single data structure holds them all. Of the three structural models, simple graphs and multigraphs fit the matrix directly, while the hypergraph keeps the separate storage promised in Chapter 1 and rejoins at the end of this chapter. The hypergraph could in principle be pressed into the same shape as a higher-order tensor, an r-dimensional array marking which r-sets are present, but that array is almost entirely zero and costs n^r entries, so the program keeps the compact list of hyperedges instead and pays nothing for the empty cells. Sampled $G(n, p)$ graphs are ordinary simple graphs and therefore use this same representation without any change. A graph or digraph on n vertices is stored as an integer matrix μ, where $\mu(u, v)$ is the multiplicity of the edge or arc from u to v. A simple graph is the case $\mu \in \{0, 1\}$, an undirected graph satisfies $\mu = \mu^\top$, and a multigraph allows larger entries. The third axis, edge against vertex separation, is

a property of the question rather than the object, so it lives in how we measure μ, not in μ itself. Figure 2.2 illustrates the encoding.

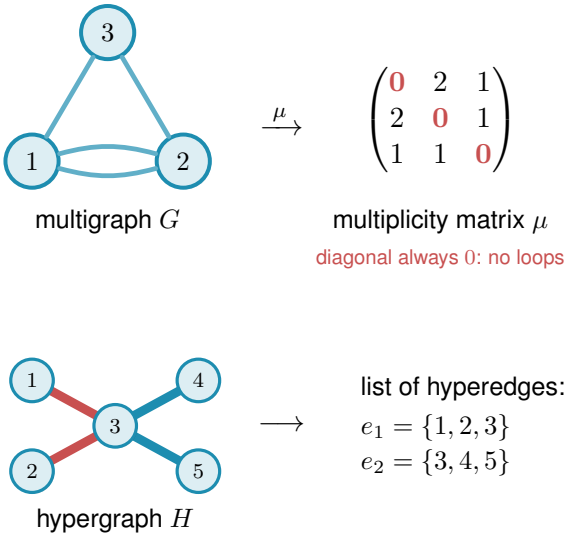


Figure 2.2. How the program stores a graph. Top: a graph or digraph on n vertices is one $n \times n$ integer matrix μ, with $\mu(u, v)$ the number of parallel edges or arcs from u to v. The double edge between 1 and 2 gives $\mu(1, 2) = 2$. The diagonal is always zero (red), since no vertex carries an edge to itself, an undirected graph keeps $\mu = \mu^\top$, a directed graph allows $\mu(u, v) \neq \mu(v, u)$, and a simple graph caps every entry at one. Bottom: the hypergraph is the one model that does not use the matrix at all, since a matrix has no room for an edge on three vertices, so it is kept as a plain list of its hyperedges, here two metro lines that share vertex 3.

`Graph(n, Variant(directed, simple))` MODEL

in the vertex count n and a `Variant`, the small record holding the two booleans

out a graph whose whole state is the matrix μ, stored as the `numpy` array `self.mu`

how one $n \times n$ integer `numpy` array is the object, an undirected graph keeps $\mu = \mu^\top$, and a simple graph caps every entry at one.

Keeping a single representation is what lets one measurement routine, one certifier, and one search serve every case. Nothing downstream asks which of the twelve problems it is solving. It asks only the two booleans (directed or undirected, simple or multi) and the matrix. Everything that builds or edits a graph is a thin wrapper around μ that respects those booleans, and these operations are so routine that they need no individual exposition, and Table 2.1 lists them together.

2.3 The connectivity checker built on maximum flow

The whole question reduces to a flow computation. A *maximum flow* asks how much can be pushed from one point to another through a network whose links each carry a limited capacity. Menger's theorem says the most edge-disjoint u–v paths a graph offers equals its smallest u–v cut, the fewest links whose removal severs every route between the two. The max-flow / min-cut

Operation	Meaning
`addEdge(G, u, v, k)`	add k parallel edges or arcs $u \to v$ (a simple graph saturates at one)
`removeEdge(G, u, v, k)`	remove k copies, never below zero
`setMultiplicity(G, u, v, c)`	set $\mu(u, v) = c$ directly
`multiplicity(G, u, v)`	read $\mu(u, v)$
`hasEdge(G, u, v)`	whether $\mu(u, v) > 0$
`degree(G, v)`	total degree of v (in- plus out-degree if directed)
`edgeCount(G)`	number of edges or arcs, counted with multiplicity
`edges(G)`, `vertices(G)`	iterate the present edges, and the vertex set
`copy(G)`	an independent duplicate

Table 2.1. The routine operations on the graph model. Each is a one-line wrapper that reads or writes the matrix μ while keeping an undirected graph symmetric and a simple graph binary. They are bookkeeping, and the chapter's attention belongs to the routines that follow.

theorem says that smallest cut is exactly the value of a maximum flow, once each multiplicity $\mu(u, v)$ is read as the capacity of the link $u \to v$. So the route count we are after is just a maximum-flow value: measure the flow and you have measured the connectivity. The routine that runs this flow and reports the route count is the connectivity *checker*. It is the single most important routine in the program: it is the only place graph theory meets computation, and every later claim about connectivity passes through it. Because the search of Chapter 3 calls it by the million on tiny graphs, the program ships an optional helper, `_erdos_fast.so`, that runs this same flow in compiled C. It is a speed-up only: it returns the identical value the pure-Python checker does, verified against it, so no number in the thesis depends on whether the helper is built.

`local_edge_connectivity(G, s, t)` $\to \mathbb{N}$ **MEASURE**

in a graph G and an ordered pair (s, t)

out $\lambda_G(s, t)$, the maximum number of edge-disjoint s–t paths

how a single `scipy.sparse.csgraph.maximum_flow` on the network whose arc capacities are the multiplicities μ, so by Menger the flow value is the path count.

`max_edge_connectivity(G)` $\to \mathbb{N}$ **MEASURE**

in a graph G

out $\lambda^{\max}(G)$, the largest local edge-connectivity over all pairs

how one max-flow per pair (ordered if directed), take the largest. This is the quantity the edge problem caps at $m - 1$.

The vertex-disjoint version follows the same idea after one construction: split each vertex v into an in-copy v^{in} and an out-copy v^{out} joined by a single unit-capacity arc (Figure 2.3). Every arc $u \to v$ of the original graph becomes $u^{\text{out}} \to v^{\text{in}}$ in the split network. The construction is not special to digraphs: for an undirected graph each edge $\{u, v\}$ is first read as the two opposite arcs $u \to v$ and $v \to u$, and then the same split applies, so one routine serves the undirected

and directed vertex problems alike. Routing through v now consumes the one unit of capacity on its internal arc, so two paths sharing v as an intermediate vertex compete for that unit and cannot coexist in any maximum flow. A max-flow from u^{out} to v^{in} in the split network therefore equals the number of internally vertex-disjoint u–v paths in the original graph.

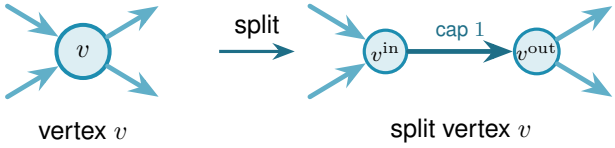
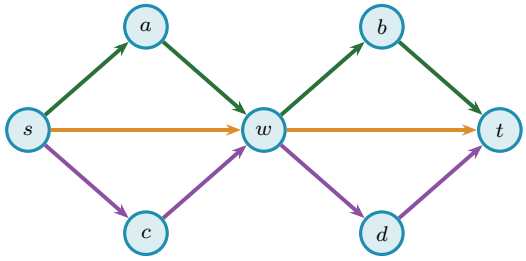


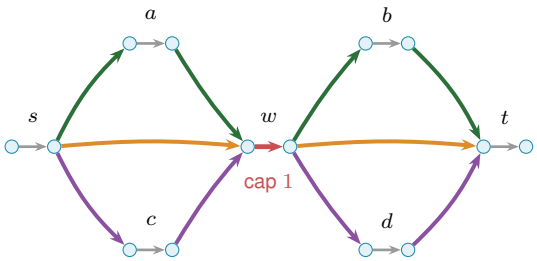
Figure 2.3. Vertex splitting for the vertex-disjoint checker. Incoming arcs attach to v^{in} and outgoing arcs leave from v^{out}, connected by a single unit-capacity arc. Any directed path that uses v as an intermediate vertex passes through this bottleneck, and two such paths would need two units of capacity and are therefore separated.

Figure 2.4 applies the split to a whole digraph. Three arc-disjoint routes are forced through one shared vertex w, so on the split network all three must cross the single unit-capacity arc inside w, which caps the number of internally vertex-disjoint routes below the edge-disjoint count.



$\lambda(s,t) = 3$: all three routes pass through w

(a) original digraph



(b) split network: $\kappa(s,t) = 1$

Figure 2.4. The vertex-split checker on a larger example. **(a)** A digraph in which three arc-disjoint s–t routes, $s \to w \to t$ (orange), $s \to a \to w \to b \to t$ (green), and $s \to c \to w \to d \to t$ (purple), all pass through the single vertex w, so $\lambda(s,t) = 3$. **(b)** The split network replaces every vertex V by an arc $V^{\text{in}} \to V^{\text{out}}$ of capacity one. All three routes must now traverse the single unit arc inside w (red), which carries only one unit, so at most one internally vertex-disjoint route gets through and $\kappa(s,t) = 1$. Splitting every vertex this way turns the vertex-disjoint count into an ordinary maximum flow.

`max_vertex_connectivity(G)` $\to \mathbb{N}$ **MEASURE**

in a graph G

out $\kappa^{\max}(G)$, the largest local vertex-connectivity over all pairs

how the same sweep as the edge checker, but each flow runs on the split capacity matrix `_split_capacity_matrix(G)` of Figure 2.3, so the unit internal arcs count internally vertex-disjoint paths. Parallel edges are given capacity one here, so they never raise κ.

The two checkers are the entire interface between graph theory and computation in this thesis. As a sanity check, applied to the directed double star on n vertices, the edge checker reports $\lambda^{\max} = 1$, and the arc count $2(n-1)$ matches the proved values $\ell_2^{\mathrm{dir}}(n) = 2, 4, 6, 8$ for $n = 2, 3, 4, 5$.

The same checker answers two different questions. Throughout this chapter it runs in its exact-value mode, returning $\lambda^{\max}$ or $\kappa^{\max}$ on the nose, and every proved or reported number rests on that exact reading. The search of Chapter 3 needs only the cheaper yes-or-no question "is the binding connectivity above the bound?", so there the same flow runs in a capped mode that stops as soon as it has seen one route too many. The capped mode never changes a reported value: it is a faster way to ask for a single bit, and Chapter 3 shows that the search built on it reproduces the exact search step for step.

The Gomory–Hu tree from Chapter 1 reappears here as an efficiency, and it is the honest kind. For undirected graphs all $\binom{n}{2}$ edge-connectivities are read off a single weighted tree, so $\lambda^{\max}$ is the heaviest tree edge rather than the maximum over $\binom{n}{2}$ separate flows. The speed-up is real, but it is bought with a theorem, not a heuristic.

`max_edge_connectivity_via_tree(G)` $\to \mathbb{N}$ **MEASURE**

in an undirected graph G

out $\lambda^{\max}(G)$, identical to the pairwise sweep

how build one Gomory–Hu tree with `networkx.gomory_hu_tree`, then return its heaviest edge, which costs $n-1$ flows in place of $\binom{n}{2}$ and cross-checks the checker.

The hypergraph is the one model that never used the multiplicity matrix, since it is stored as a plain list of hyperedges, each one a set of r vertices. The checker still has to read it, and the trick is to run the same flow computation as before on a slightly larger network that we build on the spot. The difficulty is that one hyperedge ties several vertices together at once. By the definition of a Berge route in Chapter 1, only one route may use a hyperedge at a time, so we cannot just treat it as a bundle of ordinary links. Instead, for every hyperedge we add one extra helper point and connect each of that hyperedge's member vertices to it (Figure 2.5). A route that wants to use the hyperedge now enters through one member vertex, passes through the helper point, and leaves through another. The trick is to make the helper point carry a single unit of capacity, built by the very same device as the vertex bottleneck of Figure 2.3: the helper point is split into an

in-copy and an out-copy joined by one unit-capacity arc, and every route boarding the hyperedge must squeeze through that one arc. A second route would need a second unit of capacity there and so is shut out. That one unit is what captures the rule that one hyperedge serves one route. Counting routes between two vertices is then the very same maximum flow as before, run on this enlarged network, and when a hyperedge happens to hold just two vertices the helper point collapses back to an ordinary link, so the answer agrees with the graph case. That the flow on the enlarged network counts exactly the hyperedge-disjoint Berge routes is the hypergraph version of Menger's theorem, proved in Appendix A (Theorem A.31), so the checker rests on a theorem rather than an analogy.

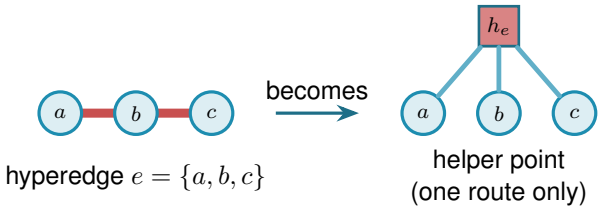


Figure 2.5. Measuring Berge connectivity by maximum flow. A hyperedge e, drawn as a metro line through its members (left), is replaced by a helper point h_e joined to each member (right). The helper point holds a single unit of capacity inside it, so at most one route may pass through h_e and the line runs one train at a time, which is what makes the hyperedge serve only one route. Counting independent routes between two vertices is then an ordinary maximum flow on this enlarged network, and a two-vertex hyperedge collapses to a single ordinary edge.

Figure 2.5 shows one hyperedge in isolation. Figure 2.6 carries the idea to a whole hypergraph in two panels. Panel (a) is the hypergraph drawn as a metro map: each of the eight hyperedges is one coloured line threading its three member stations, vertex to vertex, and a station on several hyperedges shows several coloured lines running through it. Panel (b) reads the same map as the flow network. For one pair of stations, u and v, it adds the helper point of every hyperedge a route boards, drawn as a small gate $h_\bullet$ in that line's colour, and traces the edge-disjoint Berge routes between u and v as thin lines passing through those gates, with the lines no route uses faded back. The checker reads $\lambda(u, v) = 4$: two routes run straight through the two hyperedges that contain both u and v, and two longer ones change lines at an intermediate station, four in all, while a fifth would have to reuse a gate.

One way to picture the helper point is the metro map of Chapter 1: each hyperedge is a metro line, its member vertices are the stations on it, and the helper is the line itself. Boarding at any station counts as taking that line. Because only one route may pass through the helper, only one train runs on the line at a time, so a second route must reach its destination without ever boarding it. The same flow computation that counts edge-disjoint paths through ordinary links now counts routes that share no line, which is exactly what Berge edge-disjointness means.

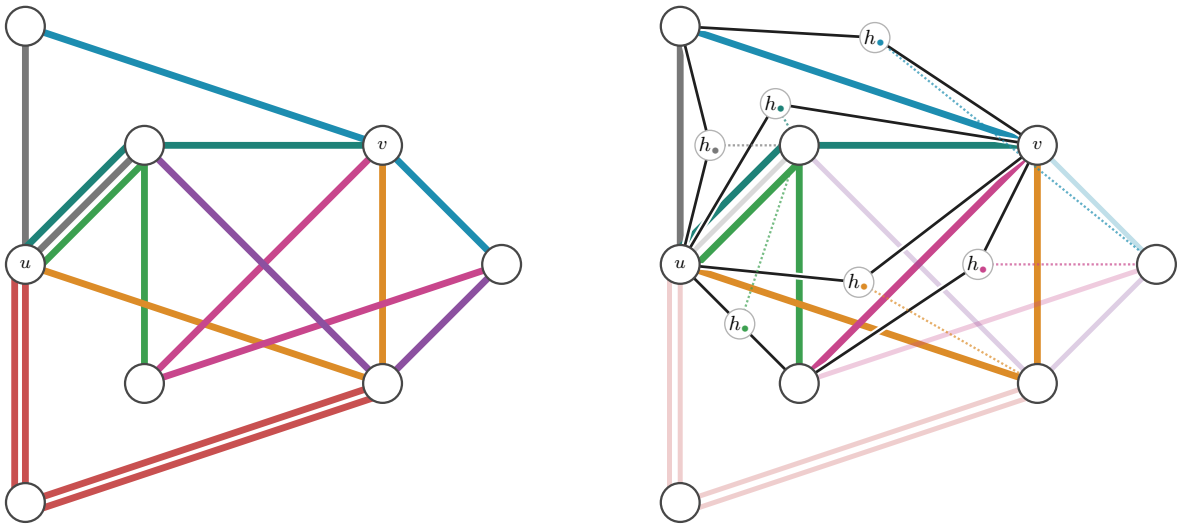


Figure 2.6. A 3-uniform hypergraph on eight stations, with the pair u, v singled out. Panel (a) draws it as a metro map: every hyperedge is one coloured line threading its three members, vertex to vertex, the metro reading of Chapter 1. Panel (b) reads the same picture as the helper-point flow network. Each hyperedge a route boards becomes its one-route gate $h_\bullet$ of Figure 2.5, shown in that line's colour and placed on the route, and the thin black lines are the edge-disjoint Berge routes from u to v threading those gates. A dotted leg ties each gate to the hyperedge's third member, the station that route does not use, and lines on no u–v route are faded. Two routes pass straight through the two hyperedges that hold both u and v, and two longer ones transfer at an intermediate station, so the checker reads $\lambda(u, v) = 4$. A fifth route would have to reuse a gate.

`max_hyperedge_connectivity(H)` $\to \mathbb{N}$ **MEASURE**

in an r-uniform hypergraph H

out $\lambda^{\max}(H)$, the largest Berge edge-connectivity over all pairs

how build the enlarged capacity matrix `_hyper_capacity_matrix(H)` in which every hyperedge becomes a helper point that only one route may pass through, then take the same maximum flow as the graph checker. On two-vertex hyperedges it matches ordinary edge-connectivity.

On the complete graph K_n the hypergraph checker returns $n-1$, and on the complete three-uniform hypergraph $K_4^{(3)}$ it returns 3, larger than the two hyperedges through each pair because longer Berge routes also carry flow. The self-test confirms both of these values, so the construction is checked rather than merely asserted. The same helper-point network also answers the vertex-disjoint hypergraph question without alteration: splitting each ordinary vertex as in Figure 2.3 counts internally vertex-disjoint Berge routes. The one remaining direction, the *directed* hypergraph, needs the helper point to learn which way is which, and that is the subject of the next subsection.

2.3.1 The directed hypergraph checker

A *directed* hyperedge carries an orientation: it is a single *tail* vertex together with $r-1$ *head* vertices, drawn as the star of Figure A.8, and a directed Berge route may only enter it at the tail and leave it at one of its heads. The undirected helper point treated a hyperedge as a line any route could board at any station. The directed one must instead be a one-way turnstile: a route may step onto the hyperedge only at its tail and step off only at a head. The gadget that enforces this is the helper point made directional. For a hyperedge $e = (a;\, b, c, \dots)$ we add a gate node g_e, send one capacity-one arc from the tail into it, $a \to g_e$, and send an arc out of it to each head, $g_e \to b$, $g_e \to c$, and so on (Figure 2.7). A route reaches g_e only through the tail a, and the single unit of capacity on $a \to g_e$ lets just one route use the hyperedge, exactly as the undirected turnstile did, but now it can leave only towards a head.

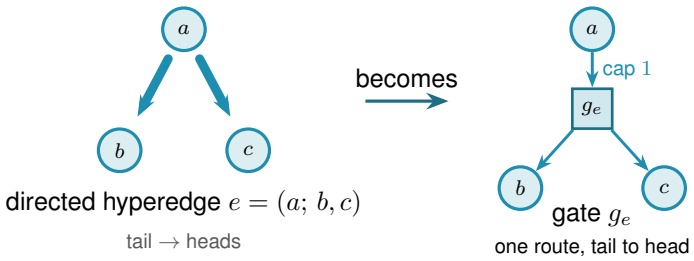


Figure 2.7. The directed helper point. A directed hyperedge $e = (a;\, b, c)$, the tail a fanning to its heads b, c (left), becomes a gate node g_e fed by one capacity-one arc from the tail and feeding an arc to each head (right). A route may reach g_e only through the tail and may leave only towards a head, so the gate is a one-way version of the turnstile of Figure 2.5, and the single unit of capacity still lets only one route use the hyperedge.

Figure 2.8 reads the same hypergraph directed, again in two panels. Panel (a) gives the whole hypergraph oriented, every hyperedge a one-way line whose arrows point away from its tail, so the middle-tailed lines fan out from their centre and the end-tailed ones run one way along their length. Panel (b) traces the routes from v to u, the gate rule of Figure 2.7, keeping each ridden hyperedge's used rail in bold and threading a black arrow from the route's entry station through the one-route gate $h_\bullet$, set in a clear gap off the rails, to its exit, with a dotted leg to the hyperedge's third member. Orientation costs connectivity. Of the four undirected routes between u and v only two survive the arrows, $\lambda(v, u) = 2$, because v can set out through only the two hyperedges it heads, and the two survivors cross at the interchange l so they share no hyperedge. The same vertex-splitting trick of Figure 2.3 laid on top then counts internally vertex-disjoint directed routes, so this one construction measures all four hypergraph variants and `solve` reaches every one of the twelve problems through a single checker.

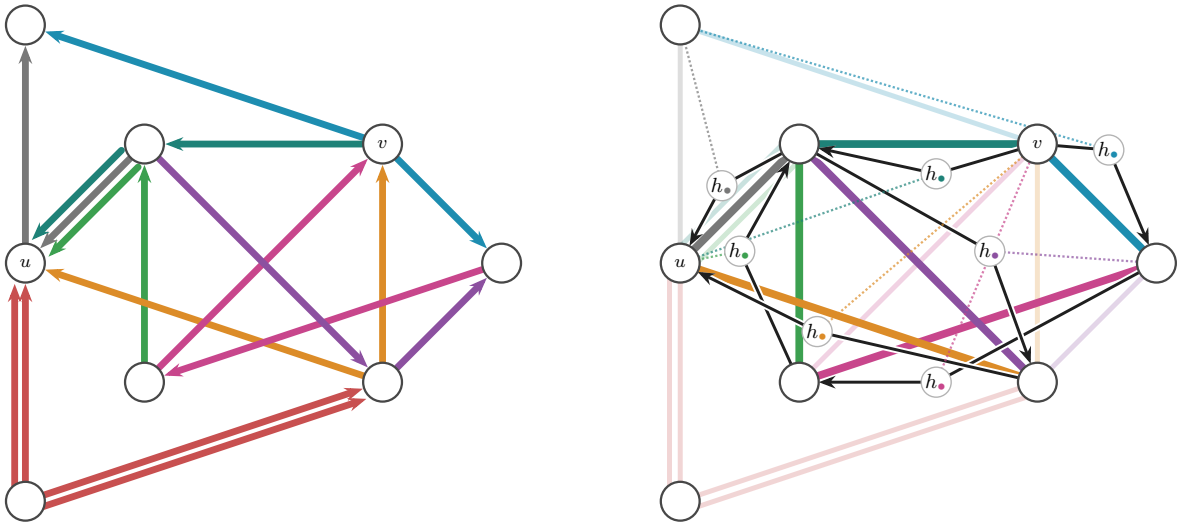


Figure 2.8. The hypergraph of Figure 2.6 read directed. Panel (a) gives the whole directed hypergraph, every hyperedge a one-way line with its arrows pointing away from the tail station, so a route may join a line at its tail and ride it to a head. Panel (b) traces the routes from v to u. Each ridden hyperedge keeps its used rail in bold colour, and a black arrow runs from the route's entry station through the hyperedge's one-route gate $h_\bullet$ to its exit, the directed turnstile of Figure 2.7, with the gate set in a clear gap off the rails. A dotted leg in the line's colour ties each gate to the hyperedge's third member, the station the route does not visit. Two edge-disjoint directed Berge routes from v to u survive. The long route $v \to j \to c \to l \to u$ rides nmj, mcj, elc and nel in turn, and the short route $v \to l \to d \to u$ rides elm, ldj and edm. They cross only at the interchange l, so they share no hyperedge, and the checker reads $\lambda(v, u) = 2$, since v can set out through only the two hyperedges it heads. Against the undirected four of Figure 2.6, the arrows have halved the count.

The gate is indifferent to how a hyperedge's vertices are split between tails and heads. It draws one capacity-one arc in from every tail and one out to every head, so the single forward tail it was built for is only the simplest case: give it a hyperedge with several tails and one head, or several of each, and it still counts the routes that enter at a tail and leave at a head. The same checker therefore measures every *orientation model* of a directed Berge edge, not just the

one-tail one, and Section 4.3.1 takes up which of those models are worth distinguishing and what the program finds for them.

With the checker in place the proposed value of Proposition 1.14 can be tested on any small hypergraph the way every other variant is, which is the subject of the next section.

2.4 Checking every graph of a fixed size

The checker measures one graph. To bound *every* graph of a given size, the most direct method is the one the base cases of Chapter 1 already needed: enumerate them all. A simple graph on n vertices is a choice of $\mu(u, v) \in \{0, 1\}$ on each off-diagonal cell, a multigraph a choice in $\{0, \ldots, m-1\}$, and a digraph varies every ordered pair rather than only the upper triangle. Walking that product, measuring each candidate with the checker, and keeping the densest feasible one settles the value exactly, and because it has looked at every graph it is a genuine upper bound, not evidence.

This is the first thing `solve` does when it is asked to prove a small case. With its exhaustive flag set it walks that product, measures each candidate with the checker, and keeps the densest feasible one. The value it returns is exact and exhausted, a true upper bound for that n, but only while the count of graphs stays small. No new routine is involved, only `solve` choosing to enumerate.

Run on the small cases this confirms the proved values directly, with no induction and no cleverness. Table 2.2 collects a spread of variants where the enumeration finishes: in every row the machine returns exactly the value Chapter 1 proved by hand. This is the first payoff of the uniform representation. One driver, `solve`, pointed at six different problems, settles all six.

Variant	m	n	`solve`	Theory
simple undirected, edge	2	5	4	$\lfloor m(n-1)/2 \rfloor = 4$
simple undirected, edge	3	5	6	$\lfloor m(n-1)/2 \rfloor = 6$
multigraph undirected, edge	3	5	8	$(m-1)(n-1) = 8$
simple undirected, vertex	2	6	5	$k_2(n) = n - 1 = 5$
simple directed, arc	2	5	8	$\max(2(n-1), \lfloor n^2/4 \rfloor) = 8$
simple directed, arc	2	6	10	$\max(2(n-1), \lceil n^2/4 \rceil) = 10$

Table 2.2. Variants that converge under plain enumeration. For each small case `solve`, in its exhaustive mode, walks every graph of the variant, measures it with the checker, and returns the densest feasible one. Every value matches the hand proof of Chapter 1 exactly.

The honesty of the method is also its limit. The number of graphs it must visit grows as $2^{\binom{n}{2}}$ for simple undirected graphs and $2^{n(n-1)}$ for digraphs, and capping multiplicities at $m-1$, so that each cell ranges over the m values $\{0, \ldots, m-1\}$, raises the base from two to m, giving $m^{\binom{n}{2}}$ undirected multigraphs and $m^{n(n-1)}$ directed ones. The enumeration is exact wherever it finishes, and it finishes only for the smallest sizes. Chapter 3 opens with exactly how fast this wall arrives. For now the point is that the wall arrives precisely where Chapter 1 needed help, at the directed base cases $n = 6, 7$, so the rest of this chapter is about reaching a little further than

blind enumeration can.

2.5 Reaching further: a pruned search for the directed base cases

The base cases $n \leq 7$ of the directed $m = 2$ theorem (Theorem 1.9) are the finite checks the induction of Appendix A depends on, and they sit just past where blind enumeration of digraphs is tractable. They can be reached by enumerating more cleverly. Feasibility is monotone: adding an arc can never lower $\lambda^{\max}$, so once a partial digraph is infeasible every completion of it is infeasible too and the whole branch is abandoned at once. A counting bound prunes further. At any partial digraph, count the arcs already placed and add the most the still-undecided cells could ever contribute. That sum is a ceiling on every completion of the branch, so if even the ceiling cannot beat the best feasible digraph found so far, the branch is dropped unexplored (Figure 2.9). What remains is a branch and bound over simple digraphs with the connectivity checker as the feasibility test, exact and complete at these sizes where the blind walk is not.

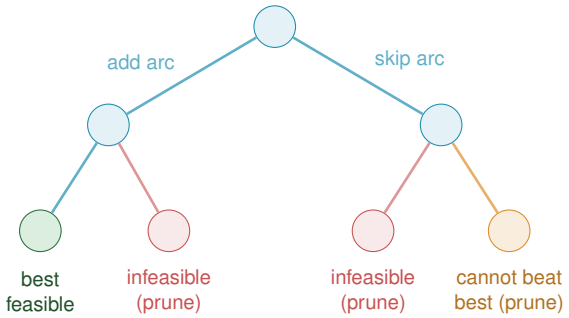


Figure 2.9. Branch and bound over digraphs. Each node fixes one more cell, arc present or absent, so the leaves are whole digraphs. Two rules cut entire subtrees before they are walked. A branch whose partial digraph already breaks the connectivity ceiling is infeasible, and so is every completion of it (red). A branch whose best possible completion still cannot beat the densest feasible digraph found so far is dropped on the counting bound (orange). Only the surviving branches are explored, which is what lets the search reach the base cases $n \leq 7$ the blind walk cannot.

Here `solve` changes its implementation without changing its interface. For a simple digraph it runs this branch and bound, `_exhaustive_directed`, in place of the blind walk, still with the connectivity checker as the feasibility test, and so reaches the base cases $n \leq 7$ that blind enumeration cannot. The booleans alone decide which of the two it uses, and the caller never asks.

Run on the directed $m = 2$ problem `solve` returns the complete sequence $2, 4, 6, 8, 10, 12$ for $n = 2, \ldots, 7$, each proved complete at its size. The densest digraph it finds at each n is exactly the extremal shape Chapter 1 named, the bidirected star (the hub) below the crossover at $n = 7$ and the one-directional bipartite digraph above it, so the search confirms not just the values but the winning constructions. These are exactly the base cases the induction requires, and with them the long proof of Theorem 1.9 rests on a single auditable search rather than on anything done by hand. The exhaustive-search transcript is in Appendix A.

2.6 Generating the directed cases faster

The $m = 3$ directed multigraph result reduces, through Remark A.30, to a handful of finite statements about multigraphs on six and seven vertices, and proving them means listing every such multigraph up to a renaming of its vertices. The program does this in two independent ways, on purpose: when two different methods return the same list, that agreement is itself a check.

The first way, `enumerate_extremal_directed_multigraphs`, is a search written from scratch. It is a depth-first walk over multiplicity matrices that abandons a branch the moment it turns infeasible or can no longer beat the best graph found, and it keeps just one representative of each isomorphism class by reducing every finished graph to a canonical relabelling with `_canonical_form`, the lexicographically smallest matrix over all relabellings. The canonical form keeps the memory small, but the time is spent walking labelled partial graphs, and time is what runs out: the full classification at $n = 5$ costs this in-house search about 456 seconds, and it climbs steeply from there.

The second way, `enumerate_extremal_directed_multigraphs_via_generation`, does not search at all, it generates, and it was suggested by Prof. Jan Goedgebeur. The idea is to hand the hard part, listing one graph per isomorphism class, to a tool that already does it supremely well: the `geng` program from the `nauty` toolkit of McKay and Piperno [9], invoked through a subprocess and decoded from its `graph6` output by `_graph6_edges`. `geng` lists the non-isomorphic *undirected* graphs on n vertices, and the program then decorates each one, cell by cell, with the directed multiplicities the problem needs, so the isomorphism reduction is inherited for free from the support graph. The companion orientation tools `directg` and `watercluster2` are not used, because each gives only one orientation per edge and so cannot express the bidirected arcs and repeated multiplicities the extremal graphs are built from. Layering the multiplicities in-house on top of `geng` is what fits the problem. To be precise about credit: Prof. Goedgebeur is not an author of `nauty`, which is the work of McKay and Piperno cited above. He suggested the generation route, and it is his suggestion that the speed-up rests on.

The two ways agree to the last graph. The generated pipeline reproduces the in-house classification exactly at every settled size, two non-isomorphic extremal families at $n = 4$ and three at $n = 5$, and it runs substantially faster, since the isomorphism work it never repeats is precisely what `geng` has already done. One difference decides which to trust further out: the in-house search leans on a conjectured counting bound once $n \geq 7$, while the generated enumeration prunes only with proved bounds, so it is a sound and complete search at every n. That makes it the practical route to the one finite case the $m = 3$ argument still waits on, the degree-bounded classification at $n = 7$ that is fact (b) of Remark A.30.

2.7 Proving every m at once

The pruned search proves one (n, m) at a time. For the directed multigraph arc problem `solve` has a sharper implementation, one that proves every m at a fixed n in a single pass, and it is

worth the extra machinery because it closes the directed multigraph case outright for the sizes it reaches.

The key is that m can be scaled out of the problem completely, so that it never appears in the computation at all. Start with any directed multigraph that is feasible at level m, so $\lambda^{\max} \le m-1$, and divide every multiplicity by $m-1$. Two things happen at once. The multiplicities become weights $w(u,v)$ between 0 and 1, and since dividing every capacity by $m-1$ divides every flow by the same factor, the maximum flow between each pair drops to at most one. The arc count, meanwhile, turns into the total weight $\sum_{u\neq v} w(u,v) = |A|/(m-1)$. So a dense feasible multigraph becomes a heavy feasible weight matrix, and the other way round. Write $M^*(n)$ for the largest total weight any such matrix can carry. It is one number for each n, with no m inside it, and

$$L_m^{\mathrm{dir}}(n) = (m-1)\,M^*(n), \qquad M^*(n) = \max\Big\{\textstyle\sum_{u\neq v} w(u,v) : \mathrm{maxflow}(s,t;w) \le 1 \text{ for all } s,t\Big\}.$$

There is no m on the right-hand side, so one solve for $M^*(n)$ settles $L_m^{\mathrm{dir}}(n)$ for every m at once. The two directions of the identity are easy to see on their own. Scaling a feasible multigraph down gives the upper bound $L_m^{\mathrm{dir}}(n) \le (m-1)M^*(n)$. For the matching lower bound, look at the heaviest matrix the solve actually finds. It is the *double star*: the bidirected star, weight one on both arcs $h \to v$ and $v \to h$ between the hub and every other vertex. Its weights are all 0 or 1, so scaling it back up by $m-1$ gives an honest integer multigraph, not a fractional one, with $2(n-1)(m-1)$ arcs and $\lambda^{\max} = m-1$. The two bounds therefore meet for every m together. And because the best weight matrix already has whole-number weights, the fractional optimisation has lost nothing: the reported $M^*(n)$ is at once the largest fractional weight and the densest integer multigraph divided by $m-1$, never an overestimate.

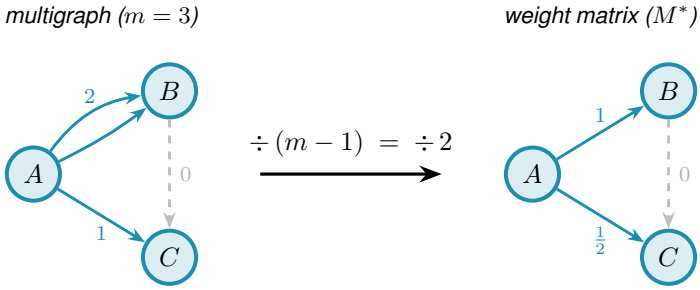


Figure 2.10. The scaling step for $m = 3$. Dividing each multiplicity by $m-1 = 2$ maps the integer multigraph (left) to a weight matrix with all values in $[0,1]$ and max-flow at most one between every pair (right). The double arc $A \to B$ (multiplicity 2) becomes weight 1, the single arc $A \to C$ (multiplicity 1) becomes weight $\frac{1}{2}$, and the absent arc $B \to C$ (multiplicity 0, shown dashed) stays absent. Crucially, m drops out on the right, so a single optimisation over all weight matrices pins $L_m^{\mathrm{dir}}(n) = (m-1)\,M^*(n)$ for every m simultaneously.

What remains is to compute $M^*(n)$, and the difficulty is that the constraint "maximum flow at most one" is itself the answer to a flow problem rather than a plain inequality on w. The max-flow / min-cut theorem removes it. The maximum s–t flow equals the smallest capacity of any *cut* separating s from t. A cut just splits the vertices into a source side holding s and a sink side

holding t, and its capacity is the total weight of the arcs that cross from the source side to the sink side. So $\mathrm{maxflow}(s,t) \le 1$ holds exactly when some such cut has capacity at most one. That lets us drop the flow entirely: instead, for every ordered pair (s,t), the optimisation picks one cut and we require its capacity to stay at or below one.

A cut is described by a yes/no label x_u^{st} on each vertex, equal to one when u is on the source side, with s fixed on the source side and t on the sink side. An arc $u \to v$ crosses the cut exactly when u is on the source side and v is not, and a helper variable p_{uv}^{st} then picks up its weight $w(u,v)$. Forcing the crossing weights to sum to at most one for every pair forces every pair to admit a cut of capacity one, hence maximum flow at most one. This is the exact feasible region, not a relaxation of it, and that is what turns the result into a proof. The optimisation reports both the best total weight it has found and a value it has shown no matrix can exceed, and when those two meet the gap is zero and $M^*(n)$ is certified.

For the directed multigraph arc problem, then, `solve` runs this cut-counting optimisation in place of any enumeration: it chooses the minimum cut for every ordered pair, and a zero gap makes the reported $M^*(n)$ a proved upper bound, from which $L_m^{\mathrm{dir}}(n) = (m-1)M^*(n)$ delivers every m at once.

`prove_directed_multigraph(n)` $\to$ `ProofResult` **PROVE**

in the vertex count n (and tightening flags such as `use_deletion_cuts`)

out $M^*(n)$ with a proof status, from which $L_m^{\mathrm{dir}}(n) = (m-1)M^*(n)$ for every m

how builds the cut-counting MILP once with `_cut_counting_model` as a `pulp` model, then `_pick_solver` runs it on the bundled CBC by default or on Gurobi by a one-line switch. A zero optimality gap is a proof. The integer-weight companion `prove_integral_arc_bound(n, m, target)` answers the single feasibility question behind fact (a) of Remark A.30.

`solve` assembles this optimisation from n alone, and a reader content to take it on the strength of the cut argument just given can skip to Theorem 2.1 without loss. Written out it reads more heavily than it behaves, so we first name every symbol in plain words, then show the one inequality that looks cryptic is a simple yes-or-no switch.

The optimisation is a *Mixed-Integer Linear Program*, or MILP for short. “Linear” means every constraint and the objective are linear functions of the variables: no products, no squares, no exponentials. “Mixed-integer” means the variables split into two kinds: most are ordinary real numbers that may take any value in a continuous range, but a few are forced to be whole numbers, either 0 or 1. The reason for the split is conceptual. Arc weights w are naturally continuous: they are scaled multiplicities and can sit anywhere in $[0,1]$. But the cut assignment for a vertex is a hard binary choice: a vertex is either on the source side of a given cut or on the sink side, with no meaningful notion of being “half on each side”. Allowing the cut labels to be fractional would describe a different, weaker condition and could report an optimistic value that no integer-weight

multigraph can actually achieve. Keeping those labels in $\{0,1\}$ is what keeps the optimal gap honest, and it is what makes the program's result a proof rather than an estimate.

Three quantities appear, and only three. The first is already familiar.

- $w(u,v) \in [0,1]$ is the *scaled multiplicity* of the arc $u \to v$, the amount of arc weight the matrix places on $u \to v$ after the division by $m-1$ of the previous section. It is what we are maximising the total of. This is a continuous variable: it may be any real number in $[0,1]$.
- $x_u^{st} \in \{0,1\}$ is the *side label* for vertex u with respect to the ordered pair (s,t). The optimisation chooses one cut for each pair, and x_u^{st} records whether u is on the source side (1) or the sink side (0). The two endpoints are pinned: $x_s^{st}=1$ and $x_t^{st}=0$. This is the integer variable: a vertex cannot straddle the cut.
- $p_{uv}^{st} \geq 0$ is the *crossing weight*, the part of $w(u,v)$ that the pair (s,t)'s chosen cut is forced to count. It equals $w(u,v)$ when the arc $u \to v$ crosses from source to sink, and is otherwise left at zero. This is a continuous variable.

With those three named, the program is short:

$$
\begin{aligned}
\text{maximise} \quad & \sum_{u \neq v} w(u,v) && \\
\text{subject to} \quad & p_{uv}^{st} \geq w(u,v) + x_u^{st} - x_v^{st} - 1 && \text{for all } (s,t),\ (u,v), \\
& \sum_{u \neq v} p_{uv}^{st} \leq 1 && \text{for all } (s,t), \\
& w(s,t) + \sum_{x} \min\bigl(w(s,x), w(x,t)\bigr) \leq 1 && \text{for all } (s,t), \\
& d(v_0) \leq d(v_1) \leq \cdots \leq d(v_{n-1}), && \text{with } d(v) = \textstyle\sum_{u \neq v} \bigl(w(u,v) + w(v,u)\bigr), \\
& w \in [0,1], \quad x \in \{0,1\}. &&
\end{aligned}
$$

The first line is the only one that looks cryptic, and it is just an indicator written as an inequality. Its right-hand side $w(u,v) + x_u^{st} - x_v^{st} - 1$ takes one of four shapes according to which side the cut puts the two endpoints u and v on, and Table 2.3 works all four out. Because $w(u,v) \leq 1$, the right-hand side is positive in one case only, the crossing case, where it equals $w(u,v)$ and so forces $p_{uv}^{st} \geq w(u,v)$. In the other three cases it is zero or negative, so the constraint reads $p_{uv}^{st} \geq$ (something ≤ 0) and leaves p_{uv}^{st} free to stay at zero, which the maximisation is happy to let it do.

Figure 2.11 shows one chosen cut and the weights it counts. The second line of the program now reads plainly. It says the total crossing weight of this pair's cut, $\sum_{u \neq v} p_{uv}^{st}$, is at most one. By max-flow / min-cut a cut of capacity at most one exists for (s,t) exactly when $\mathrm{maxflow}(s,t) \leq 1$, so the two lines together say "every ordered pair admits a cut of capacity at most one", which is precisely the feasibility we want. This is the true feasible region, not a relaxation of it, so an optimal solution reported with zero gap is a proof and not merely evidence.

x_u^{st}	x_v^{st}	meaning	$w + x_u^{st} - x_v^{st} - 1$	forces
1	0	u source side, v sink side: arc *crosses*	$w(u,v)$	$p_{uv}^{st} \geq w(u,v)$
1	1	both on the source side	$w(u,v) - 1 \leq 0$	$p_{uv}^{st} \geq 0$
0	0	both on the sink side	$w(u,v) - 1 \leq 0$	$p_{uv}^{st} \geq 0$
0	1	u sink side, v source side	$w(u,v) - 2 \leq 0$	$p_{uv}^{st} \geq 0$

Table 2.3. The first constraint, case by case. This is where the bound $w \leq 1$ earns its keep: it makes the three non-crossing rows land at or below zero, so the helper p_{uv}^{st} is pushed up to the arc weight in exactly one case, the crossing one. The constraint is therefore an exact indicator, "count $w(u,v)$ when the arc crosses this cut, and nothing otherwise", with no slack and no fudge.

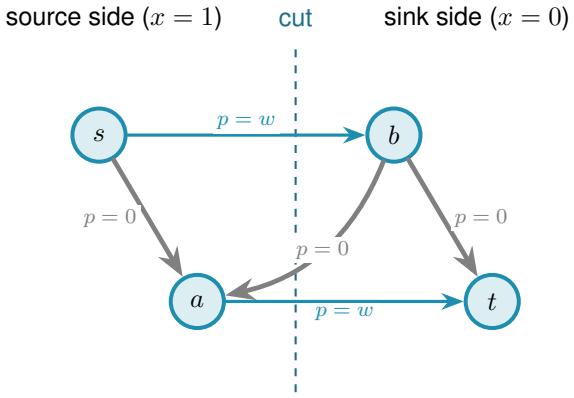


Figure 2.11. One ordered pair's chosen cut. The dashed line splits the vertices into the source side, where $x_u^{st} = 1$, and the sink side, where $x_u^{st} = 0$. Only arcs running from source side to sink side cross the cut, and the first constraint forces their helper p_{uv}^{st} up to the arc weight w (the two thick arcs). Arcs that stay on one side, and arcs that run the other way across the line, contribute nothing. The second constraint then caps the total counted weight, the thick arcs, at one.

The last two lines change no answer. They only help the solver finish in time. The third is a redundant shortcut. The direct arc $s \to t$, together with one two-step detour $s \to x \to t$ through each intermediate vertex x, already forms that many routes between s and t. So $w(s,t) + \sum_x \min(w(s,x), w(x,t)) \leq 1$ holds of any feasible matrix anyway, and stating it outright just sharpens the continuous picture the solver reasons from.

The $\min$ in that line is not a linear function, so it cannot appear in the MILP as written. To keep the program linear, each $\min(w(s,x), w(x,t))$ is replaced by a fresh helper variable z_x^{st} together with a binary selector $b_x^{st} \in \{0,1\}$. The selector records which of the two arguments is smaller: $b_x^{st} = 1$ means $w(s,x)$ is no larger than $w(x,t)$. Four linear inequalities then pin z_x^{st} exactly to the minimum. Two upper bounds, $z_x^{st} \leq w(s,x)$ and $z_x^{st} \leq w(x,t)$, prevent z from exceeding either argument. Two lower bounds, one of the form $z_x^{st} \geq w(s,x) - (1 - b_x^{st})$ and the other $z_x^{st} \geq w(x,t) - b_x^{st}$, each become active only when the selector chooses that argument: at any integer value of b exactly one lower bound bites and forces z up to the chosen weight. Together with the upper bounds this pins $z_x^{st} = \min(w(s,x), w(x,t))$ exactly. This technique is called the McCormick linearisation of a minimum. It adds one continuous variable z and one binary variable b per intermediate vertex per ordered pair, all of them appearing only in the tightening constraint and nowhere else, so the proof structure of the program is unchanged.

The fourth line orders the vertices by weighted degree. Renaming vertices cannot change any count, so insisting on this order rules out no graph, and it spares the solver the many relabelled copies of a graph that differ only in vertex names.

Theorem 2.1 (Directed multigraph value, small n)**.** *For $n \le 6$ and all $m \ge 2$, $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$, attained by the double star. Equivalently $M^*(n) = 2(n-1)$.*

It is worth being clear about what "for all m" rests on, because the theorem proves infinitely many values from finitely many solves. It is not that each m was tested. In this mode `solve` never sees m at all. It proves the single m-free number $M^*(n)$, and the identity $L_m^{\mathrm{dir}}(n) = (m-1)M^*(n)$ then carries that one fact to every m at once. The only quantity that is checked case by case, and the only genuine limit on how far the result reaches, is n.

`solve` proves Theorem 2.1 by returning $M^*(3) = 4$, $M^*(4) = 6$, $M^*(5) = 8$, and $M^*(6) = 10$, each optimal with zero gap, so that $M^*(n) = 2(n-1)$ over this range. The first three finish quickly, $M^*(3)$ and $M^*(4)$ in well under a second and $M^*(5)$ in a few seconds, while $n = 6$ takes about twenty minutes. At $n = 7$ the same encoding does not close, and we do not pretend otherwise: it is a finite obstacle of scale, not a new idea, and it marks the current edge of certified knowledge for this variant. The solver transcript is in Appendix A.

2.8 What the solver may claim

Now that the pieces inside `solve` are built, it is worth stating plainly, because the rest of the thesis depends on it, what each is allowed to claim. The checker *measures*: its connectivity values are exact. Its three proving implementations, the exhaustive enumeration, the pruned search, and the cut-counting, *prove*: an exhausted enumeration or an optimal, zero-gap solve is an upper bound for that fixed n. The search of Chapter 3 will *discover*: it produces concrete graphs, hence lower bounds, and never an upper bound. These three claims are exactly what `solve` labels its answer with, so that an exact value, a lower bound, and an upper bound can never be quietly confused with one another.

With the microscope built and aimed at proof, the variants with small answers are settled and the directed base cases the hand proof needed are in hand. What proof by enumeration cannot do is reach the sizes where the answers are only conjectured, because the number of graphs explodes long before the interesting cases arrive. The next chapter measures that explosion exactly, and then turns the same program loose to *discover* the dense graphs that blind enumeration can never reach.

2.9 Reproducibility

Every claim in this thesis that rests on computation can be re-run from a single Python driver. All of the program's logic lives in `program/erdos915_unified.py`, one self-contained file. Its core needs only `numpy` and `scipy`, because every connectivity value it reports is a single `scipy`

integer maximum flow on a capacity matrix. The model, the checker, the search, the enumeration, and the self-test all run on those two libraries alone.

A few more libraries sit on top, each for one job. The certifier uses `pulp` to build the cut-counting optimisation once for any solver, then runs it on the bundled CBC by default or on Gurobi by a one-line switch. Two others are optional and only for presentation: `matplotlib` draws the figures, and `networkx` supplies the Gomory–Hu tree behind one of them. An optional compiled helper, `_erdos_fast.so` (built from `_erdos_fast.c` by `build_fast.sh`), speeds up the hot-path `_tiny_maxflow` and `_canonical_form` calls, but the pure-Python path returns the same answers.

A handful of named entry points reach the whole pipeline. `solve` drives every variant. The measures are `max_edge_connectivity` and `max_vertex_connectivity`, with the hypergraph and Gomory–Hu views beside them. The provers are `prove_directed_multigraph` and `enumerate_extremal_directed_multigraphs`, and `search_for_dense_graph` is the discoverer. Running `python erdos915_unified.py` executes the built-in self-test, the suite of invariant checks that confirms the connectivity values quoted throughout this chapter, including the sanity checks on the directed double star and on K_n and $K_4^{(3)}$.

The figures and proofs are just as reproducible. Every figure is regenerated by the companion driver `program/make_figures.py`, which imports the same file and fixes every random seed, so the pictures are reproduced exactly rather than merely resembled, and the table in `program/README.md` records which routine produces which figure. The solver transcripts behind this chapter's certified values, the pruned-search log for Theorem 1.9 and the cut-counting certificate for Theorem 2.1, are reproduced verbatim in Appendix A, so a reader may check the proofs without running anything at all.

Chapter 3

Discovering Bounds by Search

The previous chapter proved what could be proved by looking at every graph of a fixed size. That method is honest and exact, and it stops almost at once. This chapter begins by measuring exactly how fast it stops, because the speed of that collapse is the whole reason a second kind of tool is needed. Then it builds that tool. Where the certifier walks all graphs, the search walks a few of them well, cooling like a physical system toward the densest feasible shape. What it returns is never a proof. It is a concrete graph, hence a lower bound and a conjecture, a stepping stone toward an upper bound that a later proof or certificate must supply. The chapter ends by asking the honest question about any such tool: how good is it, and how far from the truth might its answers be.

3.1 The wall: how enumeration explodes

The brute force of Chapter 2 visits every graph of a given size. The trouble is how many that is. A simple undirected graph on n vertices is a yes-or-no choice on each of the $\binom{n}{2}$ possible edges, so there are $2^{\binom{n}{2}}$ of them. A digraph chooses on each of the $n(n-1)$ ordered pairs, giving $2^{n(n-1)}$, and allowing multiplicities up to a cap c raises the base from two to $c+1$. These are not numbers that merely grow. They detonate. Figure 3.1 draws the growth on a logarithmic scale, and the message is the same in every panel: the count passes any reachable budget while n is still in the single digits.

Three readings of the figure matter for what follows. Adding vertices is the sharpest cost: each new vertex multiplies the count by an exponential in n, so a single extra vertex can move a problem from seconds to centuries. Raising the connectivity budget is the next cost, lifting the base of the exponent rather than its height, which is why the multigraph curves sit above the simple one in every panel. Direction is the third: a digraph has twice as many cells to fill as the undirected graph on the same vertices (and a directed 3-uniform hyperedge, a tail with two heads, has three times as many), so the directed panel climbs fastest of all, and those are precisely the variants whose answers Chapter 1 could not prove. The separation, by contrast, costs nothing here, because edge and vertex disjointness search the same graphs and differ only in what the checker measures, which is why a single pair of panels suffices for all of them. The certifier of Chapter 2 reaches a few vertices further than blind enumeration, because its cut-counting optimisation is polynomial in the size of the encoding rather than in the number of graphs, but solving that encoding is itself hard and the gain is measured in single vertices, not orders of magnitude. Past that, no exhaustive method reaches the interesting sizes. Something

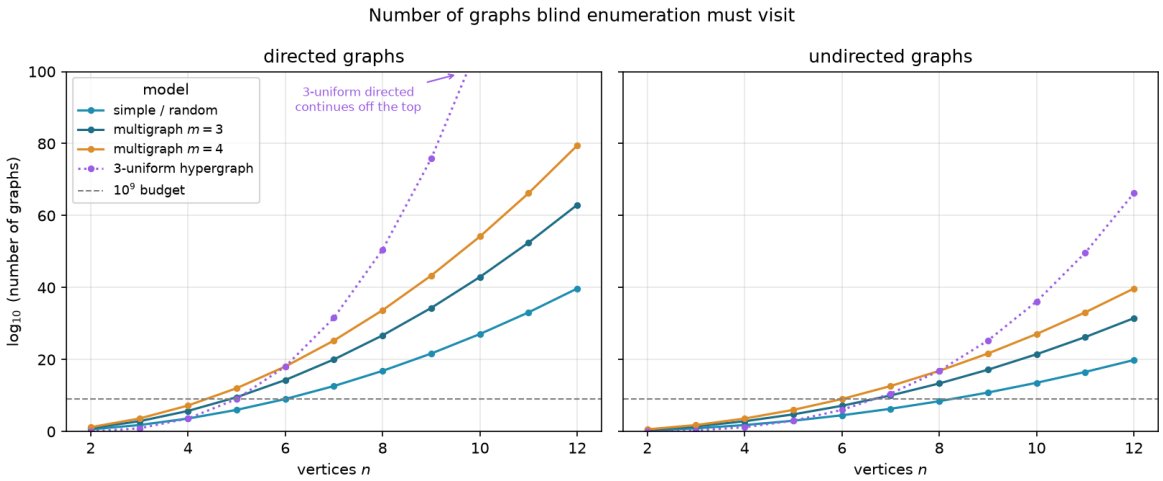


Figure 3.1. The number of graphs blind enumeration must visit, on a logarithmic vertical axis, for directed graphs (left) and undirected graphs (right). Each panel shows the simple model, two multigraph connectivity caps, and the three-uniform hypergraph (dotted), with a dashed line at a generous budget of 10^9 candidates, and every curve crosses that budget while n is still in the single digits. Writing E for the number of off-diagonal cells a model may fill, a simple graph offers 2^E candidates, a multigraph with multiplicity cap $m-1$ (the connectivity ceiling) offers m^E (one choice per cell from $\{0,\dots,m-1\}$), and the 3-uniform hypergraph offers $2^{\binom{n}{3}}$ undirected or $2^{n\binom{n-1}{2}}$ directed, since a directed hyperedge is a tail together with two heads. The count depends only on the model and the direction, never on whether one later asks for edge- or vertex-disjoint routes, so a single pair of panels covers all of those variants at once. Direction multiplies the number of cells to fill, by two for graphs (ordered versus unordered pairs) but by three for the 3-uniform hypergraph, so the directed panel sits far above the undirected one, and the directed cases, exactly the ones Chapter 1 left open, are the first to go.

that does not look at every graph is required.

3.2 The temperature search

We want, for fixed n and m, a graph with as many edges as possible subject to $\lambda^{\max} \le m-1$ (or $\kappa^{\max} \le m-1$ for the vertex variants). The space of graphs on n vertices is enormous and its feasible region is not convex: adding an edge can be harmless until, together with edges added long before, it suddenly creates a forbidden pair. A greedy walk that only ever accepts an improving move stalls at the first such collision, in a graph that is locally maximal but globally far from extremal.

The classical method for this kind of landscape is *simulated annealing*, named after the metallurgical process of cooling a heated material slowly so that its atoms settle into a low-energy crystal rather than a disordered glass. Applied to graphs, annealing escapes local optima by sometimes accepting a worsening move, and by accepting fewer of them as the run proceeds. Each graph G is a state with energy

$$E(G) = -|E(G)| + \Lambda \cdot \max\bigl(0,\ \lambda^{\max}(G) - (m-1)\bigr),$$

Algorithm 1: Temperature search for a dense feasible graph

Input: vertices n, value m, start T_0, cooling α, penalty Λ, steps S
Output: the densest feasible graph encountered

$G \leftarrow$ empty graph on n vertices;
$T \leftarrow T_0$;
for S *steps* **do**
  propose G' by toggling one adjacency, biasing removals toward low-sensitivity edges;
  $\Delta \leftarrow E(G') - E(G)$;
  if $\Delta \leq 0$ ***or*** $\mathrm{rand}() < e^{-\Delta/T}$ **then**
    $G \leftarrow G'$;
  if G *is feasible* ***and*** *denser than the best so far* **then**
    record G as the best;
  $T \leftarrow \alpha T$;
return the best recorded graph;

which rewards edges and penalises any excess connectivity. The penalty is a soft constraint rather than a hard one, so the walk may pass briefly through infeasible graphs rather than being walled inside the feasible region, and we return below to why that freedom is worth having. A move toggles one adjacency. The Metropolis rule accepts an improving move always and a worsening move with probability $e^{-\Delta E/T}$, and the temperature T falls geometrically. Algorithm 1 is the whole loop, the one place in this chapter where a body is worth showing, because the loop *is* the method.

This loop is what `solve` runs in its discovery mode. With the exhaustive flag off it anneals instead of enumerating, following the Metropolis walk of Algorithm 1 under geometric cooling, removals biased toward low-sensitivity edges, and returns the densest feasible graph it encountered together with the full temperature trace. What it returns is a witness, hence a lower bound, never a proof.

`search_for_dense_graph(n, m, variant, separation)` → `SearchResult` DISCOVER

in the size n, the ceiling m, the `Variant`, the separation, and a step budget

out a `SearchResult`: the densest feasible graph found, with the full step-by-step trace

how one Metropolis cooling run of Algorithm 1. The driver `best_of_searches` dispatches several independent runs across processor cores and keeps the densest, so a single local optimum never decides the answer.

It is fair to ask why the walk is allowed above the ceiling at all. Reaching every feasible graph does not require it. Removing an edge can only lower $\lambda^{\max}$ and $\kappa^{\max}$, never raise them, so every subgraph of a feasible graph is again feasible, and the empty graph is feasible trivially. From any feasible target we can therefore delete edges one at a time down to the empty graph without ever crossing the ceiling, and reading that path backward builds the target up the same way. The feasible region is connected through the empty graph by single-edge moves that stay inside it, so

a walk that never went infeasible could in principle visit every feasible graph there is.

We let it go infeasible anyway, because reachability and efficiency are different things. A feasible graph can be locally maximal, meaning every edge we might add creates a forbidden pair, and yet still sit far below the densest feasible graph. To improve such a graph the walk has to add an edge and then remove a different one, a swap whose first half is infeasible. A walk confined to feasible graphs could only find that better graph by retreating most of the way back to the empty graph and climbing a different slope, a detour so long that the cooling schedule usually ends first. Allowing a brief excursion above the ceiling lets the walk tunnel straight across instead, trading one load-bearing edge for a better one in two moves rather than dozens. The soft penalty also turns the search into a single smooth landscape with no hard walls for the Metropolis rule to bounce off, and the same machinery then carries over unchanged to variants whose feasible region is not so conveniently connected. Going above the bound is never the goal, then. It is the shortcut the annealer takes on its way back down to it.

3.3 Temperature and edge sensitivity

The phrase “biasing removals toward low-sensitivity edges” in Algorithm 1 is where the temperature acquires its meaning, and it is the heart of the method. For an edge e define its *sensitivity*

$$\sigma(e) = \lambda^{\max}(G) - \lambda^{\max}(G - e),$$

the amount by which deleting e relaxes the binding connectivity. A load-bearing edge sits on a tightest cut and has $\sigma(e) > 0$. Removing it would drop $\lambda^{\max}$ and so loosen the very constraint that limits how many edges the graph may hold. A free edge has $\sigma(e) = 0$: it can be removed without easing the binding pair, which usually means it can be put to better use elsewhere.

`edge_sensitivity(G, u, v)` $\to \mathbb{N}$ **MEASURE**

in a graph G and one of its edges $e = (u, v)$

out $\sigma(e) = \lambda^{\max}(G) - \lambda^{\max}(G - e)$

how delete e and let the checker remeasure, where a positive value marks a load-bearing edge. The whole-graph version `sensitivity_map(G)` reuses one shared baseline measurement. This single number is the dial the cooling turns.

When the search proposes a removal it draws the edge with probability proportional to $e^{-\sigma(e)/T}$. The dial is now explicit. At high temperature these weights flatten and the walk toggles edges regardless of how load-bearing they are, exploring widely and willing to dismantle essential structure. As the temperature falls the weights concentrate on $\sigma = 0$, so the walk almost never disturbs a load-bearing edge and instead rearranges only the free ones, until the graph crystallises onto an extremal shape. This is exactly the behaviour one wants, and it is the behaviour the formulation asks for: the temperature tracks how strongly removing an edge changes the bound. Figure 3.2 shows one such run settling onto a certified optimum. The other

variants behave identically, and their traces are collected in Section B.2.

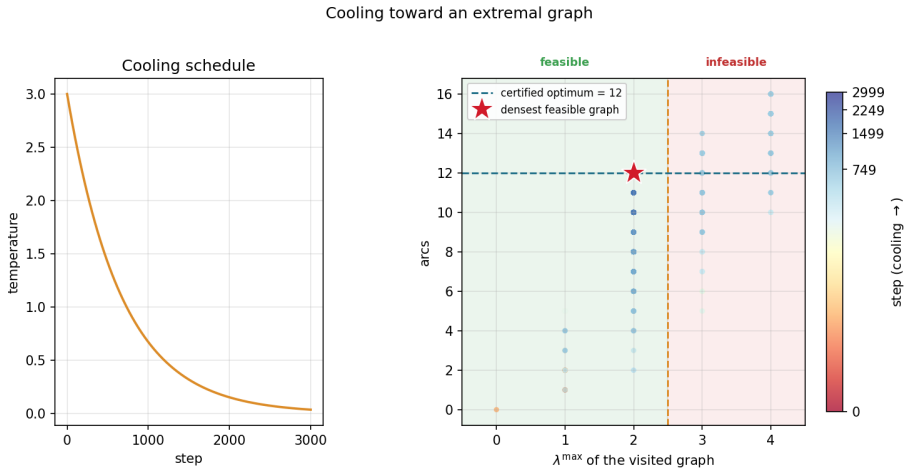


Figure 3.2. A single cooling run on the directed multigraph problem at $n = 4$, $m = 3$. *Left:* the temperature schedule, cooling geometrically from its starting value. *Right:* every visited graph plotted by its binding connectivity $\lambda^{\max}$ (horizontal) against its arc count (vertical), coloured by the step at which it was visited. Annealing uses a soft penalty rather than a hard wall, so while hot (dark points) the walk strays into the infeasible region $\lambda^{\max} > m - 1 = 2$, picking up extra arcs it is not allowed to keep. As the temperature falls (bright points) the penalty drives the walk back across the feasibility boundary and onto the densest graph that stays feasible, the certified optimum of 12 arcs at $\lambda^{\max} = 2$ (star).

The same sensitivities also explain a finished graph. Colouring a graph's arcs by σ shows at a glance which arcs are structural and which are slack. Figure 3.3 illustrates all four sensitivity levels on one directed multigraph: the darkest arcs carry the most flow and lower $\lambda^{\max}$ the most when removed, the cool arc contributes nothing. The picture is not decoration: it is the certificate of tightness made visible, and it tells the search exactly which arcs to leave alone as the temperature falls.

A single cooling schedule can still settle into a local optimum, so in discovery mode `solve` runs several independent schedules from different starting seeds and keeps the densest result across all of them. Because the restarts share no state and none depends on the outcome of another, they run in parallel. The program dispatches them across the available processor cores, so the wall-clock cost of k restarts is a small multiple of a single run rather than k times it, bounded below by the slowest restart and by the cost of starting the worker processes. A handful of parallel restarts is enough to reach the known optima at the sizes we examine. This is a reliability measure, not a change of method, and the parallelism is invisible to every number the thesis reports: the same best graph would be found by running the schedules sequentially, just more slowly.

3.4 Keeping the checker cheap inside the search

The energy of Algorithm 1 reads the binding connectivity $\lambda^{\max}(G)$, and a literal reading of the loop would call the checker afresh on the whole graph at every step. That checker is the most

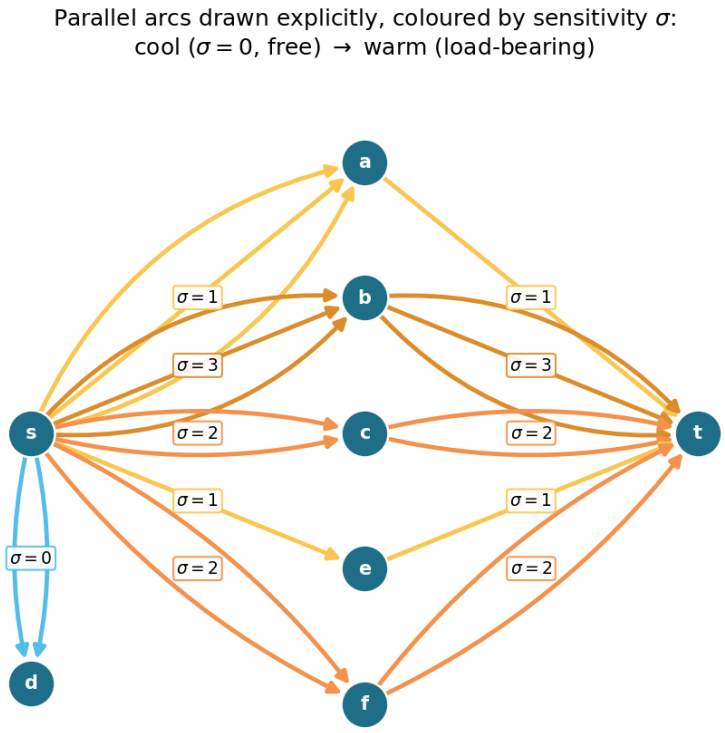


Figure 3.3. A directed multigraph on eight vertices with $\lambda^{\max} = 9$, every parallel arc drawn explicitly and coloured by its sensitivity σ, so multiplicity is read by counting arcs rather than from a label. The point is the contrast between $s \to a$ and $s \to b$: both carry three parallel arcs, yet $s \to a$ is cool ($\sigma = 1$) because the single arc $a \to t$ downstream throttles that route to one unit, while $s \to b$ is hot ($\sigma = 3$) because all three of its copies carry flow. The arcs $s \to d$ are coolest of all ($\sigma = 0$): d is a dead end, so deleting them changes nothing. Removing a hot adjacency drops $\lambda^{\max}$ by its σ while removing a cool one does not, which is exactly what tells the search which arcs to leave alone as the temperature falls.

expensive routine in the program, one maximum flow per ordered pair, and the walk runs for thousands of steps with several restarts, so such a search would spend nearly all of its time remeasuring graphs that barely changed from the step before. Two elementary facts remove almost all of that work without altering a single number the search reports.

Proposition 3.1 (Monotonicity of the binding connectivity)**.** *Let G be a graph or directed multigraph, and let G' be obtained from G by changing one adjacency by a single unit of multiplicity. Then*

- (monotonicity) *removing the unit cannot raise $\lambda^{\max}$, and adding it cannot lower $\lambda^{\max}$, and*
- (unit step) *adding the unit raises $\lambda^{\max}$ by at most one.*

Both statements hold verbatim for the vertex measure $\kappa^{\max}$.

Proof. Fix an ordered pair (s, t). By the max-flow / min-cut theorem $\lambda_G(s, t)$ is the smallest

capacity of any cut separating s from t, where the capacity of a cut is the total multiplicity of the arcs that cross it from the source side to the sink side.

For part (1), reducing one adjacency by a unit lowers the capacity of every cut that the changed arc crosses and leaves all other cut capacities untouched, so no cut capacity rises. The smallest cut capacity therefore cannot rise either, which gives $\lambda_{G'}(s,t) \leq \lambda_G(s,t)$. Taking the maximum over all pairs yields $\lambda^{\max}(G') \leq \lambda^{\max}(G)$. Adding a unit is the same statement read backwards.

For part (2), write G'' for G with one unit added to a single adjacency. That extra unit of capacity crosses any fixed cut at most once, so every cut has capacity at most one greater in G'' than in G. The smallest cut capacity thus grows by at most one, giving $\lambda_{G''}(s,t) \leq \lambda_G(s,t) + 1$, and together with part (1) we have $\lambda_G(s,t) \leq \lambda_{G''}(s,t) \leq \lambda_G(s,t) + 1$. The maximum over pairs gives $\lambda^{\max}(G'') \leq \lambda^{\max}(G) + 1$.

For $\kappa^{\max}$ the argument is identical on the vertex-split network of Figure 2.3, where changing an adjacency changes the capacity of its single arc $u^{\text{out}} \to v^{\text{in}}$ in the same way. Parallel copies leave that capacity at one and so change $\kappa^{\max}$ not at all, which only strengthens both parts. □

These two facts settle almost every step without a flow. The energy depends on $\lambda^{\max}$ only through the excess $\max(0,\ \lambda^{\max} - (m-1))$, which is zero exactly when the graph is feasible. So a removal applied to a feasible graph leaves it feasible by part (1), with excess still zero, and needs no measurement at all. An addition made while the graph sits strictly below the bound, at $\lambda^{\max} \leq m-2$, stays feasible by part (2), again with excess zero and no measurement. Only an addition made right at the boundary $\lambda^{\max} = m-1$ can tip the graph over, and only there does the search run a single flow, capped so that it stops the instant it finds one route too many. The exact connectivity value is recomputed only on the rare step whose proposal is genuinely infeasible, where the penalty term needs it.

To know which case applies, the search carries a running upper bound on $\lambda^{\max}$, raised by one whenever it accepts an addition (part (2)) and left untouched when it accepts a removal (part (1)), and resynchronised to the exact value at the cadence where it already pays for flows to refresh the sensitivity map. The bound can only ever sit at or above the truth, so any step it clears is certainly feasible.

The key point is that the energy returned on every step is, by Proposition 3.1, exactly the value the naive implementation would have computed. Every accept-or-reject decision, every random draw, and the graph the walk finally returns are therefore unchanged, and the speed-up is invisible to every number the thesis reports. This is verified rather than asserted. The regression test `test_fast_path_matches_exact` runs both implementations, the fast one and a reference that measures exactly at every step, across all four graph variants and both separations, and checks that the two agree step for step in energy, in every accept-or-reject decision, and in the final graph, and that the returned graph is feasible under the exact checker. Nothing about the optimisation hides inside the bound: the witness the search reports is always certified by an

Construction	n	m	Search	Theory
spanning tree (simple undirected, m=2)	6	2	5	5
multiplicity-$(m-1)$ star (multi undirected, m=3)	6	3	10	10
bidirected star (simple directed, m=2)	4	2	6	6
bidirected star (simple directed, m=2)	6	2	10	10
hub–bipartite tie (simple directed, m=2)	7	2	12	12
double star (multi directed, m=3)	4	3	12	12
double star (multi directed, m=3)	5	3	16	16

Table 3.1. Validation by rediscovery. For each variant the temperature search, run from scratch with a handful of restarts, recovers exactly the extremal value proved or certified in the earlier chapters. The "Search" column is an honest lower bound found by annealing, and the "Theory" column is the proved or certified value.

exact measurement, exactly as in the slower search.

It is worth noting where this method earns its keep. For undirected graphs one could resynchronise the exact value cheaply from the Gomory–Hu tree of Chapter 2, which encodes every pairwise edge-connectivity in $n - 1$ flows. The directed and vertex variants admit no such tree, so there the monotone bound and the capped predicate are the only inexpensive handle on feasibility, and they are what let one annealer serve every variant at speed.

3.5 Rediscovering the known extremisers

A search method is only worth trusting if it recovers what is already known before it is pointed at what is not. Run from the empty graph on the cases whose answers Chapter 1 and Chapter 2 settled, and told only to pack in edges without breaching the connectivity ceiling, the cooled search arrives unprompted at the named constructions. Table 3.1 reports the densest graph it found against the value the theory predicts, and in every case the two agree.

What the table hides is that the graphs themselves are the named constructions, not merely graphs of the right size. The simple undirected run at $m = 2$ settles on a spanning tree, the only shape with $n - 1$ edges and no cycle. The undirected multigraph run settles on a star with every edge at multiplicity $m - 1$, an instance of the multi-tree extremiser behind Theorem 1.3. The directed multigraph runs settle on the double star of Chapter 2, both directions of every spoke at full multiplicity. The simple directed runs find the bidirected star at $n = 4$ and, at $n = 7$, land on twelve arcs, the value at which the hub and the one-directional wall tie, exactly the crossover Theorem 1.9 predicts. The cooled search does not know these names. It rediscovers the structures because, under the connectivity ceiling, there is nowhere denser to go.

Some extremisers are past the size at which a quick search lands reliably, but the exact checker of Chapter 2 settles them immediately by measuring their connectivity. The one-directional bipartite digraph on eight vertices has 16 arcs at $\lambda^{\max} = 1$, and 16 already exceeds the linear hub value $2(n - 1) = 14$: the smallest size at which the quadratic phenomenon strictly wins, made concrete. It is also a measured illustration of the search's limit: repeated cooling runs

at $n = 8$, $m = 2$ stall at the hub value of 14, because dismantling a finished hub and rebuilding a one-directional wall is far more than the single-arc moves of the walk can tunnel through. The known answer there comes from the construction and the checker, not from the search, which is exactly the division of labour this chapter argues for. The augmented bipartite digraph on ten vertices at $m = 3$ has 30 arcs at $\lambda^{\max} = 2$, the thirty-arc counterexample of Remark 1.11, confirmed pair by pair against the naive bound of 27 it refutes. The directed double star on seven vertices at $m = 4$ has 36 arcs at $\lambda^{\max} = 3$, matching $2(n-1)(m-1)$. None of these is a proof of optimality. Each is an exactly measured feasible graph, a lower bound the checker stamps as genuine.

3.5.1 Simulated annealing versus tabu search

The annealer is not the only way to walk this landscape. A natural alternative, suggested by Jan Goedgebeur, is *tabu search*, implemented as `tabu_search_for_dense_graph`, which keeps the energy and the neighbourhood of Algorithm 1 but replaces the cooling schedule with a deterministic best-improving step. At each step it evaluates every single-edge move from the current graph and takes the one that lowers the energy most, then forbids the pair it just changed for a fixed number of steps, the *tabu tenure*, so the walk cannot immediately undo its last move and stall in a two-step cycle. When no improving move is left and the search stalls, it perturbs the incumbent with a few random moves and carries on. There is no temperature and no accept-worse coin: apart from the perturbation kicks the walk is deterministic. Where annealing explores by occasionally stepping uphill and trusts the cooling to settle it, tabu climbs greedily and trusts the tabu list to keep it from retracing its path.

The two engines rediscover the same values, but at very different speeds, and on the harder directed cases the difference is decisive. Table 3.2 runs them head to head at an equal budget of six seconds each, both restarted within that budget. On the simple undirected problem both reach the optimum at once. On the two directed problems past the certified range tabu reaches the optimum within the budget while the annealer plateaus below it, assembling the eighteen-arc simple-directed extremiser and the full twenty-four-arc double star at $n = 7$ where the annealer stalls at sixteen and twenty. The cause is the one already met at the $n = 8$, $m = 2$ hub stall above. The annealer cools onto whatever dense shape it first reaches and then cannot dismantle it within the budget, whereas tabu's best-improving step follows the steepest density gradient and the tabu list stops it sliding back, so it assembles the structured extremiser the cooled walk struggles to find.

Figure 3.4 shows the divergence directly, plotting the densest feasible graph each engine has found against wall-clock time on the two directed multigraph cases. Tabu climbs in a few decisive steps and holds at the optimum, while a single annealing schedule rises quickly and then flattens a few arcs below it. The annealer's independent restarts recover the optimum on the smaller case, as Table 3.1 records, so a single plateau is not the whole story there. But at $n = 7$ the plateau persists within the budget, and one tabu schedule already reaches what the restarted annealer does not.

			best in 6 s		time to optimum	
Variant	n	optimum	SA	tabu	SA	tabu
simple undirected	7	9	9	9	1.3 s	0.1 s
simple directed	7	18	16	18	—	2.6 s
directed multigraph	7	24	20	24	—	2.5 s

Table 3.2. Simulated annealing against tabu search at an equal six-second budget per method, the annealer restarted from fresh seeds and tabu restarted on stalls. Only the simple undirected value is a proved optimum (Mader). The two directed values are the best known rather than proved, found by search and construction past the certified range that stops below $n = 7$ for those variants (Section 3.6), so here the "optimum" column is the target each engine is trying to reach. Both reach the simple undirected optimum. On the two directed problems tabu reaches the best known value while the annealer plateaus below it. The last two columns give the wall-clock time each method first reached that value on the reference machine, with a dash where it did not reach it in the budget.

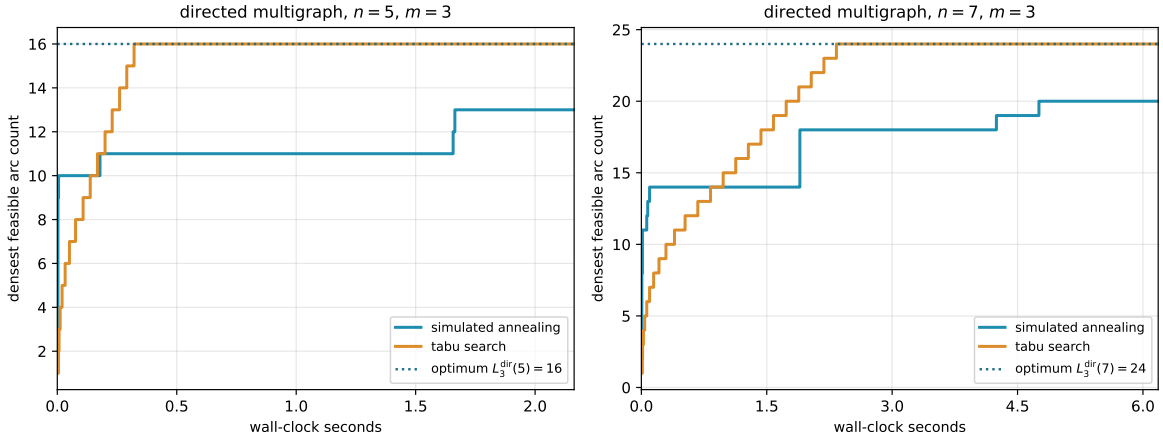


Figure 3.4. Densest feasible graph found against wall-clock time, for one simulated-annealing schedule and one tabu schedule on the directed multigraph at $n = 5$ and $n = 7$, $m = 3$. Tabu climbs in a few best-improving steps to the optimum (dotted) and holds, while a single annealing schedule rises quickly and then flattens below it. The time axis is wall-clock on the reference machine, so unlike the seed-reproducible figures elsewhere this one is a representative timed run.

This advantage comes at a cost the equal-time comparison already absorbs but is worth naming: each tabu step scans the whole neighbourhood, so it is far more expensive than the annealer's single random proposal, and tabu takes many fewer but better-aimed steps in the same wall-clock. Tabu search is therefore the program's default for solving the problems, reaching every value this thesis reports, the rediscovery of Table 3.1 and the conjectural lower bounds alike, and reaching the harder directed optima past the certified range where the annealer plateaus. Simulated annealing is kept for the self-check and verification, where the small values fall at once, the lighter per-step cost is all that is needed, and its independent restarts parallelise trivially across cores.

3.6 The trichotomy: proved, certified, conjectured

It is worth stopping to lay out, for every variant, the three kinds of knowledge the computation can hold about it, because the whole value of the pipeline is that they are kept apart and never quietly merged. About any given value the program can do one of three things. It can *prove* the value, closing a fixed size from above by exhausting every graph of that size, an upper bound with the standing of a hand proof. It can *certify* it, the same kind of upper bound but obtained by the cut-counting optimisation rather than blind exhaustion, reaching a vertex or two further. Or, where proof and certificate both run out, it can only *conjecture* the value, exhibiting a concrete dense graph that pins down a lower bound and leaving the matching upper bound to a later argument.

Proof and certificate bound the answer from above. The search bounds it only from below. Where a closed *formula* exists it can tie the two together, threading through the certified small cases and the conjectured large ones as a single curve. This is the discipline the whole thesis runs on, and it is worth stating as one: the search proposes a lower bound, a certificate or a hand proof disposes the upper bound, and only when both meet is a value called settled.

We begin with the single variant where all three kinds of knowledge are non-trivial and visibly distinct, the directed simple arc problem at a fixed $m \geq 3$, and then sweep the whole family. For that variant the knowledge splits cleanly by the size of the graph.

For small n the answer is not conjectured at all. The pruned exhaustion inside `solve` walks every simple digraph on n vertices and returns the exact maximum, complete and certain, an upper bound with the standing of a hand proof. At $m = 3$ it certifies $6, 9, 12$ for $n = 3, 4, 5$, and there the exhaustion stops within a reasonable budget, the pruned walk no longer closing $n = 6$ in tractable time. That is the certified region, finite and closed.

Beyond it the ceiling is gone and only lower bounds remain, each witnessed by a concrete feasible graph. The search finds the extremal 15 arcs at $n = 6$ on its own. Past that size it falls behind: across six restarts it reaches $17, 17, 20, 23$ for $n = 7, \ldots, 10$, while the named constructions of Chapter 1, measured exactly by the checker, supply $18, 21, 25, 30$, the last being the thirty-arc graph of Remark 1.11. The reported lower bound at each size is the better of the two, exactly as the rediscovery section combined search and checker by hand. These are conjectures with an honest floor and no matching ceiling. That is the searched region, unbounded and forever one-sided.

The two regions are tied together by a single closed form. The conjectured value

$$\ell_m^{\mathrm{dir}}(n) = \max\left(m(n-1),\ \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor \right)$$

of Conjecture 1.13 is not a fresh guess bolted on past the certified range. It is one formula that already passes through every certified value at small n and then continues, without a seam, through every searched value at large n. The directed-arc panels of Figure 3.5 show this directly, the certified squares at the small sizes and the open lower-bound circles beyond both sitting

on the one red conjecture curve, proof below the formula and witnessed graphs above it. The formula is the through-line that makes the small certified cases and the large conjectured ones a single statement rather than two.

Two features of those directed-arc panels are worth naming. The crossover from the linear hub term to the quadratic bipartite term happens at $n = 9$, where the second argument of the maximum first wins, and that is past the last certified size. The shape of the answer therefore changes in exactly the region where only the search can see it, which is precisely why a discovery tool was needed and a certifier alone would not do. And at $n = 10$ the formula reads 30, the count of the very counterexample that overturned the natural initial conjecture, so the closed form is not a smooth fit to easy data but one that survives the case that broke the previous guess.

Asking the same three questions of every variant, the answers sort into three regimes, and Figure 3.5 shows all twelve at once, one panel per variant, in exactly this proved, conjectured, and guessed language.

In the first regime the closed form is already a theorem for every n, proved by hand in Chapter 1 or cited, and the computation only confirms it. The undirected families live here, edge and vertex alike, together with the directed problems at $m = 2$. For these the certified small cases and the rediscovered constructions of Table 3.1 are validation, not new knowledge: the formula was never in doubt, and the program's role is to check that the pipeline reproduces it. That parallel edges do not help for vertex-connectivity, and that the directed vertex multigraph reduces to the simple digraph, are settled by hand, so several rows here need no computation of their own at all.

In the second regime the three columns genuinely differ and the computation is load-bearing. This is the regime of the worked example above. The directed simple arc and vertex problems at $m \geq 3$ are certified only at the small n the exhaustion reaches and conjectured by the search beyond, with the formula of Conjecture 1.13 the through-line.

The directed multigraph arc problem sits here too, with one extra economy: in its cut-counting mode `solve` proves the m-free number $M^*(n) = 2(n-1)$ outright for $n \leq 6$, and the scaling identity $L_m^{\mathrm{dir}}(n) = (m-1)\,M^*(n)$ of Chapter 2 carries that one certified number to every m at once, so the closed form $2(n-1)(m-1)$ is proved across the whole m axis at those sizes. Past the crossover at $n = 7$ the conjectured value switches to the bipartite branch $(m-1)\left\lfloor n^2/4 \right\rfloor$, where the even sizes are proved conditional on the odd ones and only a single mixed-regime minimum-degree statement keeps the induction from closing (Appendix A).

In every row of this regime the formula agrees with proof where proof reaches and with the search where it does not, exactly the linkage the worked example displayed.

In the third regime there is no formula at all, and the honesty is in drawing the guess as a guess. The undirected vertex problem at $m \geq 6$ has been open since 1974 with its extremal structure genuinely unknown, and the hypergraph variants beyond the proved edge case are unexplored, the directed ones needing a one-tail, many-head reading of a Berge edge that this

thesis defines itself only so that `solve` can measure it at all. Here the program still computes exact values at the few sizes that finish, and its search still exhibits feasible graphs at a handful more, but no closed form is in sight. The panels of Figure 3.5 carry, instead of a red conjecture curve, a yellow line that simply interpolates the machine points in straight segments, trusted no further than the eye. We draw it *through* the points rather than fitting a smooth curve above them: the points are honest lower bounds, the truth lies on or above them, and a line riding above the points would claim more than the search ever found.

Two cautions govern how to read the dotted curves. At small n the complete graph or hypergraph is itself feasible, its binding connectivity being only $n-1$, so the earliest points merely trace the trivial maximum and reveal nothing of the open behaviour: the undirected-vertex points run up $\binom{n}{2}$ until n passes m, and only then bend toward the linear growth the proved edge value predicts.

Past that bend the dotted line is only as strong as the quick search behind it. Exactly as the search stalled on the quadratic wall in the rediscovery above, the lower bound it reports climbs only linearly where it cannot build a denser construction. The clearest case is the directed hypergraph, whose bipartite family is quadratic, reaching $\sim (m-1)n^2/\big(4(r-1)\big)$ hyperarcs by Proposition A.41: there the search slips below that construction at larger n, so the dots understate the truth rather than tracing it. We therefore read the dotted trends as honest lower bounds and not as fitted laws, and the one closed-form hypothesis we record, the quadratic for the directed hypergraph, comes from the construction and not from the shape of the dots.

So to the question of whether one formula links what is proved to what is conjectured, the answer is three-valued and depends on the variant, and the grid of Figure 3.5 shows all three at a glance. Where the problem is already solved the curve is blue, a theorem the computation merely agrees with. Where the problem is open but tractable the curve is red, a conjecture the certified squares confirm below and the searched circles confirm above, and that is the case the program was built for. Where the problem is genuinely hard the curve is yellow, a guess interpolated through the search points and an open question of what, if anything, the dots truly trace. One feature stands out in the bottom row: the directed hypergraph panels, both arc and vertex, climb steeply at $m = 3$, the yellow guess curving upward rather than running straight. This is genuine and not search instability: the directed Berge edge packs a tail and $r-1$ heads into a single object, and the bipartite construction of Proposition A.41 already reaches roughly $(m-1)n^2/(4(r-1))$ hyperarcs, so the value grows with n^2. The same quadratic shape sits one column over, in the directed arc and vertex panels, whose conjectured red curves carry the $\left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor$ term and rise just as fast. Super-linear growth is the signature of *direction* across the grid, not of the hypergraph alone: every directed column bends upward while the two undirected columns stay linear. Figure 3.6 redraws the same grid at $m = 6$: the proved curves climb linearly with m exactly as their formulas predict, while the open variants' search points and their yellow interpolation climb higher without a proved curve to meet, so raising the threshold does not change which regime a variant lives in, only how far the red and yellow curves have travelled.

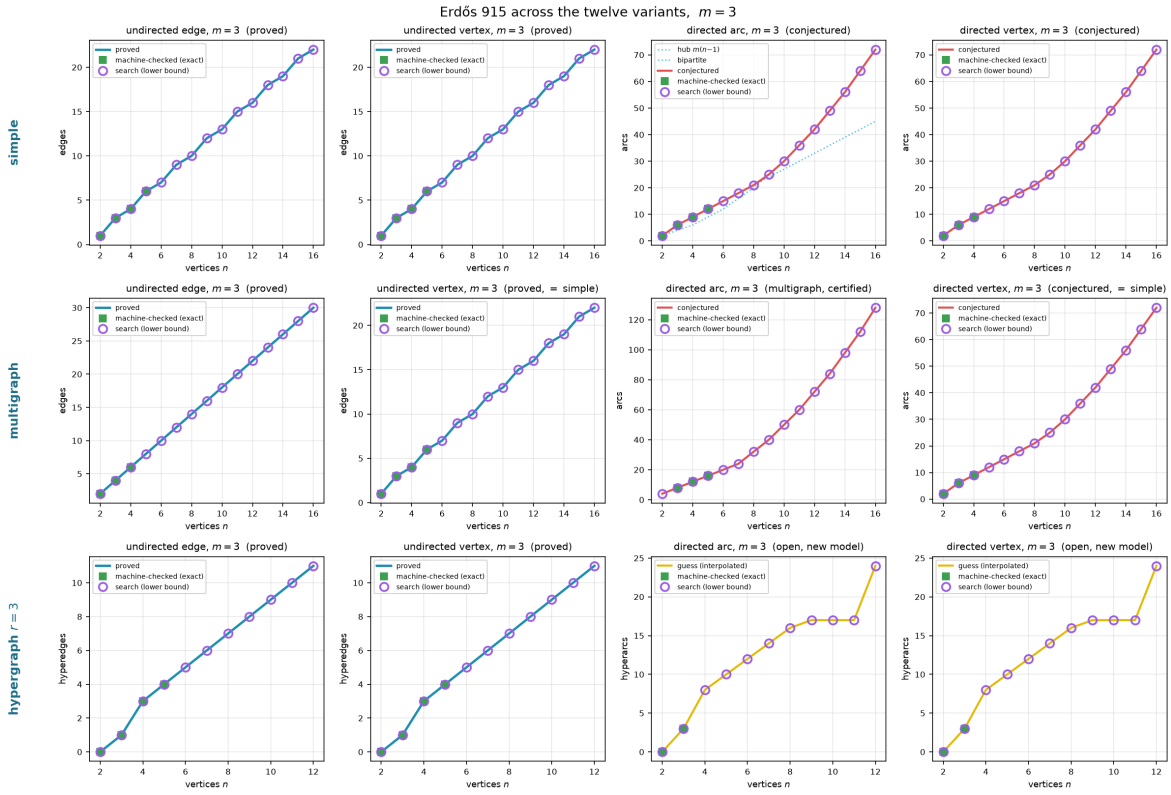


Figure 3.5. All twelve variants at $m = 3$, edges against n, laid out by model (rows: simple, multigraph, 3-uniform hypergraph) and by separation and direction (columns: undirected edge, undirected vertex, directed arc, directed vertex). A *blue* curve is a value proved for all n, a *red* curve a conjectured formula, and a *yellow* curve a bare interpolation of the machine points where no formula is known, the three told apart by colour rather than by dash pattern. Filled squares are the sizes `solve` proved or certified exactly. Open circles are lower bounds at larger n, the better at each size of the search's densest find and a named construction measured by the checker, reported as a running maximum so a feasible graph at one size is never lost at the next. On the open variants the yellow curve is the piecewise-linear interpolation *through* those circles, never above them, and the axes are scaled to the data rather than to a loose certain interval. The directed hypergraph panels in the bottom row rise steeply because their value is quadratic in n, not linear.

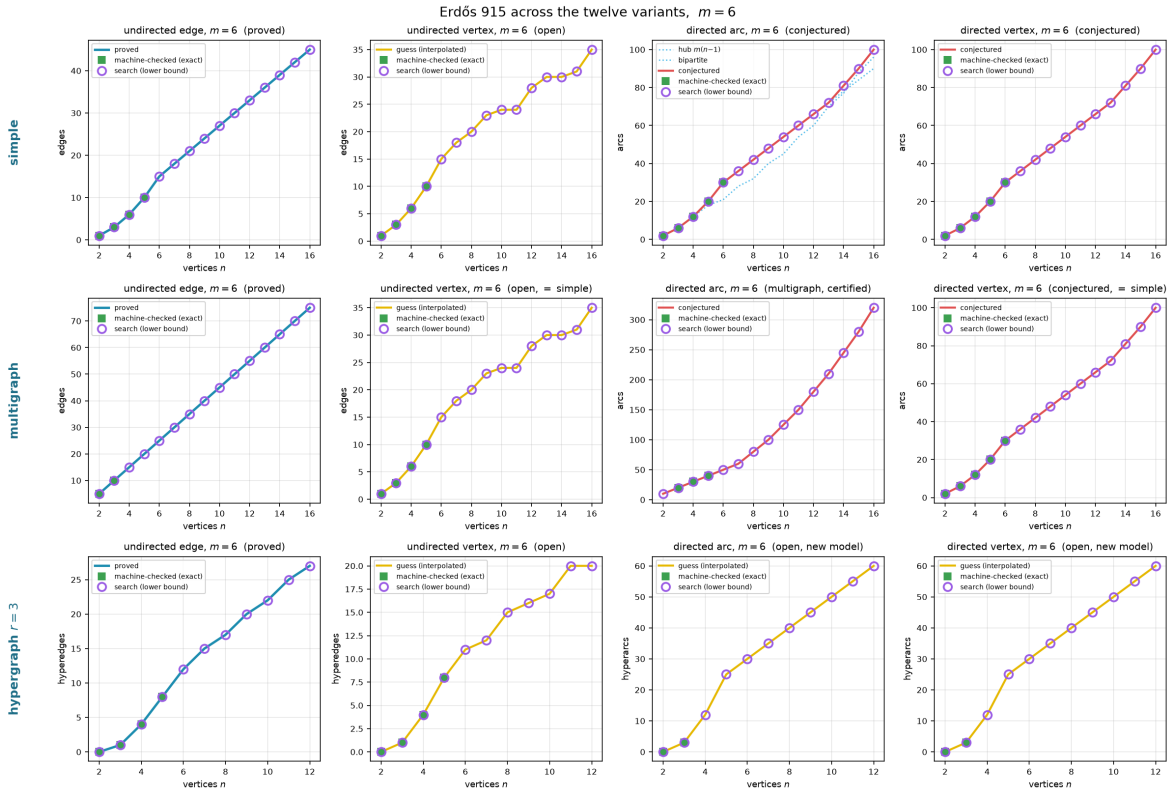


Figure 3.6. The same twelve variants at $m = 6$. The undirected edge and multigraph edge cases remain proved (Mader's theorem holds for all m), while the undirected vertex separation is the famous open problem at $m \geq 5$ and the directed cases carry the same conjectured formula scaled to $m = 6$. Comparing this figure with the $m = 3$ version above shows that the proved cases grow linearly with m as the formulas predict, while on the open cases the search circles and their yellow interpolation climb steadily without a proved curve to meet.

The same grid answers the harder question the trichotomy raises, which is what the search is worth where no upper bound exists to check it against. It says two things at once. First, wherever an upper bound exists, proved or certified, the search lands exactly on it and never above it at every plotted size, which is strong evidence that the search is not leaving edges on the table, with the one instructive exception recorded above: the directed $m = 2$ run at $n = 8$, where it stalls on the hub and the bipartite optimum must be supplied by the construction instead.

Second, on the directed arc problem at $m \geq 3$, where no upper bound is known past the certified range, the plotted lower bounds land exactly on the conjectured value $\max(m(n-1), \lfloor (n+m-2)^2/4 \rfloor)$ at every reachable n, and the certified points agree with it as far as they go, but past the first open size it is the constructions, not the cooled walk, that carry the bounds there. Run from scratch the search confirms the conjecture at $n = 6$ and then plateaus a few arcs short ($17, 17, 20, 23$ against $18, 21, 25, 30$ at $n = 7, \ldots, 10$, across six restarts), for the same reason as the $m = 2$ stall recorded earlier: the extremal shapes past the crossover are bipartite walls, and a walk that has crystallised onto a hub cannot rebuild itself into a wall by single-arc moves within the cooling budget.

So to the question "do we believe the reported lower bounds are best possible," the grid answers: wherever a proof or a certificate can check them, yes, and on the open sizes they sit on the only value anyone has conjectured, with the search confirming the first open size and the constructions carrying the rest. The gap between those lower bounds and what is proved is exactly the gap between the conjecture and the missing upper bound, which Chapter 4 names precisely. The search has not closed that gap, and it cannot. What the pipeline supplies is reproducible evidence for which side of the gap the truth lies on.

Honesty

A number found by the search is reported as a lower bound, and is marked as certified or proved only when a separate cut-counting optimisation or a hand proof supplies the matching upper bound. No extremal value is ever asserted on the strength of the search alone. The figures in this chapter use fixed random seeds, so every search point is reproducible.

Chapter 4

Synthesis, Results, and Open Problems

The journey set out from one clean theorem and followed it across twelve variants, proving what proved cleanly, building a machine where the proofs ran out, certifying the small cases the machine could close, and using the search to discover the dense graphs the proofs had singled out and to point at the ones still open. This closing chapter draws the threads together. It maps the directed frontier, where proof and certificate both still fall short of the full answer, names the two genuine surprises the landscape contains, gathers the twelve answers into one picture, and says plainly what the program changed and where it stops.

4.1 The directed frontier and the single open lemma

Two of the open directions are directed, and they share a structure worth stating exactly rather than hiding. The directed arc problem is a race between two constructions on different scales. The directed hub (Construction 1.8) reaches $m(n-1)$ arcs and grows linearly. The augmented bipartite construction (Construction 1.12) packs $\lfloor (n+m-2)^2/4 \rfloor$ arcs and grows quadratically. Which leads depends on n, and the two cross at an order linear in m, near $n = 9$ at the $m = 3$ of Figure 4.1. Conjecture 1.13 asserts that together they are the whole story.

Conjecture 1.13 is proved as a lower bound for all m, and it is a theorem at $m = 2$ by Theorem 1.9. Past $m = 2$ it sits squarely in the second regime of the trichotomy of Section 3.6: certified by exhaustion at the small sizes `solve` can close for $m = 3$, conjectured by the temperature search beyond, and never beaten anywhere the search reaches. What is missing is a general upper bound for $m \geq 3$, and the obstacle is structural rather than computational.

The route to the upper bound is to show that an extremal digraph, once it is past the hub regime, must look like the augmented bipartite construction: its vertices split into a part A and a part B with every arc running $A \to B$, and B thickened only mildly. Three structural facts, each proved in Appendix A, push in this direction. Out-neighbourhoods are mutually unreachable, second out-neighbourhoods are disjoint, and a complete $A \to B$ partition forces each vertex of B to absorb at most $m-2$ arcs from inside B. With the partition free to vary, the total arc count $|B|(|A|+m-2) = |B|(n-|B|+m-2)$ is maximised at $|B| = \lceil (n+m-2)/2 \rceil$, which gives $\lfloor (n+m-2)^2/4 \rfloor$, precisely the conjectured upper bound.

The reduction is clean except at a single point, and it is worth naming the point exactly rather

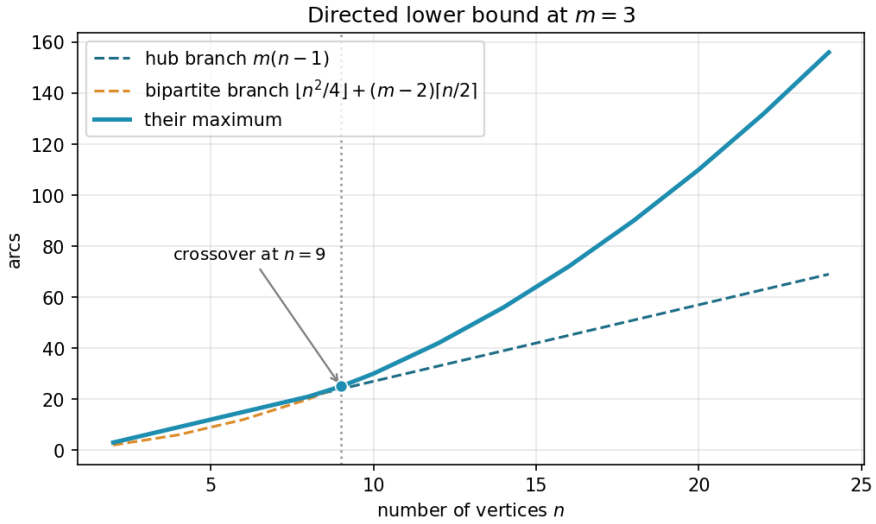


Figure 4.1. The two directed lower bounds at $m = 3$. The linear hub branch leads for small n. The quadratic augmented-bipartite branch overtakes it near $n = 9$ and leads thereafter. Conjecture 1.13 asserts their maximum is the exact value.

than hiding it. The count goes through provided the extremal digraph has *no arc from B back to A*: under that hypothesis a route between two vertices of B can never escape B, so the connectivity inside B is governed entirely by B's own arcs and the budget applies. The bare existence of a complete $A \to B$ partition is not enough on its own, because a small example shows a backward arc can manufacture an extra route. So the whole upper bound for $m \geq 3$ now rests on one missing lemma: that an extremal non-hub digraph has no backward arc. The natural delete-and-compensate exchange that would prove it is not monotone, which is the precise reason it remains open.

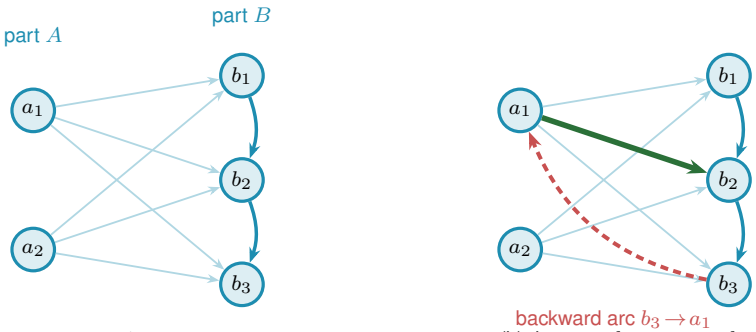
The machine has been useful here in the negative direction too: a proposed counterexample to the conjecture, a backward arc bolted onto the thirty-arc graph, was refuted by the checker, which found the third route the construction's author had missed.

The directed vertex-disjoint problem needs no separate machinery. Vertex-disjoint routes are at most as numerous as arc-disjoint ones, and the constructions that achieve the arc bounds also achieve them for vertices, so the two directed problems share both their constructions and their conjecture, and the $m = 2$ value transfers exactly in Theorem 1.10. For $m \geq 3$ the vertex value is conjectured equal to the arc value, and it will follow from the arc case the moment the missing backward-arc lemma is supplied. The directed multigraph problem has its own two-branch story, parallel to the simple one. The conjectured value is

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\,\max\!\Big(2(n-1),\; \Big\lfloor \frac{n^2}{4} \Big\rfloor\Big),$$

where the double star realises the linear branch and the one-directional bipartite multigraph, every arc at multiplicity $m-1$, realises the quadratic one.

The quadratic branch here uses the balanced partition $|A| = \lfloor n/2 \rfloor$, $|B| = \lceil n/2 \rceil$, giving



(a) every arc $A \to B$, B lightly thickened, none returns to A

(b) the route $b_3 \to a_1 \to b_2$ now escapes B through A

Figure 4.2. The directed frontier reduction and the single obstruction. **(a)** The structure the upper bound wants. Every arc runs from A to B (faint), B carries only a light internal thickening (the $m-2$ budget, dark blue), and no arc returns to A, so a route between two vertices of B never leaves B and the count $|A||B| + (m-2)|B|$ is complete. **(b)** A single backward arc $b_3 \to a_1$ (red) breaks this. The route $b_3 \to a_1 \to b_2$ (red leg then green leg) now escapes B through A, an extra route the budget never counted. Ruling out backward arcs in an extremal digraph is the one missing lemma behind the $m \geq 3$ upper bound, and the checker refuted a proposed counterexample by finding exactly such a hidden route.

$(m-1)\lfloor n^2/4 \rfloor$ arcs. This is distinct from the simple-directed conjecture above, whose optimal partition shifts to $|B| = \lceil (n+m-2)/2 \rceil$ at $m \geq 4$ but whose single-multiplicity arcs cannot benefit from scaling by $m-1$. The two formulas agree for $m \leq 3$ and diverge only at $m \geq 4$, where the multigraph's multiplicity advantage dominates the partition shift. The branches cross at $n = 7$ for every m, because the factor $m-1$ cancels, unlike the simple case, where the crossing point grows with m and already sits near $n = 9$ at $m = 3$.

On the linear side the certifier closes the problem, proving $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$ for $n \leq 6$ and every m at once (Theorem 2.1), and at $m = 2$ the whole problem collapses onto the simple digraph theorem for every n, because two parallel arcs are already two routes (Theorem A.24). On the quadratic side, a scaling-and-deletion induction proves the bipartite bound for even $n \geq 8$, conditional on the bound one vertex below. The odd case once reduced to a single minimum-degree statement (Conjecture A.22), which the program confirms in the purely saturated and purely unsaturated regimes but leaves open in the mixed one, where multiplicities strictly between 0 and $m-1$ defeat the integrality the two pure regimes lean on (Appendix A).

A sharper route now bypasses that conjecture at $m = 3$, and the way it does so is the cleanest instance in the thesis of the proof handing its remaining work to the machine. An *attachment lemma* (Lemma A.27) bounds the degree of a vertex glued onto the bipartite extremiser, and a *conditional odd step* built on it (Theorem A.29, for $m \in \{3, 4\}$) carries the value and extremal characterisation from each even level to the next odd one. At $m = 3$ that characterisation propagates through both parities (Remark A.30), so the entire problem for all $n \geq 8$ collapses onto two finite, integral statements at $n = 7$:

(a) $L_3^{\mathrm{dir}}(7) = 24$, and

(b) the only 24-arc feasible multigraphs on 7 vertices with maximum degree at most 8 are $2 \cdot B_{3,4}$,

$2 \cdot B_{4,3}$, and the doubled bidirected path P_7.

Conjecture A.22 is thereby off the critical path for $m = 3$, replaced by two machine-checkable targets: a MILP infeasibility check for fact (a), and an exhaustive enumeration of degree-bounded multigraphs for fact (b). Both stay within memory but remain runtime-limited, and neither has closed yet.

The linear branch turns out to be richer than the double star first suggested. Every doubled bidirected spanning tree is extremal, confirmed exhaustively at $n = 4$ and 5, and several distinct extremisers tie at 24 arcs for $n = 7$. Sifting and classifying them is precisely what the enumeration must do.

For $m \geq 4$ the conditional step still yields the value but uniqueness no longer propagates through odd levels, and for $m \geq 5$ even the value step stalls. These residual gaps are stated precisely among the open problems below. They are finite, identified targets rather than mysteries, and natural goals for the next round of certified computation.

4.2 Two principal phenomena

Across all the variation, two phenomena stand out as the ones a first guess would not have predicted.

The first is the divergence at $m = 5$. The edge and vertex problems agree for $m \leq 4$, and it would be natural to expect them to agree forever. Instead they part company at the very next value, with $k_5(n) = \lfloor 8n/3 \rfloor - 3$ pulling strictly above $\ell_5(n)$, and beyond it the vertex problem becomes genuinely hard and has stayed open since 1974.

The second is the quadratic bipartite phenomenon. In an undirected graph every edge contributes to the degree of both its endpoints, and Mader's theorem turns that symmetry into a hard ceiling: $\lfloor m(n-1)/2 \rfloor$ edges, linear in n. A digraph breaks the symmetry. Place all n vertices into two equal parts A and B and draw every arc from A to B, one direction only. The result has $\lfloor n^2/4 \rfloor$ arcs, quadratic in n, yet $\lambda^{\max} = 1$: every pair has at most one arc-disjoint route, because every route must cross the A-to-B wall exactly once and no arc crosses the other way to offer a second path. A graph with $\Theta(n^2)$ arcs can sit at connectivity one, something no undirected graph can do past a handful of vertices.

The initial conjecture for the directed case was linear, matching the undirected intuition. The one-directional bipartite construction refutes it outright, already at $m = 2$: for $n = 8$ the wall holds 16 arcs while the linear hub holds only 14. At $m = 3$ the refutation is the thirty-arc augmented bipartite graph of Remark 1.11, which packs 30 arcs at $\lambda^{\max} = 2$ against the naive prediction of 27. The construction generalises to every m via Construction 1.12, thickening the larger part B slightly so that each vertex of B absorbs $m - 2$ extra in-arcs from inside B: those extra arcs add exactly $m - 2$ short detours from A to each $b \in B$, bringing $\lambda^{\max}$ from 1 to $m - 1$ while the arc count climbs to $\lfloor (n + m - 2)^2/4 \rfloor$.

The quadratic term is what makes the directed cases the deepest in the thesis. The upper bound argument must explain why no configuration beats the bipartite wall, which requires ruling out backward arcs and controlling the internal structure of the large part B, and that structural task is exactly what the backward-arc lemma of Chapter 1 has not yet resolved for $m \geq 3$. The undirected problem has no analogous obstacle, because Mader's degree-counting argument closes it in a few lines. The quadratic phenomenon is therefore not just a curiosity: it is the geometric reason the directed and undirected problems live on different levels of difficulty.

That same scarcity of extremal graphs is the thread through the remaining figures. Figure 4.3 shows where the frontier runs: it plots every labelled graph at small n as a single point, its binding connectivity $\lambda^{\max}$ on the horizontal axis and its edge count on the vertical, so the whole population of graphs is visible at once. The points crowd into the low-connectivity, low-edge corner and thin out upward, where the blue extremal envelope, an integer staircase carrying one dot per connectivity level, traces the densest graph available at each level. The empty region above that staircase is precisely the part the main theorems forbid a graph to occupy. The extremal problem is the question of where that staircase runs, and the figure shows it is a thin frontier with almost nothing pressed against it from below.

Figure 4.4 steps inside each graph to the level of individual pairs. For every graph in the full enumeration it records the connectivity of every vertex pair and pools all the observations, so the histogram counts not graphs but (graph, pair) observations. Each bar is split and stacked by the connectivity of the graph the pair came from: blue for pairs drawn from the lower-connectivity graphs, red for pairs from the higher-connectivity ones. The boundary between the two is set per variant at the midpoint of the connectivity range that variant's enumeration can reach, since the twelve variants span very different ranges and a single fixed boundary would leave some panels entirely one colour. A pair whose own connectivity exceeds the boundary can only come from a graph above it, so the bars to the right of the dashed line are wholly red, while to the left the two colours mix, since a high-connectivity graph still carries many low-connectivity pairs. The blue mass sits almost entirely at the lowest levels: high-connectivity pairs are rare even inside the graphs that have one.

Figure 4.5 then shows how the same graphs distribute by sheer edge count, with the same per-variant stacked colouring. At each edge count the blue part counts the lower-connectivity graphs and the red part the higher-connectivity ones, so the bar height is the total number of graphs there and the split shows how the two populations separate. The red mass sits to the right, since denser graphs tend to carry higher connectivity, and the dotted line marks the densest graph still on the low-connectivity side, landing far out in the sparse tail. The densest low-connectivity graphs, the ones the extremal problem is about, are exceptional in every variant.

Finally, Figure 4.6 lifts the whole study onto the (n, m) grid at once, drawing the best feasible edge count as a bar surface across every variant. The three proved variants rise as smooth linear or quadratic sheets that the formulas describe exactly, while the open variants show a visible step between the machine-proved blue bars and the search's purple lower bounds, so the

Separation	Model	Value	Status
Undirected			
edge	simple	$\lfloor m(n-1)/2 \rfloor$	proved (Mader)
edge	multi	$(m-1)(n-1)$	proved
edge	hyper	$\lfloor (m-1)(n-1)/(r-1) \rfloor$	proved (cited GH)
vertex	simple	$\lfloor m(n-1)/2 \rfloor$ ($m \leq 4$), $\lfloor 8n/3 \rfloor - 3$ ($m = 5$, $n \geq 10$)	cited, open $m \geq 6$
vertex	multi	parallel edges do not help	proved
vertex	hyper	$\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ ($m \leq 3$), $\geq$ that for all m	proved $m \leq 3$ (Theorems A.36 and A.39), open $m \geq 4$
Directed			
arc	simple	$\max(2(n-1), \lfloor n^2/4 \rfloor)$ ($m = 2$)	proved, conj. $m \geq 3$
arc	multi	$(m-1)\max(2(n-1), \lfloor n^2/4 \rfloor)$	proved $n \leq 6$ and $m=2$, cond. $m=3$
arc	hyper	quadratic in n, exact value open	lower bd. (Proposition A.41)
vertex	simple	$\max(2(n-1), \lfloor n^2/4 \rfloor)$ ($m = 2$)	proved, conj. $m \geq 3$
vertex	multi	reduces to simple digraph	proved
vertex	hyper	quadratic in n, exact value open	lower bd. (Proposition A.41)

Table 4.1. The twelve structural variants of Problem 915 and their status. The threshold result $p^* = m/n$ applies uniformly across all models and separations and is discussed separately in Chapter 1 and Chapter 4 as a sampling observation. "Certified" means proved by the cut-counting optimisation for the stated sizes. "conj." means a conjecture with a proved matching lower bound. "cond." means proved conditional on a finite, identified statement (for $m = 3$ directed multigraphs, the two $n = 7$ computations of Remark A.30). "lower bd." means only a proved lower bound is known, with the exact value open.

surface makes the proved-versus-open divide of the trichotomy a single readable shape. Reading along the m axis recovers the linear scaling in the threshold, and reading along n recovers each variant's growth rate, which is the whole of Table 4.1 compressed into one object.

4.3 Summary of the twelve variants

Table 4.1 records the status of all twelve structural variants. Each entry is marked by how it is known: proved by hand here or in Appendix A, cited to the literature, certified by the cut-counting optimisation for the stated sizes, or conjectured with a proved lower bound. The distinction is the honesty contract of Chapter 2 carried through to the summary.

4.3.1 Orientation models for the directed hypergraph

The directed rows of Table 4.1 read a directed Berge edge in one way, the *forward* one of a single tail fanning to $r - 1$ heads. It is not the only reading. In general a directed r-uniform hyperedge is a split of its vertices into a non-empty tail set T and head set H, a route entering at a tail and leaving at a head, and the gate checker of Chapter 2 measures any such split without change. Three models are worth naming, *forward* ($|T| = 1$), *backward* ($|H| = 1$), and *general* (any split, with genuinely mixed ones appearing once $r \geq 4$). The natural question, and one a reader of the directed figures tends to ask, is whether they multiply the table.

They do not, and two results say why. First, backward is not a new problem. Reversing every hyperarc turns a backward hypergraph into a forward one and reverses every route along the way, so the two models share every extremal number, edge and vertex, for all r, m and n (Lemma A.42). The backward column is the forward column read upside down. Second, the general model, though it admits more hyperedges on a single vertex set, obeys the same first-

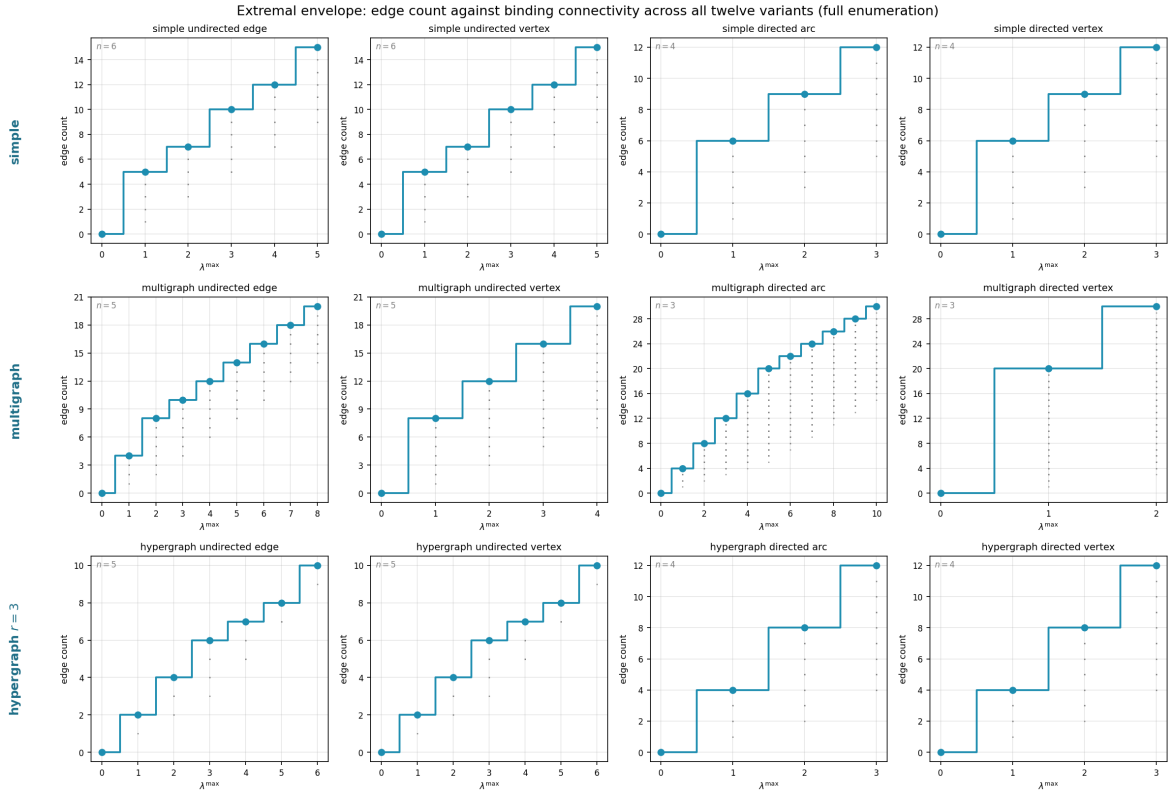


Figure 4.3. The complete landscape of all labelled graphs at small n (see panel annotations for each variant's n): every point is one graph, plotted at its binding connectivity $\lambda^{\max}$ horizontally and its edge count vertically. The blue staircase marking the densest graph at each connectivity level is the extremal envelope, the frontier the extremal problem asks about, with one dot per level so its exact value is legible. Graphs crowd the low-connectivity, low-edge region. The extremal graphs live at the top of the staircase. The emptiness above the envelope is exactly the region the main theorems forbid. The horizontal reach differs by panel because the connectivity that fits in a variant's enumeration n differs: arc-disjoint multigraph routes grow with edge multiplicity (the multigraph directed arc panel reaches $\lambda^{\max} = 10$ at $n = 3$), whereas vertex-disjoint routes are capped by the available internal vertices, so the multigraph directed *vertex* panel reaches only $\kappa^{\max} = 2$ at the same $n = 3$ (with a single internal vertex, a pair has at most the direct route plus one detour). This is a property of the separation, not of the search.

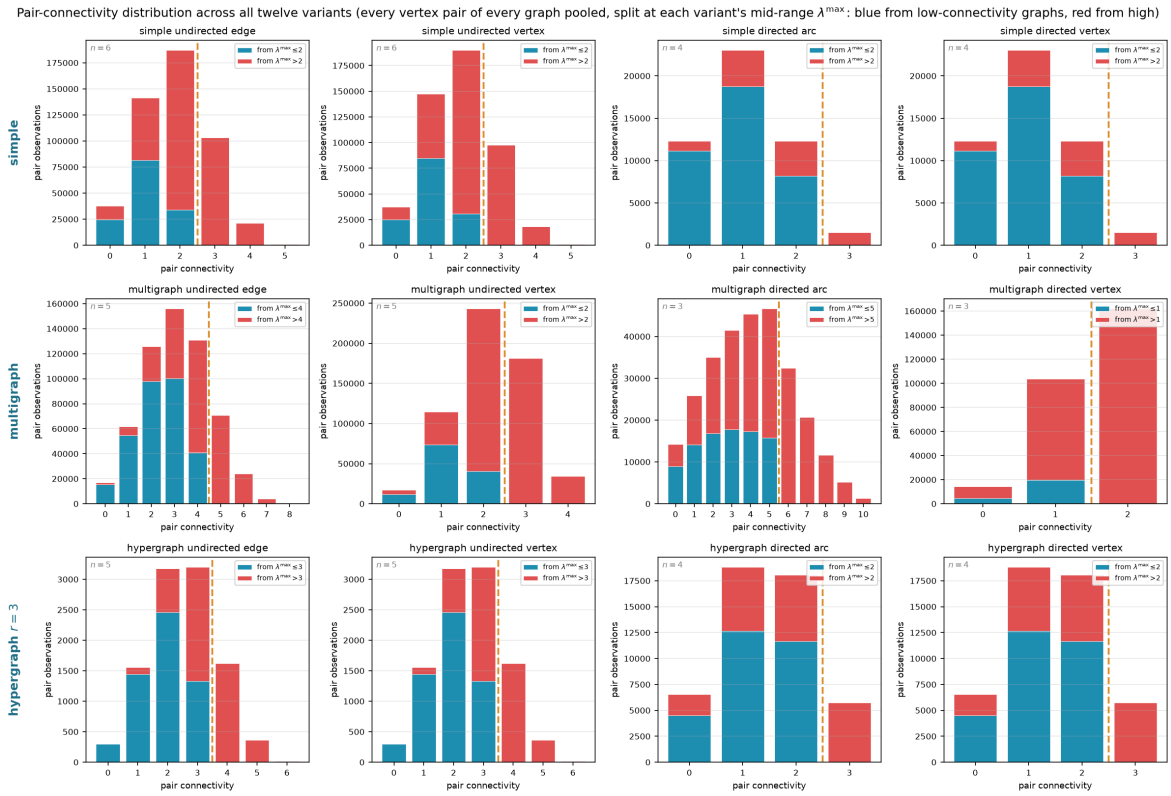


Figure 4.4. Pair-connectivity distribution across all twelve variants, pooling every vertex pair of every graph in the full enumeration. The horizontal axis is the connectivity of a single pair, and a bar's height is the number of (graph, pair) observations at that level. Each bar is split and stacked by the connectivity of the graph the pair was drawn from, blue from graphs at or below a per-panel boundary T and red from graphs above it, with the dashed line at T. The boundary is set per variant to the midpoint of the connectivity range that variant's enumeration reaches (each panel's legend gives its T), so the split stays informative in every panel instead of collapsing to one colour. Bars to the right of the dashed line are wholly red, since a pair of connectivity above T forces its graph above T. To the left the colours mix.

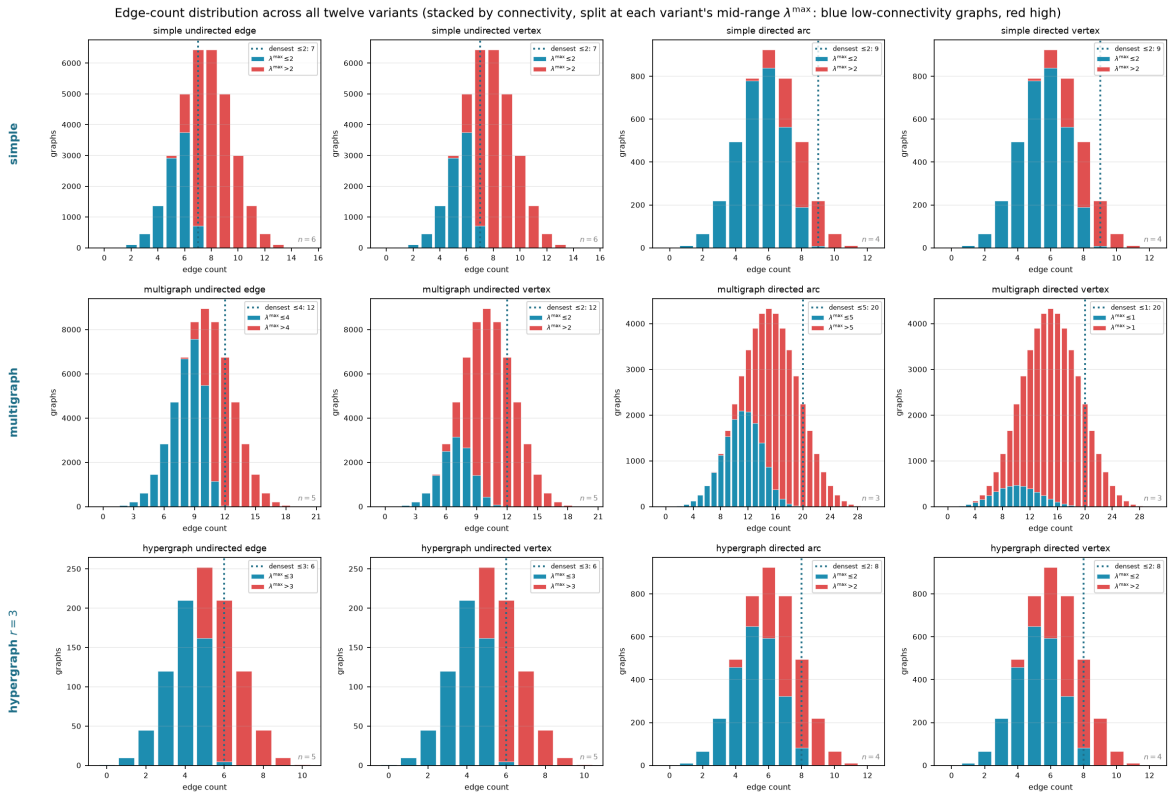


Figure 4.5. Edge-count distribution across all twelve variants, the same enumeration viewed by edge count rather than connectivity. Each bar is split and stacked by graph connectivity, blue for graphs at or below the per-panel boundary T and red above it (each panel's legend gives its T), so the full height is the total number of graphs at that edge count. The dotted vertical line marks the densest graph on the low-connectivity side. The reader can see directly how the two populations separate by edge count and how far the densest low-connectivity graphs sit from the bulk.

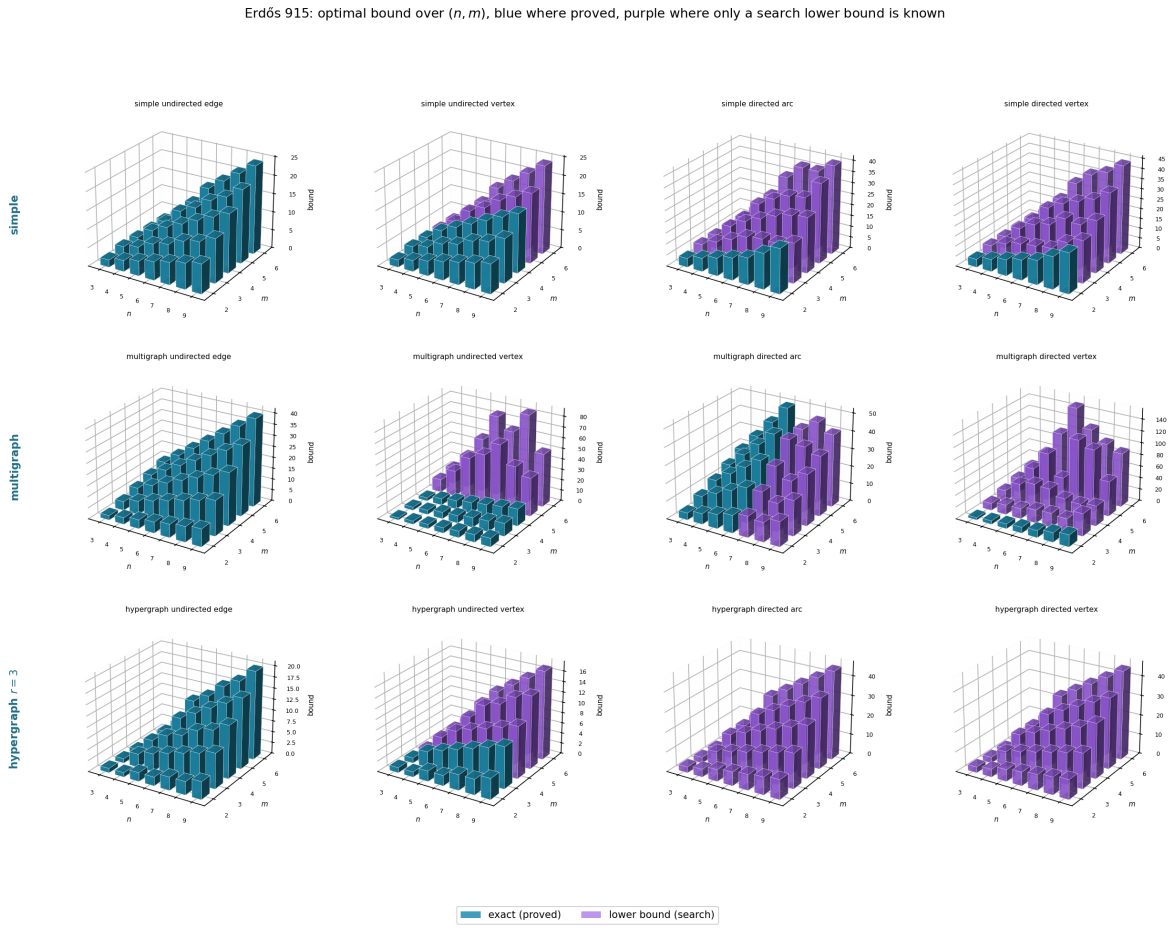


Figure 4.6. The optimal bound as a surface over the (n, m) grid for all twelve variants, every panel drawn from the same camera so the shapes compare directly. Each bar's height is the largest feasible edge count found by `solve` at that vertex count and forbidden threshold m. The single thing to read off is colour: a bar is *blue* exactly where the value is machine-proved and *purple* exactly where only a search lower bound is known, so each panel is a little map of what is settled. The three fully proved variants (Mader's undirected edge cases and the multigraph analogue) are solid blue sheets, rising linearly or quadratically as their formulas predict, while the open variants turn purple precisely where proof runs out. Reading along the m axis shows how the bound scales with the forbidden threshold, and reading along the n axis shows the growth rate for a fixed variant.

order bound. A hyperedge (T, H) occupies $|T|\,|H| \geq r-1$ ordered tail-head pairs, each usable by at most $m-1$ disjoint one-step routes, so $(r-1)|E| \leq (m-1)\,n(n-1)$, exactly the forward cap of Proposition A.41 (Proposition A.43). The forward construction is itself a set of general hyperedges, so general is squeezed between the same lower and upper bounds as forward, and its value too is quadratic in n with the same leading constant.

Where the models part company is only at small sizes, and the program shows exactly where. Table 4.2 lists the extremal hyperedge counts the branch-and-bound checker (`max_feasible_hyperedges`) proves for the forward and general models. The general one is strictly denser precisely when vertices are scarce, $n \approx r$, where the richer set of orientations lets a single vertex-set carry more hyperedges. At $n = r = 4$ it reaches eight hyperarcs against forward's four, and the surplus grows with the connectivity budget. The moment n exceeds r the gap is gone, proved equal already at $r = 3$, $n = 4$, and larger searches find no separation thereafter. This matches the bound: once the hypergraph spreads past a single edge-set, the one-tail model already saturates the Menger budget and extra orientations buy nothing. So the orientation axis collapses for the asymptotics the table records, which is why Problem 915 stays a twelve-variant question. The general model enters as a refinement at the margin, not a thirteenth column.

r	m	n	forward	general
3	3	3	3	4
3	3	4	8	8
4	3	4	4	6
4	4	4	4	8

Table 4.2. Extremal hyperedge counts the branch-and-bound checker proves exact for the forward and general directed r-uniform models, at the small sizes where they can differ. The general model is strictly denser only when $n \approx r$, doubling the forward count at $n = r = 4$, $m = 4$, and the gap closes once n exceeds r (proved equal at $r = 3$, $n = 4$). Edge- and vertex-disjoint counts agree throughout, and backward equals forward by Lemma A.42.

4.4 Contributions and limitations of the program

The program turned a per-case craft into a uniform pipeline. Before it, each variant came with its own one-off computation and its own hand argument. After it, one representation holds every variant, one exact checker measures connectivity, one certifier proves small-case upper bounds, and one cooled search discovers extremisers and the lower bounds they witness. The rediscovery of Chapter 3 is the evidence that the pipeline is faithful: pointed at solved cases, it returns the solved answers.

Its limits are as clear as its reach, and stating them is part of the honesty the thesis insists on. The search proves no upper bounds. It only ever exhibits graphs. The certifier proves upper bounds, but only for a fixed size, and the directed multigraph encoding closed unconditionally only up to $n = 6$. The deletion-cut strengthening made the two $m = 3$ computations at $n = 7$ memory-

safe, but they remain runtime-limited and have not yet returned certificates. And no amount of either replaces the two genuinely structural gaps the thesis leaves open: the backward-arc lemma that would turn Conjecture 1.13 into a theorem, and the vertex-disjoint problem at $m \geq 6$, which has resisted for half a century and which the program can probe but not crack.

4.5 Open problems

- The backward-arc lemma: prove that an extremal non-hub digraph has no arc from B to A, completing the upper bound in Conjecture 1.13 for $m \geq 3$, and with it the directed vertex case.

- The two finite $n = 7$ computations that close the directed multigraph problem at $m = 3$. By the attachment lemma and the conditional odd step (Lemma A.27, Corollary A.28, Theorem A.29), the entire $m = 3$ problem (value *and* extremal uniqueness for all $n \geq 8$) reduces along a single induction to two integral statements about multigraphs with multiplicities in $\{0, 1, 2\}$ (Remark A.30): (a) $L_3^{\mathrm{dir}}(7) = 24$, and (b) every 24-arc feasible multigraph on 7 vertices with maximum degree at most 8 is $2 \cdot B_{3,4}$, $2 \cdot B_{4,3}$, or the doubled bidirected path P_7. Both sit at sizes the certifier and the enumerator can reach, but the current implementations have not closed them: the MILP infeasibility check for (a) reaches its time limit, and the exhaustive enumeration for (b) stays within memory yet remains runtime-limited, so this is a finite and machine-checkable gap rather than a structural one. Conjecture A.22 is thereby bypassed for $m = 3$, though it remains open and of independent interest as the natural minimum-degree statement for the directed multigraph problem.

- Extremal uniqueness at odd levels for directed multigraphs with $m \geq 4$. The conditional odd step (Theorem A.29) and the propagation of Remark A.30 are stated for $m \in \{3, 4\}$. For $m = 4$ the *value* step goes through, but at an odd extremal level the counting no longer forces a vertex of degree $(m-1)k$ (all degrees $\geq (m-1)k+1$ are arithmetically consistent), so the uniqueness statement does not yet propagate. For $m \geq 5$ the same escape climbs one level and even the *value* step stalls: a surplus extremiser could be degree-regular strictly above the deletion bound, so no vertex deletion is forced to land on an extremal multigraph. Closing either hole asks for an arithmetic or structural input the present averaging argument does not supply.

- The undirected vertex-disjoint problem for $m \geq 6$, open since 1974, where even the extremal structure is unknown.

- The hypergraph vertex problem at $m \geq 4$. The problem is now settled at $m = 2$ and $m = 3$ for every uniformity and all n, with $k_m^{(r)}(n) = \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor = \ell_m^{(r)}(n)$ (Theorems A.36 and A.39). The incidence reduction stands for every m, but the rank bound that drives it (Lemma A.38) leans on the triconnected (Tutte/SPQR) decomposition exactly where $\kappa \leq 2$ lives. For $m = 4$ it would need a 4-connectivity analogue, where no comparably clean decomposition is available. Exhaustive computation gives $k_4^{(3)}(5) = 6 = \lfloor 3 \cdot 4/2 \rfloor$, so the agreement $k_m^{(r)} = \ell_m^{(r)}$ survives

at least one step past the theorem and there is no early counterexample. Where it first breaks for $r \geq 3$ (the analogue of the $m = 5$ graph divergence) is unknown.

- ★ The directed hypergraph variants. The tail-and-heads model has first bounds and a quadratic lower-bound family (Proposition A.41): a pair-codegree count caps the hyperedges at $(m-1)n(n-1)/(r-1)$, and a bipartite construction reaches $\approx (m-1)n^2/(4(r-1))$, so the value is quadratic in n. Its exact order, and the matching upper construction, are unexplored. The orientation of the Berge edge does not widen this gap: the backward model coincides with forward by reversal (Lemma A.42), and the general model obeys the same cap (Proposition A.43), beating forward only in the scarce-vertex regime $n \approx r$ (Section 4.3.1), so a single quadratic question covers all three.

The map is filled in where honest mathematics could fill it, and where it could not, the blanks are marked precisely, with a machine standing ready at each of them. That, finally, is the use of building the microscope: not that it answers every question, but that it shows exactly which questions are left, and hands the next reader a clean place to start.

Appendix A

Full Proofs and Computational Transcripts

This appendix holds the arguments deferred from the body: the ones that are either standard machinery or long case-checking, and whose length, not whose difficulty, kept them out of the narrative. The body carries the ideas. Here they are carried out in full. Because graphs are visual objects, the longer arguments here are accompanied by figures: each is drawn rather than merely described, and the prose points the reader to the picture at the moment it is needed.

How to read this appendix. The sections divide into standard machinery and the thesis's own proofs, and a reader chasing one thread of the body need not read all of them. The first two sections, on Theorem A.1, Theorem A.2, and Mader's theorem, are classical results included only so the body can cite them in the exact form it uses, and a reader who already trusts Menger, Gomory–Hu, and Mader may skip them. Three sections carry the load-bearing original arguments. The directed arc value at $m = 2$ supplies the finite base cases the body's induction rests on. The directed multigraph problem at large n is the longest argument here and the one behind the main directed contribution. The hypergraph vertex problem proves the incidence-rank lemma that settles the $m = 3$ hypergraph case. The remaining sections, on the hypergraph edge bounds and the random threshold, each support a single claim of the body and can be read on their own when that claim is reached. Finally, everything the program asserts is reproduced verbatim in the computational transcripts and the self-test section, so the machine proofs can be audited without running anything.

A.1 Menger, Gomory–Hu, and the degree bound

The two classical theorems below are the foundation of everything the thesis measures, and we state them at the level of generality at which the program actually applies them. A *capacitated graph*, or *weighted graph*, is a graph, directed or not, carrying a nonnegative capacity $c(e)$ on each edge or arc. A plain graph is read as the *unit-capacity* case $c \equiv 1$, and a multigraph as the integer-capacity case, one unit per parallel copy. A *flow* from u to v sends units along edges without exceeding any capacity, and the *capacity of a cut* is the total capacity of the edges crossing it. Both theorems below are theorems about capacitated graphs, and the thesis uses them mostly at unit and integer capacities, where the maximum flow is exactly the connectivity λ.

Theorem A.1 (Menger, as max-flow min-cut [2])**.** *In any capacitated digraph, and for distinct*

vertices u, v, the maximum flow from u to v equals the minimum capacity of a cut separating u from v. At unit capacities this is the combinatorial statement we cite throughout: the maximum number of pairwise edge-disjoint u–v paths equals the minimum number of edges whose removal separates them,

$$\lambda_G(u,v) = \min\{\,|C| : C \subseteq E(G),\ G - C \text{ has no } u\text{–}v \text{ path}\,\}.$$

A directed capacitated graph is the most general object here, and every graph model the thesis uses is a special case of it. An undirected graph is the symmetric case, each edge a pair of opposite unit-capacity arcs. A multigraph is the integer-capacity case, with $\mu(u,v)$ units on the pair uv. The hypergraph version (Theorem A.31) is this very theorem applied to the helper-point network of Figure 2.5, a capacitated digraph. The internally vertex-disjoint *version is obtained not by changing the theorem but by changing the graph it runs on, a device called* vertex splitting*: replace each internal vertex w by two copies joined by a single unit-capacity arc $w^- \to w^+$, send every arc that entered w into w^- and every arc that left w out of w^+. A flow may now pass through w at most once, so edge-disjoint paths in the split graph are exactly internally vertex-disjoint paths in the original, and a minimum edge cut there is a minimum vertex cut here. This split graph is literally what the checker of Chapter 2 builds, which is why a single maximum-flow routine measures all four separations.*

Theorem A.2 (Gomory–Hu [14])**.** *Every* undirected *capacitated graph G has a weighted tree T on the same vertex set, its* Gomory–Hu tree*, such that for all u, v the value $\lambda_G(u,v)$ equals the minimum edge weight on the u–v path in T, and each tree edge e corresponds to a minimum cut of G whose capacity is the weight of e. All $\binom{n}{2}$ pairwise values are thereby stored in the $n-1$ weights of a single tree.*

Two features of the statement fix how far it reaches, and both are used below. Being a theorem about capacities, it applies verbatim to a simple graph (unit weights), to a multigraph (integer weights, the multiplicity μ as capacity), and, in the form of Theorem A.32, to the hyperedge-connectivity of a hypergraph. But it needs the cut function to be *symmetric*, which forces G to be undirected: a digraph has $\lambda(u,v) \neq \lambda(v,u)$ in general and admits no such tree, and the vertex-connectivity function fails symmetry for the same reason. So the Gomory–Hu tree is precisely the tool for the undirected edge problems, where it drives Mader's theorem and its multigraph and hypergraph cousins (Theorems 1.2 and 1.3 and proposition 1.14) through one argument, and precisely the tool that vanishes once arcs turn one-way.

Primer: why a Gomory–Hu tree exists

The theorem is far from obvious, so here is why such a tree can always be built, in enough detail that nothing is taken on faith. Everything rests on one property of cuts. For a set of vertices S write $d(S)$ for the total capacity of the edges with exactly one endpoint in S, which is the capacity of the cut $(S, V \setminus S)$. The cut function is *submodular*: for any two vertex sets

A and B,

$$d(A) + d(B) \;\geq\; d(A \cap B) + d(A \cup B).$$

One checks this one edge at a time. An edge contributes to each side of the inequality according to where its two endpoints fall relative to A and B, and a short case check over the possible placements shows its contribution on the right never exceeds its contribution on the left. The only edges that could contribute more on the right have one endpoint in $A \setminus B$ and the other in $B \setminus A$, and those contribute 0 on the right and 2 on the left, so the inequality only gains.

Submodularity buys the single lemma the construction needs, the *uncrossing* lemma: if $(S, V \setminus S)$ is any cut and u, v lie on the same side of it, then some minimum u–v cut keeps all of S (or all of $V \setminus S$) on one side, that is, it does not cross S. The picture is that a cheapest u–v cut which sliced through S could be pushed clear of S without ever gaining capacity, and submodularity is exactly the inequality guaranteeing the push is free.

With that lemma the tree is built in $n - 1$ rounds, one minimum-cut computation each. Begin with the trivial tree, a single node holding all n vertices. In each round pick a node of the current tree that still holds two or more vertices, choose two of its vertices u and v, and compute a minimum u–v cut in the graph in which every neighbouring tree node, itself already a bag of several vertices, is shrunk to one vertex placed on whichever side the cut assigns it. Split the chosen node along the two sides of the cut and join the two halves by a new tree edge weighted by the cut's capacity. The uncrossing lemma is what makes the shrinking legitimate, because it says the minimum u–v cut can be chosen to respect every cut found in earlier rounds, so no earlier tree edge is spoiled. After $n - 1$ rounds every node holds a single vertex and T is a weighted tree on V with $n - 1$ edges.

Two facts finish the argument, and they are exactly what the body uses. Each tree edge is by construction the capacity of an actual cut of G, so its weight is a genuine minimum u–v cut value $\lambda_G(u, v)$ for the pair it separates. And for any u, v the value $\lambda_G(u, v)$ equals the lightest edge on the u–v path of T. One inequality holds because every tree edge on that path is itself a cut separating u from v, so none can be lighter than the global minimum u–v cut. The other holds because the lightest tree edge on the path is a genuine u–v cut of exactly that weight. The reader who wants the uncrossing induction spelled out in full will find it in Gomory and Hu [14]. The point here is only that the tree is an honest theorem, computed by $n - 1$ maximum-flow calls, and not a convenient fiction.

The most basic constraint on local connectivity is local: a pair cannot have more independent routes than either endpoint has edges. The constructions below use it constantly.

Lemma A.3 (Degree bound)**.** *For every pair of vertices of a graph G, $\lambda_G(u, v) \leq \min(\deg u, \deg v)$. In a digraph D, $\lambda_D(u, v) \leq \min(d^+(u), d^-(v))$.*

Proof. Every edge-disjoint u–v path uses a distinct edge at u, so there are at most $\deg u$ of them, and likewise at most $\deg v$. In the directed case each arc-disjoint path leaves u by a distinct

out-arc and enters v by a distinct in-arc. □

A.2 Mader's theorem in full

The upper bound of Theorem 1.2 rests on three short lemmas about a Gomory–Hu tree T of G. For an edge $e = \{u, v\}$ of G, write $\mathrm{dist}_T(e)$ for the number of tree edges on the unique u–v path in T. Since T is a tree on the very same vertex set, $\mathrm{dist}_T(e) \geq 1$ for every edge. Figure A.1 shows a small graph beside one of its Gomory–Hu trees and marks which edges have $\mathrm{dist}_T(e) = 1$ and which have $\mathrm{dist}_T(e) \geq 2$. It is worth keeping in view, since the three lemmas below are exactly statements about that picture. The first lemma is the one place the proof uses that G is *simple*, and it is exactly where the simple-graph bound improves on the multigraph bound.

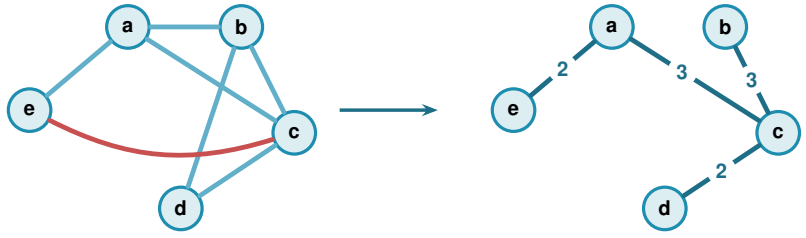


Figure A.1. A simple graph G (left) and a Gomory–Hu tree T of it (right). Both live on the same five vertices. The tree T has edges $ea\,(2)$, $ac\,(3)$, $cb\,(3)$, $cd\,(2)$, where each weight equals the local edge-connectivity of its endpoints in G. These four edges are also edges of G, so each has $\mathrm{dist}_T(e) = 1$: these are the E_1 edges of Lemma A.4, and a *simple* graph admits at most $n - 1 = 4$ of them, one per tree edge. The remaining edges ab, bd, and the red chord ec all join tree-distance-2 pairs: the path in T from e to c is e–a–c, so $\mathrm{dist}_T(ec) = 2$. Reading $\mathrm{dist}_T(e)$ as the number of tree cuts an edge crosses makes $\sum_e \mathrm{dist}_T(e) = \sum_t c(t)$ (Lemma A.6). Every edge crosses at least one cut, but only the $n-1$ tree edges cross exactly one, which is the surplus that drives Lemma A.5 and ultimately the factor of two in Mader's bound.

Lemma A.4 (Distance-one edges)**.** *At most $n-1$ edges of a simple graph G satisfy $\mathrm{dist}_T(e) = 1$.*

Proof. An edge $e \in E(G)$ has $\mathrm{dist}_T(e) = 1$ if and only if its endpoints are adjacent in T. The tree T has exactly $n - 1$ edges, and since G is simple each pair of adjacent tree vertices supports at most one edge of G. □

Lemma A.5 (Sum of tree distances)**.** $\displaystyle\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; 2|E(G)| - (n-1).$

Proof. Partition $E(G)$ into $E_1 = \{e : \mathrm{dist}_T(e) = 1\}$ and $E_{\geq 2} = \{e : \mathrm{dist}_T(e) \geq 2\}$. Then

$$\sum_{e \in E(G)} \mathrm{dist}_T(e) \;\geq\; |E_1| + 2\,|E_{\geq 2}| \;=\; 2|E(G)| - |E_1| \;\geq\; 2|E(G)| - (n-1),$$

using Lemma A.4 in the last step. □

Lemma A.6 (Double-counting identity)**.** *Let G carry* unit capacities*, so that each weight $c(t)$ of a Gomory–Hu tree T is the plain* number *of edges of G crossing the cut that t induces. Then*

$$\sum_{t \in E(T)} c(t) \;=\; \sum_{e \in E(G)} \operatorname{dist}_T(e).$$

This is the form used for Mader's theorem, where G is a simple graph and so unit-capacitated by default.

Proof. By Theorem A.2 each tree edge t induces a cut of G of capacity $c(t)$, so $c(t) = \sum_{e \in E(G)} \mathbf{1}_{e \text{ crosses } t}$. Exchanging the order of summation,

$$\sum_{t \in E(T)} c(t) = \sum_{e \in E(G)} \sum_{t \in E(T)} \mathbf{1}_{e \text{ crosses } t} = \sum_{e \in E(G)} \operatorname{dist}_T(e),$$

where the last equality holds because an edge $\{u, v\}$ crosses exactly those tree cuts induced by the tree edges on the unique u–v path in T. □

Proof of Theorem 1.2, upper bound. Let G be a simple graph on n vertices with $\lambda_G^{\max} \le m - 1$ and let T be a Gomory–Hu tree of G. Each tree edge $t = \{x, y\}$ has weight $c(t) = \lambda_G(x, y) \le m - 1$, so summing over the $n - 1$ tree edges, $\sum_t c(t) \le (m-1)(n-1)$. Combining with Lemmas A.5 and A.6,

$$2|E(G)| - (n-1) \;\le\; \sum_{e \in E(G)} \operatorname{dist}_T(e) \;=\; \sum_{t \in E(T)} c(t) \;\le\; (m-1)(n-1).$$

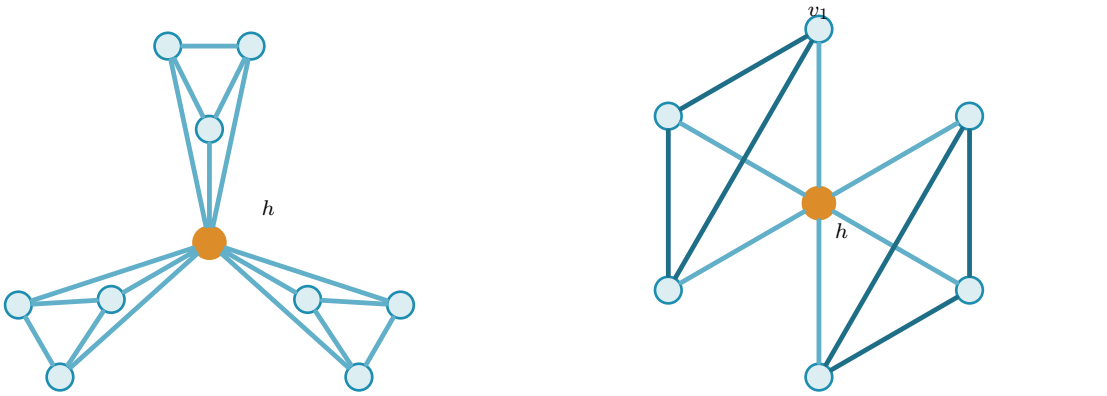
Rearranging gives $2|E(G)| \le m(n-1)$, and since $|E(G)|$ is an integer, $|E(G)| \le \left\lfloor \frac{m(n-1)}{2} \right\rfloor$. □

Note where the factor of two came from: every edge crosses at least one tree cut, but a simple graph can afford at most $n - 1$ edges that cross only one, so on average each edge is charged closer to twice. For multigraphs that refinement is unavailable, since parallel edges between tree-adjacent pairs each have $\operatorname{dist}_T = 1$, and the same chain of inequalities stops at $|E| \le \sum_e \operatorname{dist}_T(e) \le (m-1)(n-1)$, which is exactly the multigraph bound of Theorem 1.3.

The lower bound is a pair of constructions, drawn side by side in Figure A.2. The first is the shape the bound was guessed from, and works when $m - 1$ divides $n - 1$. The second generalises it to every $n \ge m \ge 2$.

Construction A.7 (Shared-clique graph)**.** For $m \ge 3$ with $(m-1) \mid (n-1)$, take $k = (n-1)/(m-1)$ copies of the complete graph K_m and identify one vertex of each copy into a single shared hub vertex h. The result has $k(m-1) + 1 = n$ vertices and $k\binom{m}{2} = \frac{m(n-1)}{2}$ edges. Every non-hub vertex has degree $m - 1$, and any pair of vertices contains a non-hub vertex, so $\lambda^{\max} \le m - 1$ by Lemma A.3.

Construction A.8 (Hub-and-spokes graph)**.** For $n \geq m \geq 2$, let $H_{m,n}$ have vertex set $\{h, v_1, \ldots, v_{n-1}\}$ with (i) all $n-1$ *hub edges* $\{h, v_i\}$, and (ii) a *spoke graph* on $\{v_1, \ldots, v_{n-1}\}$ with $\left\lfloor \frac{(m-2)(n-1)}{2} \right\rfloor$ edges in which every v_i has spoke degree at most $m-2$. Such a spoke graph exists by Lemma A.9.



(a) Shared-clique graph: three K_4's glued at one hub h ($m = 4$, $k = 3$). Each non-hub vertex has degree $m - 1 = 3$.

(b) Hub-and-spokes $H_{m,n}$: the hub edges $\{h, v_i\}$ (light) plus a $(m-2)$-regular *spoke graph* (dark) in two $K_{m-1} = K_3$ cliques.

Figure A.2. The two extremal families. **(a)** The shared-clique graph of Construction A.7 works when $(m-1) \mid (n-1)$: every pair contains a non-hub vertex of degree exactly $m-1$, so $\lambda^{\max} \leq m-1$ by the degree bound, while the edge count hits $\frac{m(n-1)}{2}$. **(b)** The hub-and-spokes graph $H_{m,n}$ of Construction A.8 reaches the same bound for *every* $n \geq m$: each spoke vertex v_i has total degree $1 + (m-2) = m-1$, and choosing the spoke graph as disjoint cliques K_{m-1} recovers (a). The hub is drawn in orange in both.

Lemma A.9 (Near-regular graphs exist)**.** *For $N \geq 1$ and $0 \leq r \leq N-1$ there is a graph on N vertices with $\left\lfloor \frac{rN}{2} \right\rfloor$ edges and maximum degree at most r.*

Proof. If rN is even we build an r-regular circulant graph on vertex set $\{0, \ldots, N-1\}$: for even r connect each i to $i \pm j \pmod N$ for $j = 1, \ldots, r/2$, and for odd r (then N is even) use $j = 1, \ldots, (r-1)/2$ together with the diameter distance $N/2$, which contributes degree one. Both are r-regular with $rN/2$ edges. If rN is odd, then r and N are both odd: take the $(r-1)$-regular circulant with distances $1, \ldots, (r-1)/2$ and add the near-perfect matching that pairs vertex i with $i + (N+1)/2 \pmod N$ for $i = 0, \ldots, (N-3)/2$, which covers all vertices but one. The total is $\frac{(r-1)N}{2} + \frac{N-1}{2} = \frac{rN-1}{2} = \left\lfloor \frac{rN}{2} \right\rfloor$ edges, with one vertex of degree $r-1$ and all others of degree r. □

Theorem A.10 (Extremal graphs exist)**.** *For all $n \geq m \geq 2$ the graph $H_{m,n}$ has $\left\lfloor \frac{m(n-1)}{2} \right\rfloor$ edges and $\lambda^{\max} \leq m-1$.*

Proof. The edge count is $(n-1) + \left\lfloor \frac{(m-2)(n-1)}{2} \right\rfloor = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, since $(m-2)(n-1)$ and $m(n-1)$ have the same parity. For the connectivity, any two distinct vertices include at least one spoke vertex v_i, whose total degree is $1+(m-2) = m-1$. Lemma A.3 then gives $\lambda(u, v) \leq m-1$. When $(m-1) \mid (n-1)$, choosing the spoke graph as k disjoint cliques K_{m-1} recovers Construction A.7, so the hub-and-spokes family genuinely generalises the shared-clique one. □

Together the upper bound and Theorem A.10 prove Theorem 1.2. The proof also reveals which graphs are extremal: equality forces both inequalities in the chain to be tight.

Theorem A.11 (Characterisation of equality)**.** *Let G be simple with $\lambda_G^{\max} \le m-1$ and $|E(G)| = \left\lfloor \frac{m(n-1)}{2} \right\rfloor$, and let T be a Gomory–Hu tree of G. If $m(n-1)$ is even, then every tree edge has weight exactly $m-1$, every tree edge is also an edge of G, and every edge of G joins vertices at tree distance at most 2. If $m(n-1)$ is odd, there is exactly one unit of slack, located in exactly one of three places: one tree edge has weight $m-2$, or one tree edge is not an edge of G, or one edge of G joins vertices at tree distance 3. Everything else is tight.*

Proof. Write the chain of the upper-bound proof as three nonnegative slacks. The first, $S_1 = (m-1)(n-1) - \sum_t c(t)$, counts tree weights below $m-1$. The second, $S_2 = \sum_{e:\,\mathrm{dist}_T(e)\ge 2}(\mathrm{dist}_T(e)-2)$, counts edges beyond tree distance two. The third, $S_3 = (n-1)-|E_1|$, counts tree edges that support no edge of G. Retracing the proof, $\sum_e \mathrm{dist}_T(e) = 2|E| - |E_1| + S_2$ and $\sum_e \mathrm{dist}_T(e) = \sum_t c(t) = (m-1)(n-1) - S_1$, so altogether $2|E| = m(n-1) - (S_1+S_2+S_3)$. If $m(n-1)$ is even, equality $|E| = m(n-1)/2$ forces $S_1 = S_2 = S_3 = 0$. If $m(n-1)$ is odd the floor absorbs exactly one unit, so $S_1 + S_2 + S_3 = 1$ and exactly one of the three slacks equals one. □

For $m = 2$ the characterisation says the extremal graphs are exactly the spanning trees: every tree edge must carry weight one and lie in G, so G contains its own Gomory–Hu tree and has only $n-1$ edges to spend. For $m = 4$ and $n \equiv 1 \pmod 3$ it is consistent with the trees of K_4 blocks glued at shared cut vertices, the case of Construction A.7 that the search of Chapter 3 rediscovers.

A.3 The directed arc value at $m = 2$

Three short lemmas feed the induction. The first shows a hub forces a linear bound for every m, and is the proof behind the hub branch of Conjecture 1.13. The three lemmas below and the proof of Theorem 1.9 that they support are the author's own, and its finite base cases $2 \le n \le 7$ were verified by the author's program, through the exhaustive search of Chapter 2 whose transcript is reproduced at the end of this appendix.

Lemma A.12 (Two-hop lemma)**.** *Let D be a simple digraph with $\lambda_D^{\max} \le m-1$ containing a vertex r with $d^+(r) = n-1$. Then every $v \ne r$ has at most $m-2$ in-arcs from $V \setminus \{r, v\}$.*

Proof. If $u_1, \dots, u_{m-1}$ were distinct in-neighbours of v other than r, that is, distinct vertices each sending an arc to v, the paths $r \to v$ and $r \to u_i \to v$ for $i = 1, \dots, m-1$ would be m arc-disjoint routes, since the u_i are distinct and $d^+(r) = n-1$ supplies every starting arc. That contradicts $\lambda_D(r, v) \le m-1$. □

Theorem A.13 (Hub upper bound)**.** *If a simple digraph D with $\lambda_D^{\max} \le m-1$ has a vertex r with $d^+(r) = n-1$, then $|A(D)| \le m(n-1)$.*

Proof. Split the arcs into those leaving r (exactly $n-1$), those entering r (at most $n-1$), and the internal arcs avoiding r. By Lemma A.12 each of the $n-1$ other vertices receives at most $m-2$ internal arcs, so there are at most $(m-2)(n-1)$ internal arcs, and the total is at most $m(n-1)$. □

Lemma A.14 (No transitive triangle)**.** *A simple digraph with $\lambda^{\max} \leq 1$ contains no three vertices x, y, z with all of $(x,y),(y,z),(x,z)$ present, since $x \to z$ and $x \to y \to z$ would be two arc-disjoint routes.*

Lemma A.15 (Deletion monotonicity)**.** *Write $D - v_0$ for the digraph obtained from D by deleting the vertex v_0 together with all arcs incident to it. For any digraph D and vertex v_0, $\lambda^{\max}(D-v_0) \leq \lambda^{\max}(D)$, since arc-disjoint paths in $D - v_0$ are still arc-disjoint paths in D.*

Proof of Theorem 1.9. Write $M(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$. A direct computation shows $2(n-1) \geq \lfloor n^2/4 \rfloor$ exactly for $n \leq 7$, with equality at $n = 7$, so $M(n) = 2(n-1)$ for $n \leq 7$ and $M(n) = \lfloor n^2/4 \rfloor$ for $n \geq 7$. The lower bound $\ell_2^{\mathrm{dir}}(n) \geq M(n)$ is witnessed by the directed hub construction (Construction 1.8 at $m = 2$, the bidirected star) and the one-directional bipartite construction (Construction 1.7).

For the upper bound we show by strong induction on n that $\lambda_D^{\max} \leq 1$ forces $|A(D)| \leq M(n)$.

Base cases $2 \leq n \leq 7$. All six are settled uniformly by the pruned exhaustive search of Chapter 2, which walks every simple digraph with $\lambda^{\max} \leq 1$ on up to seven vertices and reports the maxima $2, 4, 6, 8, 10, 12$, matching $M(n)$ at each size. The transcript is reproduced below, and the cut-counting certifier independently confirms the values within its reach through the $m = 2$ reduction of Theorem A.24. The two smallest are also immediate by hand, as a check on the machine rather than a separate argument: at $n = 2$ there are at most 2 arcs, and at $n = 3$ a fifth arc would leave at most one arc of the complete bidirected triangle missing, so three of the rest form a transitive triangle, which Lemma A.14 forbids. The point of the theorem is precisely that $n = 4, 5, 6, 7$ are *not* immediate by hand, which is what makes handing the case-check to the program the right move.

Inductive step $n \geq 8$. Let D be a simple digraph on n vertices with $\lambda_D^{\max} \leq 1$ and put $a := |A(D)|$. Choose v_0 minimising the total degree $d(v) = d^+(v) + d^-(v)$. Since $\sum_v d(v) = 2a$, averaging gives $d(v_0) \leq 2a/n$. By Lemma A.15 the digraph $D - v_0$ on $n-1 \geq 7$ vertices satisfies the induction hypothesis, so $a - d(v_0) \leq M(n-1) = \left\lfloor \frac{(n-1)^2}{4} \right\rfloor$. Combining,

$$a \;\leq\; \frac{2a}{n} + \left\lfloor \frac{(n-1)^2}{4} \right\rfloor \qquad \Longrightarrow \qquad a \;\leq\; \frac{n}{n-2} \left\lfloor \frac{(n-1)^2}{4} \right\rfloor .$$

It remains to check that this right-hand side never exceeds $\lfloor n^2/4 \rfloor$, and the two parities behave differently enough to treat them in turn.

For even $n = 2k$ with $k \geq 4$, first simplify $\lfloor (n-1)^2/4 \rfloor$. Expanding $(2k-1)^2 = 4k^2 - 4k + 1$

and dividing by 4 gives $k^2 - k + \frac{1}{4}$, whose floor is $k(k-1)$. The bound then reads

$$a \;\le\; \frac{2k}{2k-2}\,k(k-1) \;=\; \frac{k}{k-1}\,k(k-1) \;=\; k^2,$$

and since $\lfloor n^2/4 \rfloor = \lfloor (2k)^2/4 \rfloor = k^2$ as well, the bound lands exactly on the target with no slack left over.

For odd $n = 2k+1$ with $k \ge 4$, the same simplification gives $\lfloor (n-1)^2/4 \rfloor = \lfloor (2k)^2/4 \rfloor = k^2$, so the bound is $a \le \frac{2k+1}{2k-1}\,k^2$. To compare this fraction with the target, split off its integer part by dividing $(2k+1)k^2$ by $2k-1$. The remainder is exactly k, because $(2k-1)(k^2+k) = 2k^3+k^2-k$ and $(2k+1)k^2 = 2k^3 + k^2$ differ by k, so

$$\frac{n}{n-2}\left\lfloor \frac{(n-1)^2}{4} \right\rfloor \;=\; \frac{(2k+1)k^2}{2k-1} \;=\; k(k+1) + \frac{k}{2k-1}.$$

The leftover fraction $\frac{k}{2k-1}$ lies strictly between 0 and 1 for every $k \ge 1$, so the bound puts a at most an integer $k(k+1)$ plus a positive amount below one. Since a is itself an integer it cannot reach the next integer up, so $a \le k(k+1)$. The target $\lfloor n^2/4 \rfloor = \lfloor (2k+1)^2/4 \rfloor = k^2 + k = k(k+1)$ agrees, so for odd n it is integrality, not the raw arithmetic, that closes the last fraction of a unit. In both parities $a \le \lfloor n^2/4 \rfloor \le M(n)$, completing the induction. □

Two remarks on the shape of this proof. First, the minimum-degree deletion closes the step on its own, and no separate hub case is needed, because the averaging bound holds whether or not D contains a hub, although Theorem A.13 shows directly that a hub digraph never exceeds the linear branch. Second, for even n the bound is exactly tight against the one-directional bipartite construction, which is $\frac{n}{2}$-regular in total degree, so every step of the chain is achieved by the conjectured extremiser, and for odd n the slack $\frac{k}{2k-1} < 1$ is absorbed by integrality. The proof is long not because it is deep but because the two regimes of $M(n)$ must be carried through the arithmetic separately, and that bookkeeping is precisely what Chapter 2 hands to the machine at the base cases.

Proof of Theorem 1.10. By Whitney's inequality $\kappa^{\max} \le \lambda^{\max}$, every digraph feasible for the vertex problem is feasible for the arc problem, so $k_2^{\mathrm{dir}}(n) \le \ell_2^{\mathrm{dir}}(n)$. Conversely the hub and one-directional bipartite constructions have $\kappa^{\max} = 1$, so they are feasible for the vertex problem and give the matching lower bound. Hence $k_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n) = \max(2(n-1), \lfloor n^2/4 \rfloor)$. □

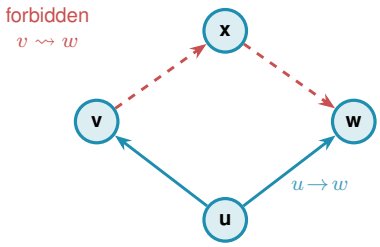
A.4 Structural propositions on the directed frontier

These are the proved ingredients of the reduction discussed in Chapter 4. The upper bound for $m \ge 3$ is conditional only on the backward-arc lemma stated there.

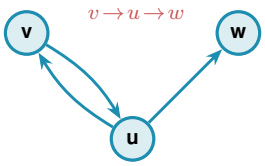
Proposition A.16 (Mutually unreachable out-neighbourhoods)**.** *Let D be a simple digraph with $\lambda_D^{\max} \le 1$. For any vertex u and distinct out-neighbours v, w of u, there is no directed path from*

v *to* w *in* $D - u$.

Proof. If P were a directed v–w path avoiding u, then $u \to w$ and $u \to v \xrightarrow{P} w$ would be two arc-disjoint u-to-w routes: the first uses only the arc (u, w), the second starts with (u, v) and continues along P, none of whose arcs touch u. This contradicts $\lambda_D(u, w) \leq 1$. The restriction to $D - u$ is not cosmetic, and Figure A.3 draws both the argument and the reason for the restriction: in the digraph with arcs (u, v), (v, u), (u, w) every local connectivity is 1, yet $v \to u \to w$ reaches w from v through u. □



(a) If a directed $v \rightsquigarrow w$ path (red) existed in $D - u$, then $u \to w$ and $u \to v \rightsquigarrow w$ would be two arc-disjoint u–w routes, against $\lambda(u, w) \leq 1$.



(b) The counterexample $\{(u, v), (v, u), (u, w)\}$: $\lambda^{\max} = 1$, yet w *is* reachable from v, but only *through* u.

Figure A.3. Why Proposition A.16 must exclude paths through u. **(a)** For out-neighbours v, w of u in a digraph with $\lambda^{\max} \leq 1$, no $v \rightsquigarrow w$ path can avoid u: such a path would combine with the lone arc $u \to w$ into two arc-disjoint routes. **(b)** The restriction to $D - u$ is essential. In the three-vertex digraph $\{(u, v), (v, u), (u, w)\}$ we have $\lambda^{\max} = 1$, yet v reaches w via $v \to u \to w$, a route that reuses the vertex u, which the proposition allows.

Proposition A.17 (Disjoint second out-neighbourhoods)**.** *Let* D *be a simple digraph with* $\lambda_D^{\max} \leq 1$*, let* $u \in V$*, and let* $v_1 \neq v_2$ *be out-neighbours of* u*. Then* $(N^+(v_1) \setminus \{u\}) \cap (N^+(v_2) \setminus \{u\}) = \emptyset$.

Proof. A common out-neighbour $w \neq u$ would give the two arc-disjoint routes $u \to v_1 \to w$ and $u \to v_2 \to w$, contradicting $\lambda_D(u, w) \leq 1$. □

Proposition A.18 (Two-hop budget on bipartite partitions)**.** *Let* D *be a simple digraph with* $\lambda_D^{\max} \leq m - 1$*. If* V *admits a partition* $A \cup B$ *with every arc from* A *to* B *present, then every* $b \in B$ *has in-degree at most* $m - 2$ *from inside* B*, and consequently the number of arcs inside* B *is at most* $(m - 2)|B|$.

Proof. Fix $a \in A$ and $b \in B$, and let $b'_1, \ldots, b'_d$ be the in-neighbours of b inside B, where $d = \deg_B^-(b)$. The direct arc $a \to b$ together with the two-step detours $a \to b'_i \to b$ are $1 + d$ arc-disjoint a-to-b routes, all present because $A \to B$ is complete. Feasibility forces $1 + d \leq m - 1$, so $d \leq m - 2$, and summing over B bounds the internal arcs by $(m - 2)|B|$. □

We now combine Proposition A.18 with the best choice of partition. Suppose every arc of D either runs from A to B or stays inside B: the partition $A \to B$ is complete, no arc lies inside A, and crucially *no arc runs from* B *back to* A. Write $b = |B|$, so that $|A| = n - b$. The arcs then come in exactly two kinds. The arcs crossing the partition number $|A|\,|B| = (n - b)\,b$, one for

each ordered A–B pair. The arcs inside B number at most $(m-2)\,b$: since no route between two vertices of B ever needs to leave B (there is no arc back to A to leave by), Proposition A.18 applies to B on its own and caps the in-degree of each $b \in B$ from inside B at $m-2$. There are no other arcs, so, adding the two kinds,

$$|A(D)| \;\le\; (n-b)\,b \;+\; (m-2)\,b \;=\; b\,\big(n+m-2-b\big).$$

The right-hand side is a downward parabola in b. Writing $s = n+m-2$, it is $b(s-b) = -b^2 + sb$, maximised at $b = s/2$, so over integers the best partition takes $b = \lceil s/2 \rceil$ (equivalently $\lfloor s/2 \rfloor$), and the maximum value is

$$\max_{b}\; b\,(s-b) \;=\; \left\lfloor \frac{s^2}{4} \right\rfloor \;=\; \left\lfloor \frac{(n+m-2)^2}{4} \right\rfloor .$$

This is the quadratic branch of Conjecture 1.13, reached uniformly in m. The only thing assumed beyond feasibility was that B sends nothing back to A, so that single no-back-arc hypothesis is exactly the gap between this bound and an unconditional proof.

A.5 The directed multigraph problem at large n

The body proves $L_m^{\mathrm{dir}}(n) = 2(n-1)(m-1)$ for $n \le 6$ (Theorem 2.1). This section records what is known beyond, where the one-directional bipartite construction takes over. Throughout, the scaling reduction of Chapter 2 is used in the following form.

Lemma A.19 (Scaling reduction)**.** *Let D be a directed multigraph with $\mu(u,v) \le m-1$ on every ordered pair and $\lambda^{\max}(D) \le m-1$. Then $w := \mu/(m-1)$ takes values in $[0,1]$, satisfies $\mathrm{maxflow}(s,t;w) \le 1$ for all $s \ne t$, and $|A(D)| = (m-1)\sum_{u\ne v} w(u,v)$.*

Proof. Set $w = \mu/(m-1)$. Every multiplicity obeys $0 \le \mu(u,v) \le m-1$, so each $w(u,v)$ lies in $[0,1]$. Dividing every capacity by the constant $m-1$ divides every flow value by $m-1$, and by Theorem A.1 the maximum flow under the capacities μ is $\lambda_D(s,t) \le m-1$, so under w it is at most 1. For the arc count, recall that the number of arcs is the total multiplicity, $|A(D)| = \sum_{u\ne v}\mu(u,v)$, since each parallel copy is one arc. Substituting $\mu = (m-1)w$,

$$|A(D)| \;=\; \sum_{u\ne v}\mu(u,v) \;=\; \sum_{u\ne v}(m-1)\,w(u,v) \;=\; (m-1)\sum_{u\ne v} w(u,v),$$

which is the claimed identity. □

Lemma A.20 (Arithmetic identity)**.** *For all $n \ge 2$, $\left\lfloor \frac{(n-1)^2}{4} \right\rfloor + \lfloor n/2 \rfloor = \left\lfloor \frac{n^2}{4} \right\rfloor$.*

Proof. For $n = 2k$ the left side is $k(k-1) + k = k^2$, and for $n = 2k+1$ it is $k^2 + k = k(k+1)$. Both match the right side. □

Proposition A.21 (Bipartite upper bound for even $n \geq 8$, conditional)**.** *Let $n = 2k \geq 8$ be even. Suppose every fractional weighting w' on $n-1$ vertices with values in $[0,1]$ and all pairwise maximum flows at most one has total weight at most $\left\lfloor \frac{(n-1)^2}{4} \right\rfloor$. Then every directed multigraph D on n vertices with $\lambda^{\max}(D) \leq m-1$ satisfies $|A(D)| \leq (m-1)\left\lfloor \frac{n^2}{4} \right\rfloor$.*

Proof. By Lemma A.19 the arc count is $|A(D)| = (m-1)M$ with $M := \sum_{u \neq v} w(u,v)$, so it suffices to bound $M \leq \lfloor n^2/4 \rfloor$ for the scaled weighting w, all of whose pairwise maximum flows are at most 1.

Averaging. Give each vertex its weighted total degree $d(v) = \sum_u \big(w(u,v) + w(v,u)\big)$. Summing over all vertices counts the weight of each ordered pair exactly twice, once at its tail and once at its head, so $\sum_v d(v) = 2M$, and some vertex v_0 has $d(v_0) \leq 2M/n$.

Deletion. The number $d(v_0)$ is exactly the total weight carried by pairs touching v_0, so deleting v_0 leaves a weighting on $n-1$ vertices of total $M - d(v_0)$. Removing a vertex cannot raise any maximum flow, so that restricted weighting still has all pairwise flows at most 1, and the hypothesis applies to it:

$$M - d(v_0) \;\leq\; \left\lfloor \frac{(n-1)^2}{4} \right\rfloor \;=\; \left\lfloor \frac{(2k-1)^2}{4} \right\rfloor \;=\; k(k-1),$$

the last step because $(2k-1)^2/4 = k^2 - k + \frac{1}{4}$ floors to $k(k-1)$.

Combining. Since $n = 2k$ we have $d(v_0) \leq 2M/n = M/k$, hence

$$M - \frac{M}{k} \;\leq\; M - d(v_0) \;\leq\; k(k-1), \qquad \text{that is} \qquad M \cdot \frac{k-1}{k} \;\leq\; k(k-1).$$

Dividing by $(k-1)/k > 0$ gives $M \leq k^2 = \lfloor n^2/4 \rfloor$, and multiplying back by $m-1$ gives $|A(D)| \leq (m-1)\lfloor n^2/4 \rfloor$. □

The small cases $n \in \{4, 6\}$ genuinely do not satisfy the conclusion, and must not: there $2(n-1) > \lfloor n^2/4 \rfloor$ and the double star beats the bipartite weighting, which is why the certified values of Theorem 2.1 sit on the linear branch. For odd n the same deletion gives $M \leq k(k+1) + \frac{k}{2k-1}$, an overshoot of less than one that integrality no longer absorbs, because fractional weights need not have integer totals. The induction would close if some vertex always had weighted degree at most $(m-1)k$ before scaling, which is the missing minimum-degree statement.

Conjecture A.22 (Minimum-degree lemma for odd n)**.** *In any directed multigraph D on $n = 2k+1$ vertices ($k \geq 3$) with $\lambda^{\max}(D) \leq m-1$ and $m \geq 3$, some vertex has total degree $d(v) \leq (m-1)k$, counting arcs with multiplicity.*

At $m = 2$ the statement is a proposition rather than a conjecture, through a reduction that also settles the whole multigraph problem at $m = 2$.

Proposition A.23 (Parallel-arc elimination at $m = 2$)**.** *A directed multigraph with $\lambda^{\max} \leq 1$ has no parallel arcs, since two parallel arcs are already two arc-disjoint routes. Hence it is a simple*

digraph.

Theorem A.24 (Directed multigraphs at $m = 2$)**.** *For all* $n \geq 2$, $L_2^{\mathrm{dir}}(n) = \ell_2^{\mathrm{dir}}(n) = \max\big(2(n-1), \left\lfloor \frac{n^2}{4} \right\rfloor\big)$.

Proof. Immediate from Proposition A.23 and Theorem 1.9. □

Proposition A.25 (Minimum degree at $m = 2$, odd n)**.** *In any directed multigraph on* $n = 2k+1 \geq 7$ *vertices with* $\lambda^{\max} \leq 1$*, some vertex has* $d(v) \leq k$*.*

Proof. By Proposition A.23 and theorem 1.9, $|A(D)| \leq \max\big(2(n-1), \lfloor n^2/4 \rfloor\big) = \lfloor n^2/4 \rfloor = k(k+1)$, the maximum being the quadratic branch because $n \geq 7$. The average total degree is $\frac{2k(k+1)}{2k+1} < k+1$, and some integer degree is at most k. The hypothesis $n \geq 7$ is needed: on the linear branch the statement can fail, as the bidirected star on three vertices has minimum total degree $2 > 1$. □

For $m \geq 3$ the two extreme regimes of Conjecture A.22 are settled. In the *purely saturated* regime, where every multiplicity is 0 or $m-1$, the scaled weighting is the indicator of a simple digraph with $\lambda^{\max} \leq 1$, its degrees are integers, and the argument of Proposition A.25 applies verbatim after multiplying back by $m-1$. In the *all-unsaturated* simple regime at $n = 7$, $m = 3$, the program certifies by a cut-counting infeasibility that no simple digraph satisfies all two-hop inequalities with every total degree at least 7. The open case is the *mixed* regime, where some multiplicity lies strictly between 0 and $m-1$: there the scaled degrees are genuinely fractional, the averaging bound $\frac{2k(k+1)}{2k+1}$ lands strictly between k and $k+1$, and no integrality rescues it. This is the single identified gap that separates the certified values of Theorem 2.1 from the full conjecture

$$L_m^{\mathrm{dir}}(n) \;=\; (m-1)\,\max\big(2(n-1),\; \left\lfloor \frac{n^2}{4} \right\rfloor\big),$$

whose lower bound is unconditional: the double star realises $2(n-1)(m-1)$ and the one-directional bipartite multigraph, every arc at multiplicity $m-1$, realises $(m-1)\lfloor n^2/4 \rfloor$ with $\lambda^{\max} = m-1$.

Remark A.26 (Why the mixed regime resists, and its kinship with the back-arc gap)*.* Two facts explain why the mixed regime is genuinely hard rather than merely unfinished, and both are worth recording so the conjecture is not mistaken for a routine averaging exercise.

First, averaging alone cannot close it, even granted the strongest input available. Work in the scaled weighting $w = \mu/(m-1)$ of Lemma A.19, with total weight $M = \sum_{u \neq v} w(u,v)$ and weighted total degree $d(v)$, so the conjecture asks for a vertex with $d(v) \leq k$. Granting the fractional even bound that drives Proposition A.21, the odd deletion already gives $M \leq k(k+1) + \frac{k}{2k-1} = \frac{(2k+1)k^2}{2k-1}$. Since the minimum degree never exceeds the average $\frac{2M}{2k+1}$,

$$\min_v d(v) \;\leq\; \frac{2M}{2k+1} \;\leq\; \frac{2k^2}{2k-1} \;=\; k + \frac{k}{2k-1} \;<\; k+1.$$

In the saturated and $m = 2$ regimes the degrees are integers, so this rounds down to k and the conjecture follows, exactly as in Proposition A.25. In the mixed regime $d(v)$ is genuinely fractional and the leftover $\frac{k}{2k-1}$ cannot be rounded away. The obstruction is not the particular bound chosen. To force $\min_v d(v) \le k$ by averaging one would need $M \le k^2 + \frac{k}{2}$, yet the one-directional bipartite weighting with $|A| = k$ and $|B| = k+1$ already carries $M = k(k+1) = k^2 + k$, so no averaging argument can finish for any m. Closing the gap needs a structural step that exhibits a specific low-degree vertex, which is precisely what Lemma A.27 supplies at $m = 3$ in order to *bypass* the conjecture rather than prove it.

Second, the conjecture is tight, and the natural way to break it fails for a reason that ties this gap to the back-arc obstruction of Conjecture 1.13. At the bipartite extremiser the minimum degree is exactly k, attained on the B side. The obvious attempt to raise it bolts a little backward weight onto the saturated bipartite point, sending $w(b, a_0) = \epsilon$ from every $b \in B$ to one fixed source $a_0 \in A$, which would lift each B-degree to $k + \epsilon$. It is infeasible for every $\epsilon > 0$. With the forward layer at full weight the backward arcs open the recirculating routes $a \to b \to a_0 \to b'$, so the flow from a source $a \neq a_0$ to a sink b' rises to $1 + k\epsilon > 1$. A single backward arc laid on a saturated forward layer manufactures an extra route, the same mechanism that keeps the upper bound of Conjecture 1.13 open and the reason the bipartite extremisers are locally rigid under the searches of Chapter 3. The minimum-degree gap and the back-arc gap are two faces of one obstruction. A systematic fractional hill-climb (`fractional_search`) with three initialisation types and eight seeds each, run at $n = 9$, 11, and 13, finds no feasible weighting with minimum weighted degree above k, confirming the bipartite point's rigidity for $k = 4$, 5, and 6. Feasibility of a fractional weighting is checked by `fractional_flows_feasible`, which calls a float-capacity Edmonds–Karp max-flow (`_float_maxflow_value`), since scipy's `csgraph` max-flow is integer-only.

Lemma A.27 (Attachment lemma)**.** *Let $m \ge 2$ and let D_0 be the one-directional complete bipartite multigraph with parts A and B, $|A| = p \ge 2$, $|B| = q \ge 2$, and $\mu(a, b) = m - 1$ for every $a \in A$, $b \in B$, with no other arcs. Let D be obtained from D_0 by adding one new vertex v together with any multiset of arcs incident to v (multiplicities at most $m - 1$). If $\lambda_D^{\max} \le m - 1$, then $d(v) \le (m - 1)\max(p, q)$.*

The proof is a bookkeeping argument on the arcs at v, and Figure A.4 is its scaffold. Sort those arcs into the four classes i_A, o_A, i_B, o_B of the figure, by direction and by which part they touch. Three combinations are impossible, because each completes an m-th arc-disjoint route between two old vertices on top of the $m - 1$ parallel arcs they already share: an i_A arc with an o_B arc makes $a \to v \to b$; an i_A arc with an o_A arc to a *different* source makes $a \to v \to a' \to b$; an i_B arc with an o_B arc to a *different* sink makes $a \to b \to v \to b'$. Forbidding these forces v into one of a few rigid shapes, a pure sink touching a single source, a pure source touching a single sink, or a small mixed remnant, and in each shape a count of two-step routes through v caps its degree. The three cases below are exactly those shapes, and the bound $(m - 1)\max(p, q)$ is whichever of them is largest.

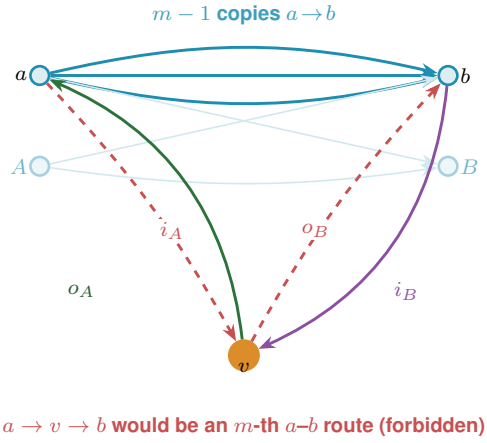


Figure A.4. The attachment lemma (Lemma A.27): a new vertex v is glued onto $(m-1)\cdot B_{p,q}$, the one-directional complete bipartite multigraph in which every $a \in A$ sends $m-1$ parallel arcs to every $b \in B$ (the bundle along the top, faint for the pairs other than a, b). The arcs at v fall into four classes by direction and side: i_A into v from A, o_A out of v to A, i_B into v from B, and o_B out of v to B. The two red legs i_A and o_B are drawn together because they are the worst case: the route $a \to v \to b$ laid on top of the $m-1$ existing $a \to b$ arcs is an m-th arc-disjoint a–b route, so $\lambda(a,b) \geq m$, which feasibility forbids. Ruling this out, with two siblings like it, caps $d(v) \leq (m-1)\max(p,q)$.

Proof. The strategy is to sort the arcs at the new vertex v into four groups by their direction and which side they touch, rule out the combinations that would manufacture an m-th route between two old vertices, and then bound whatever survives. Write $\mathrm{i}_A = \sum_a \mu(a,v)$, $\mathrm{i}_B = \sum_b \mu(b,v)$, $\mathrm{o}_A = \sum_a \mu(v,a)$, $\mathrm{o}_B = \sum_b \mu(v,b)$ for the total multiplicities into v from A, into v from B, out of v to A, and out of v to B, so that $d(v) = \mathrm{i}_A + \mathrm{i}_B + \mathrm{o}_A + \mathrm{o}_B$. Three route patterns are forbidden outright, because each adds an m-th arc-disjoint route on top of the $m-1$ parallel arcs of a bipartite pair:

- $\mu(a,v) \geq 1$ and $\mu(v,b) \geq 1$: the path $a \to v \to b$ together with the $m-1$ arcs $a \to b$ gives $\lambda(a,b) \geq m$.
- $\mu(a,v) \geq 1$ and $\mu(v,a') \geq 1$ with $a' \neq a$: the path $a \to v \to a' \to b$ (any $b \in B$) gives $\lambda(a,b) \geq m$.
- $\mu(b,v) \geq 1$ and $\mu(v,b') \geq 1$ with $b' \neq b$: the path $a \to b \to v \to b'$ (any $a \in A$) gives $\lambda(a,b') \geq m$.

Each bound below rests on one principle. The local connectivity $\lambda(s,t)$ is at least the number of pairwise arc-disjoint s-to-t routes we can exhibit, because any such family of routes is an integral flow from s to t. Feasibility says $\lambda \leq m-1$, so the moment we can name m arc-disjoint routes between two vertices we have a contradiction.

Case I: $\mathrm{i}_A \geq 1$. Some arc enters v from a vertex $a \in A$. Pattern (1) then forces $\mathrm{o}_B = 0$, because any arc $v \to b$ would complete the route $a \to v \to b$, an m-th route on the pair (a,b) laid on top of its $m-1$ parallel arcs.

Suppose first that $\mathrm{o}_A \geq 1$, so v also sends an arc to some vertex of A. Pattern (2) makes

this rigid: an arc into v from a and an arc out of v to a different $a' \neq a$ would give the route $a \to v \to a' \to b$ for any b, again an m-th route. So every arc between v and A, in either direction, must touch one single vertex a^*. Counting arc-disjoint routes from a^* to v, there are the $\mu(a^*, v)$ direct arcs, and for each $b \in B$ the two-step route $a^* \to b \to v$, which can be run $\min(\mu(a^*, b), \mu(b, v))$ times on disjoint arcs. Different b use different arcs, and none reuse a direct arc, so

$$\lambda(a^*, v) \;\geq\; \mu(a^*, v) + \sum_b \min(\mu(a^*, b), \mu(b, v)) \;=\; \mu(a^*, v) + \mathrm{i}_B,$$

the last equality because $\mu(a^*, b) = m - 1$ is at least $\mu(b, v)$, so each minimum is $\mu(b, v)$ and the sum is i_B. Feasibility caps the left side at $m - 1$, so $\mu(a^*, v) + \mathrm{i}_B \leq m - 1$. Every arc at v now touches either a^* or B, so $d(v) = \mu(a^*, v) + \mathrm{i}_B + \mu(v, a^*)$, and with the first two summing to at most $m - 1$ and the last at most $m - 1$ we get $d(v) \leq 2(m - 1)$.

Suppose instead $\mathrm{o}_A = 0$. With $\mathrm{o}_B = 0$ already, v is now a pure sink. The same two-step count applies to *every* $a \in A$ at once, giving $\lambda(a, v) \geq \mu(a, v) + \mathrm{i}_B$, hence $\mu(a, v) \leq (m - 1) - \mathrm{i}_B$ for each of the p vertices of A. Summing over A bounds the in-flow from A by $\mathrm{i}_A \leq p(m - 1) - p\,\mathrm{i}_B$, so

$$d(v) = \mathrm{i}_A + \mathrm{i}_B \leq p(m - 1) - (p - 1)\,\mathrm{i}_B \leq p(m - 1),$$

the final step because $(p - 1)\,\mathrm{i}_B \geq 0$.

Case II: $\mathrm{i}_A = 0$ *and* $\mathrm{o}_B \geq 1$. This is Case I read backwards. Reversing every arc turns $(m-1)\cdot B_{p,q}$ into $(m-1)\cdot B_{q,p}$, swaps in-arcs with out-arcs, and sends pattern (2) to pattern (3), so the same argument applies with the roles of A and B exchanged. It yields either $d(v) \leq 2(m-1)$, or v is a pure source for which $\lambda(v, b) \geq \mu(v, b) + \mathrm{o}_A \leq m - 1$ holds for every b, and summing over the q vertices of B gives $d(v) \leq q(m - 1) - (q - 1)\,\mathrm{o}_A \leq q(m - 1)$.

Case III: $\mathrm{i}_A = \mathrm{o}_B = 0$. Now v receives nothing from A and sends nothing to B, so its only arcs are the i_B in-arcs from B and the o_A out-arcs to A. For any $a \in A$ the two-step routes $a \to b \to v$ over all b give $\lambda(a, v) \geq \mathrm{i}_B$, so $\mathrm{i}_B \leq m - 1$, and for any $b \in B$ the routes $v \to a \to b$ over all a give $\lambda(v, b) \geq \mathrm{o}_A$, so $\mathrm{o}_A \leq m - 1$. Hence $d(v) = \mathrm{i}_B + \mathrm{o}_A \leq 2(m - 1)$.

Every case bounds $d(v)$ by either $2(m - 1)$ or $\max(p, q)(m - 1)$. Since $p, q \geq 2$ we have $2(m - 1) \leq (m - 1)\max(p, q)$, so in all cases $d(v) \leq (m - 1)\max(p, q)$. □

Corollary A.28 (Attachment equality)**.** *In the setting of Lemma A.27 with* $\max(p, q) \geq 3$*, suppose* $d(v) = (m - 1)\max(p, q)$*. If* $q > p$*, then* v *is a pure source with* $\mu(v, b) = m - 1$ *for every* $b \in B$*, so* $D = (m - 1) \cdot B_{p+1,q}$ *with* v *joining the source side. If* $p > q$*, then* v *is symmetrically a pure sink at full multiplicity, and if* $p = q$*, it is one of the two.*

Proof. Rerun the cases of Lemma A.27. The mixed cases cap $d(v)$ at $2(m - 1) < (m - 1)\max(p, q)$. The pure-sink case gives $d(v) \leq p(m - 1) - (p - 1)\mathrm{i}_B$, which reaches $(m - 1)\max(p, q)$ only when $p \geq q$, $\mathrm{i}_B = 0$ and every $\mu(a, v) = m - 1$. The pure-source case symmetrically forces $\mathrm{o}_A = 0$ and every $\mu(v, b) = m - 1$ when $q \geq p$. Feasibility of the resulting digraph is the bipartite construction itself. □

Theorem A.29 (Conditional odd step, $m \in \{3,4\}$)**.** *Fix $k \geq 4$ and $m \in \{3,4\}$, and assume (i) $L_m^{\mathrm{dir}}(2k) = (m-1)k^2$, and (ii) every directed multigraph attaining it is the one-directional complete bipartite multigraph $(m-1) \cdot B_{k,k}$. Then $L_m^{\mathrm{dir}}(2k+1) = (m-1)k(k+1)$.*

Proof. The lower bound is $(m-1) \cdot B_{k,k+1}$. The upper bound is a counting squeeze. If the arc count were even one over the target, then deleting any single vertex would still leave a feasible multigraph on $2k$ vertices, which by (i) cannot exceed $(m-1)k^2$, so every vertex would have to carry a large degree. Yet the average degree is only barely above that floor, so some vertex must sit exactly on it, and deleting that one lands precisely on the even extremiser $(m-1) \cdot B_{k,k}$ of (ii). The attachment lemma then caps how much that vertex could have carried, below what the surplus requires, and the contradiction forces the surplus to vanish. We now make this precise. For the upper bound let D on $n = 2k+1$ vertices have $\lambda^{\max} \leq m-1$ and $|A(D)| = (m-1)k(k+1) + j$ with $j \geq 1$. Deleting any vertex v leaves a feasible multigraph on $2k$ vertices, so by (i) $d(v) \geq |A(D)| - (m-1)k^2 = (m-1)k + j$ for *every* v. The average degree is

$$\frac{2|A(D)|}{2k+1} \;=\; (m-1)k + j + \frac{(m-1)k - (2k-1)j}{2k+1},$$

by direct computation. If $(m-1)k - (2k-1)j < 0$ the average lies below the minimum, a contradiction. Otherwise the correction term is nonnegative and, because $m \leq 4$ and $j \geq 1$, strictly below one: $(m-1)k - (2k-1)j \leq (m-3)k + 1 \leq k+1 < 2k+1$. The average therefore lies in $[(m-1)k + j,\ (m-1)k + j + 1)$ while every degree is at least $(m-1)k + j$, so some vertex v has $d(v) = (m-1)k + j$ exactly, and $D - v$ has exactly $(m-1)k^2$ arcs. By (ii), $D - v = (m-1) \cdot B_{k,k}$, and Lemma A.27 caps $d(v) \leq (m-1)k < (m-1)k + j$, the final contradiction. Hence $j \leq 0$. □

Remark A.30 (The $m = 3$ chain, sharpened). Theorem A.29 removes Conjecture A.22 from the critical path: the odd case no longer needs a minimum-degree statement, only the extremal *characterisation* one level down, and for $m = 3$ that characterisation propagates along the same induction in both parities. *Odd levels* ($n = 2k+1 \geq 9$): an extremiser has $|A| = 2k(k+1)$ arcs and every degree at least $2k$ (deletion bound). All degrees $\geq 2k+1$ is impossible since $(2k+1)^2 > 4k(k+1) = \sum_v d(v)$, so some $d(v) = 2k$ exactly, $D - v$ is the even extremiser $2 \cdot B_{k,k}$, and Corollary A.28 (the $p = q$ case) forces v to rejoin as a full-multiplicity pure source or sink: $D = 2 \cdot B_{k+1,k}$ or $2 \cdot B_{k,k+1}$. *Even levels* ($n = 2k \geq 10$): equality in the averaging step forces $2k$-regularity, hence $D - v$ extremal at $2k-1$ for *every* v. Odd uniqueness one level down leaves $2 \cdot B_{k,k-1}$ or $2 \cdot B_{k-1,k}$, and Corollary A.28 with $d(v) = 2k = 2\max(k-1, k)$, $q > p$, rebuilds exactly $2 \cdot B_{k,k}$. *The seam at $n = 8$*: the same regularity argument runs from $n = 7$, where the branches tie at 24 arcs and several extremisers coexist. On the linear branch the extremisers are richer than the double star alone: every *bidirected spanning tree at full multiplicity* $m-1$ is feasible with $2(n-1)(m-1)$ arcs (the only arcs crossing the bipartition under a tree edge are that edge's own $m-1$ copies, so every local connectivity is exactly $m-1$), and exhaustive enumeration (`enumerate_extremal_directed_multigraphs`) confirms that at $n = 4, 5$, $m = 3$ these doubled trees are the *only* extremisers (2 and 3 of them,

matching the tree shapes). A deleted vertex of the 8-regular D leaves a 24-arc extremiser with maximum degree at most 8: among doubled trees only the doubled path qualifies (tree degree ≤ 2), the double star (hub degree 24) being excluded. If $D - v$ were the doubled path P_7, the degree count would force v to attach with weight 4 to each path end, i.e. bidirected at full multiplicity, making D the doubled bidirected cycle C_8, which is infeasible since the two doubled directions around the cycle give $\lambda = 4 > 2$ between any pair. Figure A.5 draws this collapse of the doubled path into the infeasible doubled cycle. So every deletion is bipartite-type, the attachment equality applies, and $D = 2 \cdot B_{4,4}$ again. The whole $m = 3$ problem therefore rests on two finite integral statements about multigraphs with multiplicities in $\{0, 1, 2\}$: (a) $L_3^{\mathrm{dir}}(7) = 24$, equivalently $M^*(7) = 12$, and (b) every 24-arc feasible multigraph on 7 vertices with maximum degree at most 8 is $2 \cdot B_{3,4}$, $2 \cdot B_{4,3}$, or the doubled bidirected path P_7. Both targets sit at sizes the certifier and the enumerator can realistically reach with the strengthened relaxation now wired into `prove_directed_multigraph` (`use_deletion_cuts`: deletion bounds from the certified $M^*(5)$, $M^*(6)$ and the degree-pair inequality $d^+(s) + d^-(t) - w(s,t) \leq n - 1$, all proved valid), fact (a) being the single feasibility query `prove_integral_arc_bound(7, 3, 25)` and fact (b) the run `enumerate_extremal_directed_multigraphs(7, 3, 24, max_degree=8)`. For $m \geq 4$ one hole remains: at an odd extremal level the counting no longer forces a vertex of degree $(m-1)k$ (all degrees $\geq (m-1)k + 1$ is arithmetically consistent once $k(m-3) \geq 1$), so the uniqueness statement does not yet propagate through odd levels, although the *value* at odd levels follows from even uniqueness whenever that is available. For $m \geq 5$ the same escape climbs one level: a hypothetical extremiser carrying $j \geq 1$ surplus arcs could have *every* degree strictly above the deletion bound (all degrees $\geq (m-1)k + j + 1$ is consistent exactly when $(2k-1)j \leq (m-1)k - 2k - 1$, which admits $j = 1$ once $m \geq 5$), so no vertex deletion is forced to land on an extremal multigraph and even the *value* step stalls. This is why Theorem A.29 is stated for $m \in \{3, 4\}$.

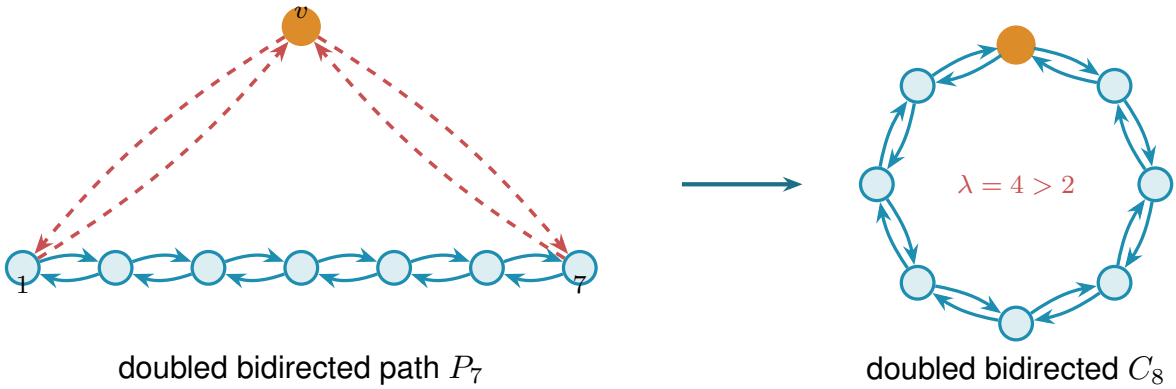


Figure A.5. The seam argument of Remark A.30. A degree count forces a deleted vertex of the 8-regular extremiser D to leave the doubled bidirected path P_7 (left, each path edge a pair of opposing arcs). To reach degree 8 the new vertex v must attach with full multiplicity to the two *ends* 1 and 7 (red), bidirected. That closes the path into the doubled bidirected cycle C_8 (right), which is *infeasible*: the two directions around the cycle give two arc-disjoint routes each way, so $\lambda = 4 > 2$ between any pair. The contradiction rules out the path branch, forcing every deletion to be bipartite-type and hence $D = 2 \cdot B_{4,4}$.

A.6 The hypergraph bounds

The body's hypergraph claims rest on two theorems: a Menger theorem for Berge routes, which also certifies the helper-point checker of Chapter 2, and a hypergraph Gomory–Hu theorem from the recent literature.

Theorem A.31 (Menger for hypergraphs)**.** *For a hypergraph $\mathcal{H}$ and vertices u, v, the maximum number of hyperedge-disjoint Berge u–v paths equals the minimum size of a set C of hyperedges whose removal separates u from v.*

Proof. Build the helper-point network of Figure 2.5: for each hyperedge e a helper arc $e^- \to e^+$ of capacity one, and arcs $x \to e^-$ and $e^+ \to x$ of unbounded capacity for each member $x \in e$. We match the two sides by translating in each direction.

Berge paths become flow. A Berge path $u, e_1, v_1, \dots, e_k, v$ becomes the directed walk $u \to e_1^- \to e_1^+ \to v_1 \to \cdots \to v$, crossing one helper arc per hyperedge it uses. The e_i along a single Berge path are distinct, and two hyperedge-disjoint Berge paths share no hyperedge at all, so they use disjoint sets of helper arcs and translate to *arc-disjoint* flow paths, paths that share no arc, each carrying one unit of flow.

Flow becomes Berge paths. Conversely, because every capacity is an integer, a maximum flow may be taken integral, and an integral flow of value k splits into k arc-disjoint unit paths. Reading a helper crossing $\cdots x \to e^- \to e^+ \to y \cdots$ back as the single hyperedge step x, e, y, that is, *suppressing* the two helper nodes, turns each unit path into a Berge path, and no two of them can share a hyperedge, since that would send two units across some capacity-one helper arc.

Cuts. Finally, the member arcs are uncapped, so any cut of finite capacity uses helper arcs only, and a set of helper arcs separates u from v in the network exactly when the corresponding hyperedges separate them in $\mathcal{H}$. The max-flow min-cut theorem equates the largest number of arc-disjoint flow paths with the smallest helper-arc cut, which is precisely the claim. □

Theorem A.32 (Hypergraph Gomory–Hu, Bérczi et al. [8])**.** *Every r-uniform hypergraph $\mathcal{H}$ on vertex set V has a weighted tree T^* on V, its* hypergraph Gomory–Hu tree*, such that $\lambda_{\mathcal{H}}(u, v)$ equals the minimum weight on the u–v tree path, and each tree edge t induces a vertex partition $(S_t, V \setminus S_t)$ with $|\delta_{\mathcal{H}}(S_t)| = c^*(t)$, where $\delta_{\mathcal{H}}(S)$ is the set of hyperedges incident to both sides, that is, having at least one vertex in S and at least one outside it.*

This is the exact hypergraph echo of Theorem A.2, the same tree-of-cuts summary of all pairwise connectivities, with the single change that the capacity of a cut now counts the hyperedges straddling it, $|\delta_{\mathcal{H}}(S)|$, in place of the edges. Bérczi, Chandrasekaran, Király and Kulkarni [8] obtain the tree by the same uncrossing strategy sketched for the graph case above, applied now to the submodular hyperedge-cut function $|\delta_{\mathcal{H}}(\cdot)|$. We use only the two properties in the statement, the path-minimum reading of $\lambda_{\mathcal{H}}$ and the identity $|\delta_{\mathcal{H}}(S_t)| = c^*(t)$.

Proof of Proposition 1.14. Upper bound. Let $\mathcal{H}$ be r-uniform with $\lambda_{\mathcal{H}}^{\max} \le m-1$, and let T^* be its hypergraph Gomory–Hu tree (Theorem A.32), a weighted tree on the same n vertices with weights c^*. The plan is to charge each hyperedge to tree edges and count the charges two ways, exactly as in the Mader proof, with the one change that a hyperedge spans r vertices rather than two.

Step 1: each hyperedge crosses at least $r-1$ tree cuts. Fix a hyperedge e, a set of r vertices, and let $\mathrm{ST}(e)$ be its *smallest connected subtree*, the minimal subtree of T^* that contains all r of them (its Steiner tree). A tree spanning at least r vertices has at least $r-1$ edges, so $\mathrm{ST}(e)$ has at least $r-1$ of them. Each such edge t is a genuine cut for e: deleting t from T^* splits the vertices into the two sides $(S_t, V \setminus S_t)$, and because t lies on $\mathrm{ST}(e)$ the hyperedge e has at least one vertex on each side. So e touches both sides, that is, $e \in \delta_{\mathcal{H}}(S_t)$. Therefore

$$\big|\{\, t \in E(T^*) : e \in \delta_{\mathcal{H}}(S_t)\,\}\big| \;\ge\; r-1 \qquad \text{for every hyperedge } e.$$

Step 2: count the incidences two ways. Sum that inequality over all hyperedges, then exchange the order of summation. This is legitimate because both of the middle sums below simply count the same pairs (e,t) with $e \in \delta_{\mathcal{H}}(S_t)$, once grouped by the hyperedge and once by the tree edge:

$$(r-1)\,|E(\mathcal{H})| \;\le\; \sum_{e} \big|\{\, t : e \in \delta_{\mathcal{H}}(S_t)\,\}\big| \;=\; \sum_{t} \big|\{\, e : e \in \delta_{\mathcal{H}}(S_t)\,\}\big| \;=\; \sum_{t} |\delta_{\mathcal{H}}(S_t)|.$$

Step 3: read off the bound. By the Gomory–Hu property each tree edge has $|\delta_{\mathcal{H}}(S_t)| = c^*(t)$, and each weight $c^*(t)$ is a local connectivity $\lambda_{\mathcal{H}}$ of its endpoints, hence at most $m-1$ by feasibility. Summing over the $n-1$ tree edges, $\sum_t |\delta_{\mathcal{H}}(S_t)| = \sum_t c^*(t) \le (m-1)(n-1)$. Chaining Steps 2 and 3,

$$(r-1)\,|E(\mathcal{H})| \;\le\; (m-1)(n-1),$$

and dividing by $r-1$, then rounding down because $|E(\mathcal{H})|$ is an integer, gives $|E(\mathcal{H})| \le \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$. *Lower bound.* When $(r-1) \mid (n-1)$, take a hub h and split the other $n-1$ vertices into groups $G_1, G_2, \dots$ of size $r-1$ each. Form the star hypertree whose hyperedges are $\{h\} \cup G_i$, one per group, and give each hyperedge multiplicity $m-1$. Every non-hub vertex lies in exactly $m-1$ hyperedges, which form a cut isolating it, so by Theorem A.31 every pair has Berge connectivity at most $m-1$, while the count is $\frac{(m-1)(n-1)}{r-1}$. For general n under the hypothesis $m-1 \le \binom{n-2}{r-2}$, Theorem A.33 below attains the floor exactly, without even needing repeated hyperedges. □

The multiplicities in the lower bound can in fact be avoided: for $r \ge 3$, *simple* r-uniform hypergraphs already attain the same bound, in sharp contrast to the graph case $r=2$, where simplicity costs the factor that separates Mader's $\lfloor m(n-1)/2 \rfloor$ from the multigraph value $(m-1)(n-1)$.

Theorem A.33 (Simplicity is free for $r \geq 3$)**.** *Let $r \geq 3$, $n \geq r$, and $2 \leq m$ with $m - 1 \leq \binom{n-2}{r-2}$. Then the maximum number of hyperedges in a* simple *r-uniform hypergraph on n vertices with Berge edge-connectivity at most $m - 1$ is exactly $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$.*

Proof. The upper bound is Proposition 1.14, since a simple hypergraph is a multihypergraph. For the lower bound, fix a hub h and let $V' = V \setminus \{h\}$. By Lemma A.35 applied with rank $r-1$ and degree ceiling $d = m-1$ on the $n-1$ vertices of V', there is an $(r-1)$-uniform simple hypergraph $\mathcal{G}^*$ on V' with maximum degree at most $m-1$ and exactly $\left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor$ hyperedges. Adding h to each hyperedge produces distinct r-sets, so the result $\mathcal{H}$ is simple, and every non-hub vertex w lies in $\deg_{\mathcal{G}^*}(w) \leq m-1$ hyperedges, which form a cut isolating w. By Theorem A.31, $\lambda^{\max}_{\mathcal{H}} \leq m - 1$. □

The sparse hypergraph the construction needs is a corollary of a classical partition theorem, which we state before using it.

Theorem A.34 (Baranyai [15])**.** *If r divides n, the $\binom{n}{r}$ distinct r-subsets of an n-element set can be partitioned into $\binom{n-1}{r-1}$ classes, each a* perfect matching*, meaning a family of n/r disjoint r-sets that together cover every vertex exactly once. More generally, for any prescribed class sizes $a_1 + \cdots + a_t = \binom{n}{r}$, and with no divisibility assumed, the r-subsets can be partitioned into classes of exactly those sizes so that in the class of size a_i every vertex lies in either $\lfloor ra_i/n \rfloor$ or $\lceil ra_i/n \rceil$ of the sets, as evenly balanced as the size permits.*

The first statement is the memorable one. When r divides n the complete r-uniform hypergraph, the one holding every r-set, comes apart into $\binom{n-1}{r-1}$ perfect matchings, the exact hypergraph analogue of splitting a regular bipartite graph into perfect matchings by König's theorem. The proof is an induction that keeps the partial partition as balanced as possible at every step, an integer-flow argument in the spirit of Hall's marriage theorem rather than an explicit construction, and its full form is in Baranyai [15]. We need only the general equitable form, and only its conclusion about degrees.

Lemma A.35 (Sparse uniform hypergraphs exist)**.** *Let $r \geq 2$, $n \geq r$, and $0 \leq d \leq \binom{n-1}{r-1}$. There exists an r-uniform simple hypergraph on n vertices with maximum degree at most d and exactly $\left\lfloor \frac{dn}{r} \right\rfloor$ hyperedges.*

Proof. Put $e := \left\lfloor \frac{dn}{r} \right\rfloor \leq \frac{n}{r}\binom{n-1}{r-1} = \binom{n}{r}$. Apply the equitable form of Theorem A.34 with two classes, of sizes e and $\binom{n}{r} - e$. Within each class every vertex is covered either $\lfloor re/n \rfloor$ or $\lceil re/n \rceil$ times, so the class of size e is a simple r-uniform hypergraph with $e = \left\lfloor \frac{dn}{r} \right\rfloor$ hyperedges and maximum degree at most $\lceil re/n \rceil \leq \lceil d \rceil = d$, the last step because $re/n \leq r(dn/r)/n = d$ and d is an integer. □

A.7 The hypergraph vertex problem

The whole vertex problem translates into a graph problem through one observation, the one the rest of this section rests on and which Figure A.6 draws in full. Write $I(\mathcal{H})$ for the bipartite *incidence graph* of a hypergraph $\mathcal{H}$: one node per vertex, one node per hyperedge, an edge when the vertex lies in the hyperedge. A Berge path with distinct hyperedges is exactly a path of $I(\mathcal{H})$ between two vertex-class nodes, hyperedge-disjoint Berge paths use disjoint hyperedge-nodes, and internally vertex-disjoint ones use disjoint interior vertex-nodes. Hence

$$\kappa_{\mathcal{H}}(u, v) \;=\; \kappa_{I(\mathcal{H})}(u, v),$$

the maximum number of internally disjoint u–v paths in $I(\mathcal{H})$, and the hypergraph vertex problem asks for the maximum number of degree-r nodes on one side of a bipartite graph whose other side holds n nodes, subject to no two of those n nodes being joined by m internally disjoint paths. Figure A.6a traces a single Berge path through both pictures, and Figure A.6b shows the star hypertree of Theorem A.36, whose incidence graph is a tree.

Theorem A.36 (Hypergraph vertex value at $m = 2$)**.** *For $r \geq 2$ and $n \geq 1$, the maximum number of hyperedges of an r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 1$ is*

$$k_2^{(r)}(n) \;=\; \left\lfloor \frac{n-1}{r-1} \right\rfloor,$$

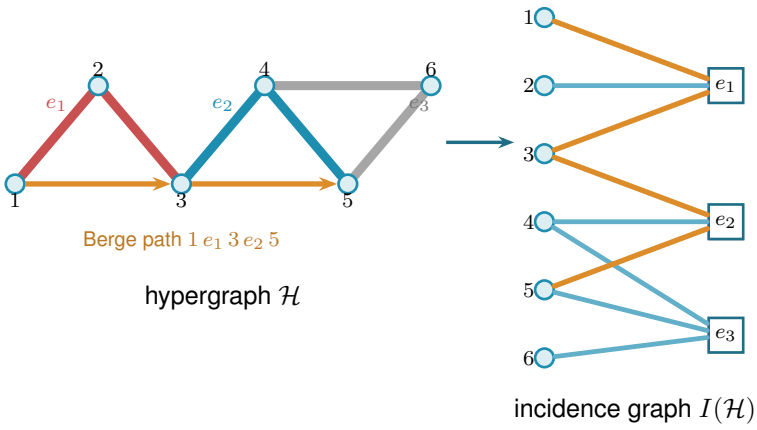
attained by the star hypertree. In particular $k_2^{(r)}(n) = \ell_2^{(r)}(n)$: the vertex and edge problems agree at $m = 2$ for every uniformity, and repeated hyperedges never help.

Proof. We claim $\kappa_{\mathcal{H}}^{\max} \leq 1$ holds if and only if $I(\mathcal{H})$ is a forest. If $I(\mathcal{H})$ contains a cycle $v_1, e_1, v_2, e_2, \ldots, v_\ell, e_\ell, v_1$ (it alternates classes, so $\ell \geq 2$, all v_i and e_i distinct), then v_1, e_1, v_2 and $v_1, e_\ell, v_\ell, e_{\ell-1}, \ldots, e_2, v_2$ are two Berge v_1–v_2 paths with disjoint hyperedge sets and disjoint interior vertex sets, so $\kappa(v_1, v_2) \geq 2$. (At $\ell = 2$ this is the repeated pair: two hyperedges sharing two vertices.) Conversely, two such paths between u and v concatenate to a cycle of $I(\mathcal{H})$. This proves the claim. A forest on $n + q$ nodes has at most $n + q - 1$ edges, while $I(\mathcal{H})$ has exactly rq edges, so $rq \leq n + q - 1$, i.e. $q \leq (n-1)/(r-1)$, and q is an integer. The star hypertree has $\lfloor (n-1)/(r-1) \rfloor$ hyperedges and a tree as incidence graph (each block vertex lies in one hyperedge, every cycle would need two hyperedges sharing two vertices), so the floor is attained. □

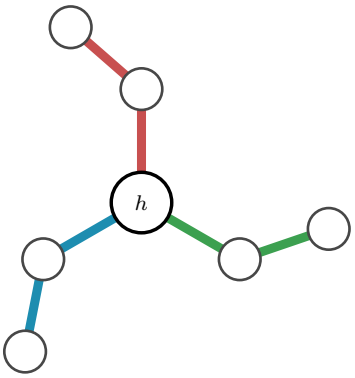
Proposition A.37 (General lower bound)**.** *For $r \geq 3$, $m \geq 2$ and $m - 1 \leq \binom{n-2}{r-2}$,*

$$k_m^{(r)}(n) \;\geq\; \left\lfloor \frac{(m-1)(n-1)}{r-1} \right\rfloor,$$

attained by a simple hypergraph.



(a) A 3-uniform hypergraph $\mathcal{H}$ (left) and its bipartite incidence graph $I(\mathcal{H})$ (right): circles for vertices, hollow squares for hyperedges, an edge when the vertex lies in the hyperedge. The Berge path $1\, e_1\, 3\, e_2\, 5$ is traced in orange in both pictures. In $\mathcal{H}$ it rides the red line e_1 and the blue line e_2, changing at the shared vertex 3, while the grey line e_3 lies off it. In $I(\mathcal{H})$ it is the ordinary path $1-e_1-3-e_2-5$ alternating sides.



(b) A star hypertree: every hyperedge $\{h\} \cup G_i$ shares only the hub h (orange), and the other $r-1=2$ vertices of each block sit in a single hyperedge. Its incidence graph is a tree, so $\kappa^{\max} \leq 1$.

Figure A.6. The translation the vertex problem rests on. Internally vertex-disjoint Berge paths in $\mathcal{H}$ correspond exactly to internally disjoint paths in $I(\mathcal{H})$, so $\kappa_{\mathcal{H}}(u,v) = \kappa_{I(\mathcal{H})}(u,v)$. **(top)** A hypergraph and its incidence graph with one Berge path drawn in both. **(bottom)** The star hypertree of Theorem A.36, whose incidence graph is a tree.

Proof. The simple construction of Theorem A.33 has Berge edge-connectivity at most $m-1$, and Whitney's inequality $\kappa \leq \lambda$ holds pairwise (an internally vertex-disjoint family is in particular hyperedge-disjoint under our definition), so the same hypergraph is feasible for the vertex problem. □

The $m = 3$ case also closes completely. The engine is a rank bound for graphs in which one side of a partition must stay below local 3-connectivity. The hypergraph theorem then falls out of the incidence translation. Throughout, $\operatorname{rank}(G) = |E(G)| - |V(G)| + 1$ denotes the cycle rank of a connected multigraph, that is, the number of independent cycles it carries. A tree has cycle rank 0, and each edge added beyond a spanning tree closes exactly one new cycle and raises the rank by one, so bounding the rank is the same as bounding how many independent cycles the graph can hold.

Primer: global connectivity and the pieces of a graph

The lemma below speaks of a graph being 2-connected or 3-connected and of taking it apart at small separators. These are the global cousins of the local $\kappa(u, v)$ from Chapter 1, so we fix the words once. A graph on more than k vertices is *k-connected* if it stays connected after the removal of any $k-1$ vertices. By Menger this is the same as asking that every pair of vertices be joined by at least k internally vertex-disjoint paths, so k-connected means $\kappa(u, v) \geq k$ for all pairs at once, the uniform version of the local measure. Thus 2-connected means no single vertex disconnects the graph, and 3-connected means no two vertices do. Three named obstructions go with this. A *cut vertex* is a vertex whose removal disconnects the graph, so a connected graph on at least three vertices is 2-connected exactly when it has no cut vertex. A *separating pair* is a set of two vertices whose removal disconnects the graph, the obstruction a 2-connected graph must lack in order to be 3-connected. Finally a *block* is a maximal piece with no cut vertex of its own, hence either a single bridge edge or a maximal 2-connected subgraph. Two blocks overlap in at most one vertex, always a cut vertex, and the blocks and cut vertices hang together in a tree, the *block-cut tree*, with one node per block and one per cut vertex. Step 1 of the proof is precisely an induction over that tree.

Lemma A.38 (Incidence rank lemma)**.** *Let G be a connected multigraph with vertex set $X \cup Z$ such that (i) Z is independent, (ii) no edge incident to Z is parallel to another edge, (iii) every $z \in Z$ has degree at least 2, and (iv) $\kappa_G(x, x') \leq 2$ for all distinct $x, x' \in X$. Then $\operatorname{rank}(G) \leq |X| - 1$.*

Proof. Induction on $|V| + |E|$. The proof peels G down to a core that cannot exist. Step 1 handles a cut vertex by arguing block by block. Steps 2 and 3 remove a degree-two Z-vertex or X-vertex without changing the bound. What survives is 2-connected with minimum degree at least three, and Step 4 shows that such a graph already violates the connectivity ceiling (iv), so it never arises. *Base* $|V| \leq 2$. A single vertex lies in X by (iii), and $0 \leq |X| - 1$. On two vertices, (i)–(iii) force both into X, parallel edges are internally disjoint paths, so (iv) caps the multiplicity at 2 and $\operatorname{rank} \leq 1 = |X| - 1$.

Step 1: a cut vertex. Suppose G has a cut vertex, so it is not yet 2-connected. Break it into its *blocks* $B_1, \dots, B_c$ with $c \geq 2$, a block being a maximal piece with no cut vertex of its own, that is, either a single bridge edge or a maximal 2-connected subgraph. Two blocks meet in at most one vertex, and any shared vertex is a cut vertex of G. No cycle can pass through two different blocks, so the cycle rank adds up over blocks, $\operatorname{rank}(G) = \sum_i \operatorname{rank}(B_i)$, and it is enough to bound each $\operatorname{rank}(B_i)$ by $|X \cap B_i| - 1$ and sum.

Each block inherits the hypotheses. A bridge block is one edge, so its rank is 0, and by (i) at least one endpoint lies in X, giving $0 \leq |X \cap B| - 1$. Every other block is 2-connected, hence has minimum degree at least 2, so (i) to (iv) hold inside it and the induction hypothesis gives $\operatorname{rank}(B_i) \leq |X \cap B_i| - 1$.

Adding these requires care, because a cut vertex is shared by several blocks and would be counted several times. Write $b(v)$ for the number of blocks containing v, so $b(v) = 1$ except at cut vertices. Counting each X-vertex once for every block it lies in,

$$\sum_i |X \cap B_i| = |X| + \sum_{x \in X \text{ cut}} (b(x) - 1).$$

The number of blocks satisfies the block–cut tree identity $c = 1 + \sum_{v \text{ cut}} (b(v) - 1)$, which holds because the blocks and cut vertices form a tree in which each cut vertex v is joined to the $b(v)$ blocks that contain it. Subtracting this count of blocks from the previous display and separating the cut vertices into their X- and Z-parts leaves

$$\operatorname{rank}(G) = \sum_i \operatorname{rank}(B_i) \leq \sum_i (|X \cap B_i| - 1) = |X| - 1 - \sum_{z \in Z \text{ cut}} (b(z) - 1) \leq |X| - 1,$$

where the last inequality holds because each $b(z) - 1 \geq 0$, so the Z-cut sum only pushes the bound further below $|X| - 1$.

Step 2: a Z-vertex of degree two. From now on G is 2-connected, every cut vertex having been handled. Suppose some $z_0 \in Z$ has degree exactly 2. Both its neighbours lie in X by (i), and they are distinct, since two edges to the same vertex would be parallel at a Z-vertex, which (ii) forbids. *Suppress* z_0 by deleting it and joining its two neighbours x, x' with one new edge. This loses two edges and one vertex and gains one edge, so $|E|$ and $|V|$ each drop by one and $\operatorname{rank} = |E| - |V| + 1$ is unchanged. The new edge runs X to X, so $|X|$ is unchanged and (i), (ii), (iii) still hold for the remaining Z-vertices. Replacing the path x, z_0, x' by a direct edge alters no route among the surviving vertices, so every $\kappa(x_1, x_2)$ is preserved and (iv) holds. The graph is strictly smaller, so the induction hypothesis applies and returns the bound for G.

Step 3: an X-vertex of degree two. Now G is 2-connected and, Step 2 being used up, every Z-vertex has degree at least 3. Suppose some $x_0 \in X$ has degree 2, and delete it. Deleting one vertex from a 2-connected graph leaves it connected. Stripping a degree-2 vertex removes two edges and one vertex, so rank drops by exactly one. Each neighbour of x_0 loses one edge, and a neighbour in Z only falls from degree at least 3 to degree at least 2, so (iii) survives, while deleting

a vertex cannot raise a connectivity, so (iv) survives. Here $|X|$ drops by one, and $|X| \geq 2$ to begin with, for if $|X| \leq 1$ then by (i) and (ii) every z would send all its edges to one X-vertex without repeats, hence have degree at most 1, against (iii). The smaller graph satisfies the hypotheses, so induction gives $\mathrm{rank}(G) - 1 \leq (|X| - 1) - 1$, that is $\mathrm{rank}(G) \leq |X| - 1$.

Step 4: the remaining case is impossible. Now G is 2-connected with $|V| \geq 3$ and every degree at least 3. First, G is simple: a parallel class lies inside X by (ii), and if $\mu(x, x') \geq 2$ then a third internally disjoint route exists: walk from x to any third vertex w inside $G - x'$, then from w to x' inside $G - x$. The concatenation visits x only first and x' only last, so it contains an x–x' path with at least one interior vertex, giving $\kappa(x, x') \geq 3$, against (iv). If G is 3-connected, Menger puts every pair at $\kappa \geq 3$, so $|X| \leq 1$ and every z has degree at most 1 by (i), absurd.

Otherwise G is 2-connected but not 3-connected, so it can be taken apart at its 2-vertex separators.

Primer: the triconnected (SPQR) decomposition

A 2-connected graph that is not 3-connected must have a separating pair, and cutting along every such pair sorts any 2-connected graph into a few standard shapes. This is a classical theorem of Tutte [16], made into a linear-time algorithm by Hopcroft and Tarjan [17].

Theorem (triconnected decomposition). Every 2-connected graph G on at least three vertices decomposes into a *tree of pieces*, unique up to relabelling, in which each piece is one of three kinds: a *cycle* of length at least three (an S-piece, for *series*), a *bond*, meaning two vertices joined by three or more parallel edges (a P-piece, for *parallel*), or a 3-connected simple graph on at least four vertices (an R-piece, for *rigid*). Two adjacent pieces share exactly one *virtual edge* $\{a, b\}$, a placeholder that records a separating pair of G at which the two pieces were split apart. The three initials S, P, R, together with Q for a trivial single-edge piece, give the structure its usual name, the *SPQR tree*.

The tree is built by the recursion that also defines it. If G is already a cycle, a bond, or 3-connected, it is a single piece and the tree is one node. Otherwise choose a separating pair $\{a, b\}$, split G into the parts that this pair separates, add the virtual edge $\{a, b\}$ to each part so the split is remembered, and recurse on the parts. Every split makes the parts strictly smaller, so the recursion halts, and Tutte's theorem is that the resulting collection of pieces does not depend on the order in which the separating pairs are taken. The one fact the proof draws from all this is that a virtual edge $\{a, b\}$ always stands for a genuine a–b path through the rest of G, so an argument that must cross from a to b outside a given piece can always do so on the far side of that piece's virtual edge.

The decomposition just stated is sketched in Figure A.7, which also marks the leaf at which the argument bites. Two properties are all the proof needs from it, and both are visible in the figure. First, a vertex of a piece that touches none of that piece's virtual edges belongs to that piece alone, so it keeps in G exactly the degree it has in the piece. Second, for any virtual edge $\{a, b\}$ the part of G on the far side of the cut contains a genuine a–b path whose interior avoids the piece, so the virtual edge can always be replaced by a real route.

Now G itself is none of the three piece types, since it is not a single cycle (its degrees are at least three), not a bond (it is simple), and not 3-connected (the case just handled), so the tree has at least two leaves. Let L be one, with unique virtual edge $\{a, b\}$. If L is a cycle, a vertex of L off $\{a, b\}$ has degree 2 in G, absurd. If L is a bond, it holds at least two real parallel edges, absurd in a simple graph. If L is 3-connected, then any two of its vertices are joined by three internally disjoint paths inside L, at most one through the virtual edge, which the far-side path replaces: so $\kappa_G(u, v) \geq 3$ for *all* $u, v \in V(L)$, forcing $|X \cap V(L)| \leq 1$ by (iv). But $|V(L)| \geq 4$ leaves a Z-vertex $z^* \notin \{a, b\}$. Its neighbours lie in $X \cap V(L)$ by (i), at most one vertex, reached by at most one edge since G is simple, giving degree at most 1, which is absurd. □

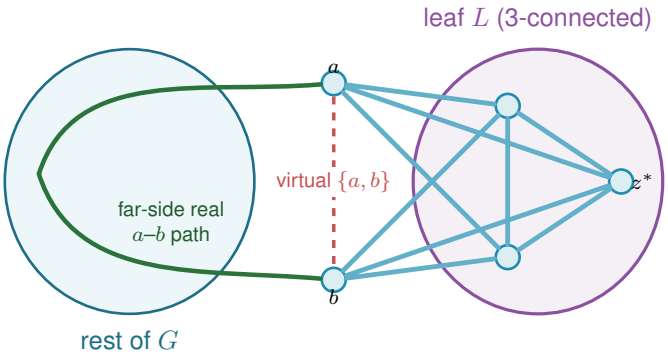


Figure A.7. The triconnected (Tutte/SPQR) decomposition in Step 4 of Lemma A.38, schematic: G splits into a tree of pieces (blobs) glued along virtual edges, and the analysis works at a *leaf* L with its single virtual edge $\{a, b\}$ (dashed red). The far side of the tree supplies a *real* a–b path (green) that internally avoids L. If L is 3-connected, any two of its vertices are joined by three internally disjoint paths inside L, at most one using the virtual edge, and the far-side path replaces it, so $\kappa_G \geq 3$ for every pair in L. With $|V(L)| \geq 4$ that forces a hyperedge-node $z^* \notin \{a, b\}$ of degree at most one, which is the contradiction that closes the lemma.

Theorem A.39 (Hypergraph vertex value at $m = 3$)**.** *For $r \geq 2$, every r-uniform multihypergraph on n vertices with $\kappa^{\max} \leq 2$ has at most $\left\lfloor \frac{2(n-1)}{r-1} \right\rfloor$ hyperedges. For $r \geq 3$ with $2 \leq \binom{n-2}{r-2}$ the bound is attained by a simple hypergraph, so*

$$k_3^{(r)}(n) \;=\; \left\lfloor \frac{2(n-1)}{r-1} \right\rfloor \;=\; \ell_3^{(r)}(n).$$

Proof. Apply Lemma A.38 to each connected component of the incidence graph $I(\mathcal{H})$, with X the vertex class and Z the hyperedge class: Z is independent, incidences are simple even when hyperedges repeat, hyperedge-nodes have degree $r \geq 2$, components without an X-node are impossible, and $\kappa_I(u, v) = \kappa_{\mathcal{H}}(u, v) \leq 2$ for vertex pairs. Sum $\mathrm{rank} \leq |X_c| - 1$ over the C components. The incidence graph $I(\mathcal{H})$ has rq edges (each of the q hyperedges contributes r incidences) and $n + q$ nodes, so $\sum_c \mathrm{rank} = rq - (n + q) + C$, while $\sum_c (|X_c| - 1) = n - C$. The inequality reads $rq - (n + q) + C \leq n - C$, hence $q(r - 1) \leq 2(n - 1) + 2(1 - C) \leq 2(n - 1)$, the last step using $C \geq 1$. The lower bound is Proposition A.37 at $m = 3$. □

Remark A.40 (Edges as gates at $r = 2$, and the boundary of the method). At $r = 2$ the theorem speaks of multigraphs under the hyperedge-as-gate convention, in which parallel edges are

distinct one-step routes, giving the cactus value $2(n-1)$, complementing the body's convention for graphs, where parallel adjacencies count once. The proof of Lemma A.38 leans on the tricon-nected decomposition exactly where $\kappa \le 2$ lives. For $m = 4$, the analogous question asks for a rank bound relative to $\kappa \le 3$, where no comparably clean decomposition exists, and the graph case ($m = 5$, Sørensen–Thomassen) warns that the pattern $k_m^{(r)} = \ell_m^{(r)}$ must fail eventually. Ex-haustive computation confirms $k_4^{(3)}(5) = 6 = \lfloor 3 \cdot 4/2 \rfloor$, so the agreement survives at least one step beyond the theorem, and where it first breaks for $r \ge 3$ is open.

Proposition A.41 (Directed hypergraphs: first bounds)**.** *Model a directed r-uniform hyperedge as a tail vertex together with $r-1$ head vertices, a Berge route entering at the tail and leaving at a head (the program's convention, drawn in Figure A.8). If every pair of vertices has at most $m-1$ arc-disjoint directed Berge routes, then*

$$|E(\mathcal{H})| \;\le\; \frac{(m-1)\,n(n-1)}{r-1},$$

and for $m-1 \le \binom{n-\lceil n/2\rceil-1}{r-2}$ there are feasible constructions with $\max_{1\le\alpha\le n-1}\ \alpha\left\lfloor \frac{(m-1)(n-\alpha)}{r-1}\right\rfloor \approx (m-1)n^2/(4(r-1))$ hyperedges, so the value is quadratic in n.

Proof. Upper bound: a hyperedge with tail u serves the $r-1$ ordered pairs (u,h), h a head. If some ordered pair (u,v) were served by m distinct hyperedges, those would be m arc-disjoint one-step routes, so each ordered pair is served at most $m-1$ times and $(r-1)|E| \le (m-1)n(n-1)$. Lower bound: split V into tails A, $|A| = \alpha$, and heads B. By Lemma A.35 there is a simple $(r-1)$-uniform hypergraph $\mathcal{G}^*$ on B with maximum degree $m-1$ and $\lfloor (m-1)|B|/(r-1)\rfloor$ edges. Give every tail $a \in A$ the hyperedges (a,H), $H \in \mathcal{G}^*$. No vertex of B is a tail, so every route is a single step and $\lambda(a,b) = \deg_{\mathcal{G}^*}(b) \le m-1$, while pairs inside A or starting in B have no routes. □

Figure A.8. How the thesis draws a single hyperedge on its r vertices, in the metro colours of Figure 1.3. *Left:* an undirected hyperedge is one metro line threading all its members, with no arrows. *Right:* a directed hyperedge, a tail with $r-1$ heads, is a star, the tail sending one thick arc to each head, so a Berge route enters at the tail and leaves at any head. The hypergraph panels of Figure 1.4 use this same star. The shape is a drawing convention, not a path. Each coloured line or star is a *single* edge on several vertices, not a sequence of ordinary edges, so the line on the left is one hyperedge and not three. One colour means one hyperedge, and where two of them meet they cross at a shared station rather than joining into a longer line. What might be mistaken for a hyperedge is a *route*, which does run edge by edge from one vertex to another, but a route is drawn as a thin thread laid over the lines, so the thick coloured hyperedge and the thin route never look alike.

The forward model is one of three ways to orient a Berge edge. In general a directed r-uniform hyperedge is a split of its r vertices into a non-empty *tail set* T and a non-empty *head set* H, a route entering at a tail and leaving at a head; the *forward* model is $|T| = 1$, the *backward* model is $|H| = 1$, and the *general* model allows any split (genuinely mixed splits, both parts at least two, exist from $r \geq 4$). The next two results place all three. The first is that backward needs no separate study, and the second is that the general model, despite admitting more hyperedges on a single vertex set, obeys the very same first-order bound as forward.

Lemma A.42 (Backward is the reverse of forward)**.** *Reversing every hyperarc, $(T, H) \mapsto (H, T)$, is an edge-count-preserving bijection between the forward directed r-uniform hypergraphs and the backward ones on the same vertices, and the reverse $\mathcal{H}^R$ satisfies $\lambda_{\mathcal{H}}(u, v) = \lambda_{\mathcal{H}^R}(v, u)$ and $\kappa_{\mathcal{H}}(u, v) = \kappa_{\mathcal{H}^R}(v, u)$. Hence $\lambda^{\max}$ and $\kappa^{\max}$ are invariant under reversal, and the backward edge and vertex extremal numbers equal the forward ones, for every r, m and n.*

Proof. Reversal sends a backward hyperarc ($|T| = r - 1$, $|H| = 1$) to a forward one ($|T| = 1$) and is its own inverse, so it is an edge-count-preserving bijection of the two families. Let $u = x_0, e_1, x_1, \ldots, e_k, x_k = v$ be a directed Berge route in $\mathcal{H}$, where x_{i-1} is a tail and x_i a head of e_i. The reversed edge $e_i^R = (H_i, T_i)$ has x_i as a tail and x_{i-1} as a head, so $v = x_k, e_k^R, x_{k-1}, \ldots, e_1^R, x_0 = u$ is a directed Berge route from v to u in $\mathcal{H}^R$ on the same edges and the same internal vertices. This is a bijection between u–v routes in $\mathcal{H}$ and v–u routes in $\mathcal{H}^R$ that preserves which edges and which internal vertices any two routes share, so it carries an edge-disjoint (respectively internally vertex-disjoint) family to one of equal size. Therefore $\lambda_{\mathcal{H}}(u, v) = \lambda_{\mathcal{H}^R}(v, u)$ and $\kappa_{\mathcal{H}}(u, v) = \kappa_{\mathcal{H}^R}(v, u)$, and maximising over ordered pairs gives $\lambda^{\max}(\mathcal{H}) = \lambda^{\max}(\mathcal{H}^R)$ and the same for κ. Since reversal is a bijection preserving feasibility and edge count, the two models share every extremal number. □

Proposition A.43 (The general model obeys the forward bound)**.** *Let a directed r-uniform hyperedge be any split into a non-empty tail set T and head set H. If every ordered pair has at most $m - 1$ arc-disjoint, or at most $m - 1$ internally vertex-disjoint, directed Berge routes, then*

$$|E(\mathcal{H})| \;\leq\; \frac{(m-1)\, n(n-1)}{r-1},$$

the same bound as the forward model of Proposition A.41. The forward construction of that proposition is itself a set of general hyperedges, so the general value is quadratic in n with the same leading constant.

Proof. Fix an ordered pair (u, v). A hyperedge (T, H) with $u \in T$ and $v \in H$ carries the one-step route $u \to v$ that enters at the tail u and leaves at the head v. Distinct such hyperedges give routes that share no edge and no internal vertex, so if k hyperedges had u in their tail set and v in their head set then $\lambda(u, v) \geq k$ and $\kappa(u, v) \geq k$; feasibility forces $k \leq m - 1$. Each hyperedge (T, H) is counted by exactly the $|T|\,|H|$ ordered pairs (u, v) with $u \in T$ and $v \in H$, so summing the per-pair bound over all pairs gives $\sum_{(T,H)} |T|\,|H| \leq (m-1)\, n(n-1)$. As $|T| + |H| = r$ with

both parts at least one, $|T|\,|H| \geq 1 \cdot (r-1) = r-1$, whence $(r-1)|E| \leq (m-1)n(n-1)$. The forward family of Proposition A.41 consists of general hyperedges, so it witnesses the same $\approx (m-1)n^2/(4(r-1))$ lower bound. □

A.8 The random threshold

The probabilistic ingredients are two Chernoff bounds, quoted in the form we use them.

Primer: what a Chernoff bound says

Both proofs below need to say that a sum of many independent yes-or-no trials almost never lands far from its average. That is the content of a *Chernoff bound*, and the intuition is worth stating even though we only quote the result. Let $X = X_1 + \cdots + X_N$ count the successes among independent trials, with mean $\mu = \mathbb{E}[X]$. A Chernoff bound says the chance that X overshoots μ by a factor $1+\delta$, or undershoots it by a factor $1-\delta$, decays *exponentially* in μ, not merely polynomially as Chebyshev's inequality would give. The mechanism is a single trick. Rather than bound $\Pr[X \geq a]$ directly, bound $\Pr[e^{tX} \geq e^{ta}]$ by Markov's inequality for a parameter $t > 0$, use independence to factor $\mathbb{E}[e^{tX}] = \prod_i \mathbb{E}[e^{tX_i}]$ into a product over the trials, and then choose t to make the resulting bound as small as possible. The exponential in the answer is exactly the e^{tX} that was fed through Markov. We quote the two tails in the shape the threshold proof consumes.

Theorem A.44 (Chernoff bounds, see [18, Cor. 4.4, Thm. 4.4])**.** *Let X be a sum of independent $\{0,1\}$-valued random variables with mean μ. For $\delta > 0$, $\Pr[X \geq (1+\delta)\mu] \leq \exp\bigl(-\frac{\min(\delta,\delta^2)\,\mu}{4}\bigr)$, and for $\delta \in (0,1)$, $\Pr[X \leq (1-\delta)\mu] \leq \exp\bigl(-\frac{\delta^2\mu}{2}\bigr)$.*

Corollary A.45. *Let $X \sim \mathrm{Binomial}(n-1,p)$ with $p = c\,m/n$ for a constant $c \in (0,1)$. Then for all sufficiently large m, $\Pr[X \geq m] \leq e^{-\alpha m}$ with $\alpha = \alpha(c) > 0$.*

Proof. The mean is $\mu = (n-1)p \leq cm$, so $m \geq (1+\delta)\mu$ with $\delta \geq \frac{1-c}{c} > 0$, and Theorem A.44 applies with $\mu \geq cm/2$ for $n \geq 2$, giving $\alpha = \min\bigl(\frac{1-c}{c}, (\frac{1-c}{c})^2\bigr)\frac{c}{8}$. □

Proof of Theorem 1.15. Recall the hypotheses: $m = m(n) \geq 2$ with $m/\ln n \to \infty$ and $m = o(n)$, and $p = c\,m/n$ for a constant $c > 0$.

Case $c > 1$. The edge count $|E|$ of $G(n,p)$ is binomial with mean $\mu = \binom{n}{2}p = \frac{c\,m(n-1)}{2}$. With $\delta = 1 - \frac{1}{c}$ the lower-tail bound of Theorem A.44 gives

$$\Pr\Bigl[\,|E| \leq \tfrac{m(n-1)}{2}\Bigr] \;\leq\; \exp\Bigl(-\tfrac{(c-1)^2\,m(n-1)}{4c}\Bigr) \;\to\; 0,$$

so with high probability $|E| > \bigl\lfloor \frac{m(n-1)}{2} \bigr\rfloor$, and the contrapositive of Mader's theorem (Theorem 1.2) forces a pair with $\lambda \geq m$. Hence $\Pr[\lambda^{\max} \geq m] \to 1$.

Case $c < 1$. Each degree is $\mathrm{Binomial}(n-1,p)$, so by Corollary A.45 and a union bound over

the n vertices,

$$\Pr[\Delta(G) \geq m] \;\leq\; n\, e^{-\alpha m} \;=\; e^{\ln n - \alpha m} \;\to\; 0,$$

since $m/\ln n \to \infty$. By the degree bound (Lemma A.3) $\lambda^{\max} \leq \Delta(G)$, so $\Pr[\lambda^{\max} \geq m] \to 0$. □

Remark A.46 (The vertex and directed analogues). Below the threshold, $\kappa^{\max} \leq \lambda^{\max}$ gives the vertex statement for free, and in $D(n,p)$ the out-degree version of the same union bound applies. Above the threshold, the minimum degree exceeds m with high probability by the lower-tail bound. Mader's theorem alone does not carry this to the vertex problem, since $\kappa^{\max} \leq \lambda^{\max}$ runs the wrong way to deduce $\kappa^{\max} \geq m$ from the edge bound. What supplies it instead are the theorems of Bollobás [19] for $G(n,p)$ and its directed analogue [20], which state that in this regime the global connectivities equal the minimum degree (respectively minimum semi-degree) with high probability, so $\kappa^{\max}$ and the directed $\lambda^{\max}$ also cross m at $p^* = m/n$. The growth hypothesis $m/\ln n \to \infty$ is what lets the union bound beat the factor n. For fixed m, the bound is vacuous and finer tools (Poisson approximation of low-degree vertex counts) would be needed.

A.9 Computational transcripts

The cut-counting optimisation formulation that certifies Theorem 2.1 is given in Chapter 2. Here are the solver transcripts that back it, together with the exhaustive-search output that settles the directed base cases. The complete commented source accompanies the document as a single self-contained file, `erdos915_unified.py` in the `program/` directory. Its core runs on `numpy` and `scipy` alone, the certifier adds `pulp`, and `matplotlib` and `networkx` are optional and used only for the figures. An optional compiled helper accelerates selected small hot paths but is not needed for correctness.

```
Cut-MILP certification of the directed multigraph problem
M*(n) is the scaled optimum; L_m^dir(n) = (m-1) M*(n).

n=3: status=OPTIMAL, M*=4.000, target=2(n-1)=4, proof=True, time=0.0s, L_3^
    ↪ dir(n)=8
n=4: status=OPTIMAL, M*=6.000, target=2(n-1)=6, proof=True, time=0.3s, L_3^
    ↪ dir(n)=12
n=5: status=OPTIMAL, M*=8.000, target=2(n-1)=8, proof=True, time=5.9s, L_3^
    ↪ dir(n)=16
n=6: status=OPTIMAL, M*=10.000, target=2(n-1)=10, proof=True, time=1259.9s,
    ↪  L_3^dir(n)=20
```

Listing A.1. Solver output: $M^*(n) = 2(n-1)$, optimal with zero gap, for $n = 3, 4, 5, 6$.

The cut-counting optimisation closes the gap up to $n = 5$ inside a short budget and reaches $n = 6$ in about twenty minutes, after which the branch count defeats it. The base cases of the directed induction (Theorem 1.9) run to $n = 7$, and those are settled instead by the pruned exhaustive search over all simple digraphs with $\lambda^{\max} \leq 1$. That search is exact and complete at these sizes, and it agrees with $\ell_2^{\mathrm{dir}}(n) = M(n)$ throughout.

```
Exhaustive search over all simple digraphs with lambda^max <= 1 (directed,
    ↪ m=2).
Branch and bound; reported max arc count = ell_2^dir(n) = max(2(n-1), floor
    ↪ (n^2/4)).

  n=2:  max= 2,  formula= 2,  proved (complete),  nodes=5,          0.0s
  n=3:  max= 4,  formula= 4,  proved (complete),  nodes=22,         0.0s
  n=4:  max= 6,  formula= 6,  proved (complete),  nodes=537,        0.0s
  n=5:  max= 8,  formula= 8,  proved (complete),  nodes=17823,      0.1s
  n=6:  max=10,  formula=10,  proved (complete),  nodes=788101,     9.1s
  n=7:  max=12,  formula=12,  proved (complete),  nodes=45122129,   718.1s

The cut-MILP certifier of the multigraph problem closes n <= 6 (n = 6 in
    ↪ about
21 minutes); the directed m = 2 base cases through n = 7 that the induction
requires are settled by this exhaustive search instead.
```

Listing A.2. Exhaustive-search output for the base cases $2 \leq n \leq 7$: the maximum arc count equals $M(n) = 2, 4, 6, 8, 10, 12$, proved complete at each n.

A.10 How the program checks itself

The invariants are checked by the self-test at the foot of `erdos915_unified.py`, run simply with `python erdos915_unified.py`. It confirms the checker against the proven $m = 2$ values $2, 4, 6, 8$, the named constructions against their predicted sizes and connectivities, the Gomory–Hu shortcut against brute-force max-flow, the Berge connectivities of the hypergraph checker, the Monte Carlo appearance threshold and Whitney's inequality on every sample, the cut-counting optima for small n, the search reaching the certified optimum, and the driver returning the proven value or a feasible witness in each mode. Every check passes. A single companion script, `make_figures.py`, regenerates every figure in the thesis from the same file, with the mapping listed in `program/README.md`.

Appendix B

Supplementary Figures

This appendix collects the supplementary figures that the program produces but that the chapters reference rather than reproduce in full. Each one is named where it belongs in the body, and the file name printed in its caption matches the output of `make_figures.py` and the figure table in `program/README.md`. Figures already shown in the running text are not repeated here.

B.1 A gallery of extremal graphs

The extremal graphs are not abstractions. Figure B.1 draws nine of the named families the body argues about, each at a deliberately large representative size rather than at the smallest case, so the structure is visible. These are the special, structured extremisers, not the trivial small trees: the directed double star and bidirected star (hub at the centre, with arcs in both directions to every leaf), the dense bidirected complete graphs, the multigraph star at full multiplicity, and the star hypertrees, the last drawn as metro maps in which each hyperedge is one coloured line through its members. The program builds every one of these directly and, at the small sizes the enumeration reaches, also proves them optimal rather than merely feasible, so the gallery is the analysis made concrete at a size a reader can take in.

B.2 Search traces across the variants

Figure 3.2 in Chapter 3 follows one cooling run in detail. The two runs below repeat that experiment for two further, deliberately dissimilar variants, one undirected and simple, one directed and multigraph, to show the behaviour is the same across the family rather than special to one case. Each plots every visited graph by its binding connectivity (horizontal) against its size (vertical), colouring each point by the step at which it was visited, from the hot early exploration in warm tones to the converged tail in blue. In both the walk strays past the feasibility boundary while hot and is driven back onto the densest feasible graph as the temperature falls, exactly as in the body figure. The seeds are fixed, so each picture reproduces verbatim.

B.3 The variant landscape at $m = 6$ and in three dimensions

This appendix shows the per-graph connectivity distribution at the fixed forbidden threshold $m = 6$, alongside a three-dimensional view of the appearance threshold (Figure B.5). They confirm that raising the forbidden threshold relocates the feasibility boundary without changing the

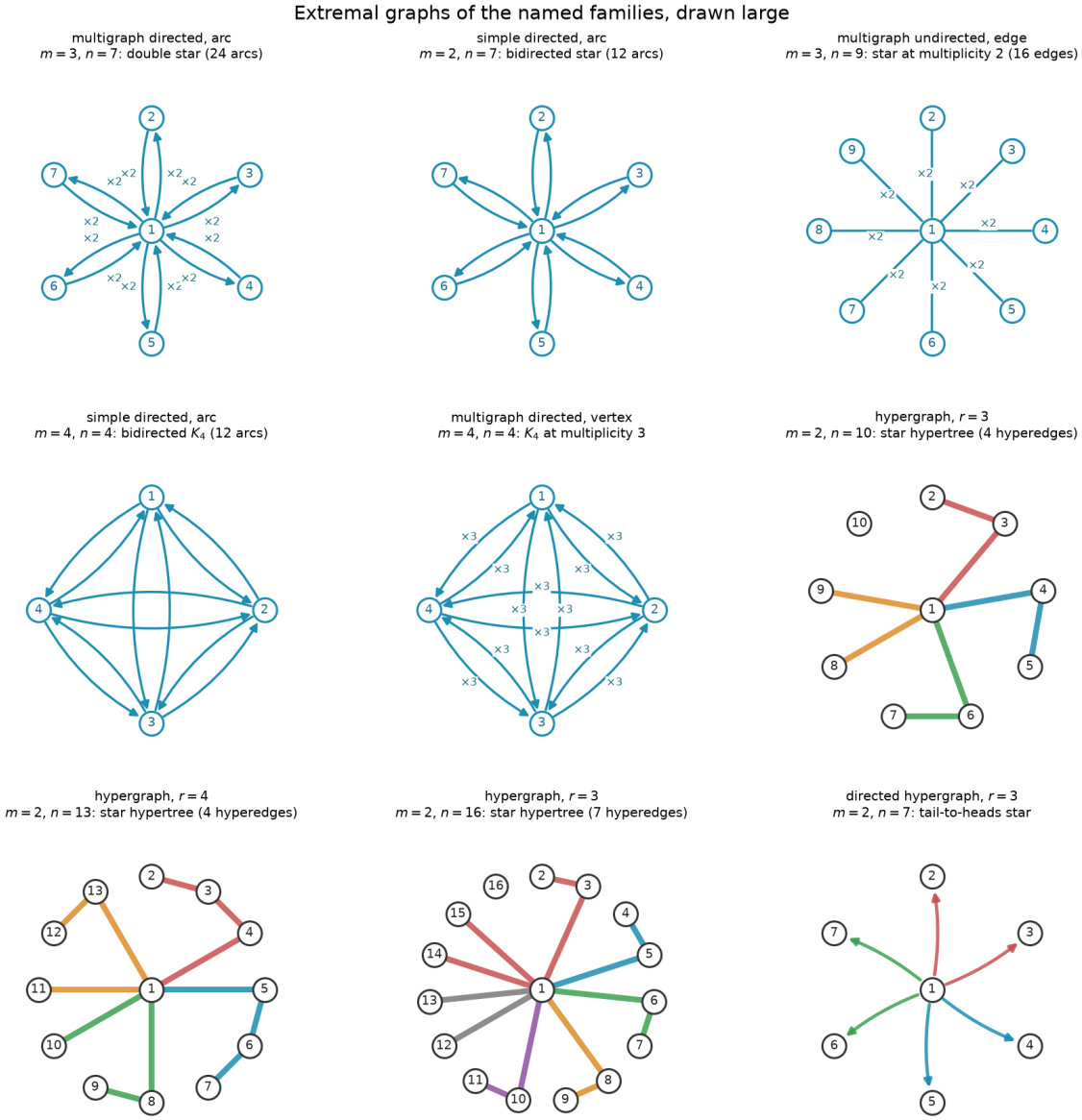


Figure B.1. Extremal graphs of the named families, drawn at a large representative size. *Top and middle rows:* matrix variants, with a central hub where the family has one. An arc multiplicity above one is shown by a $\times k$ label rather than parallel curves, and directed arcs carry arrowheads. *Hypergraph panels:* each hyperedge is a metro line through its member vertices, in its own colour, with the hub at the centre, and the directed hypergraph fans coloured arrows from each tail to its heads. Every graph shown is feasible at the stated m. The matrix and undirected-hypergraph families are extremal for their variant, the two directed stars drawn at the crossover $n = 7$, the largest size at which the linear star still ties the one-directional wall before the wall overtakes it. The final panel is the directed-hypergraph analogue, a tail-to-heads star, shown for the one variant whose exact value is still open (Proposition A.41). File `extremal_graphs_gallery.png`.

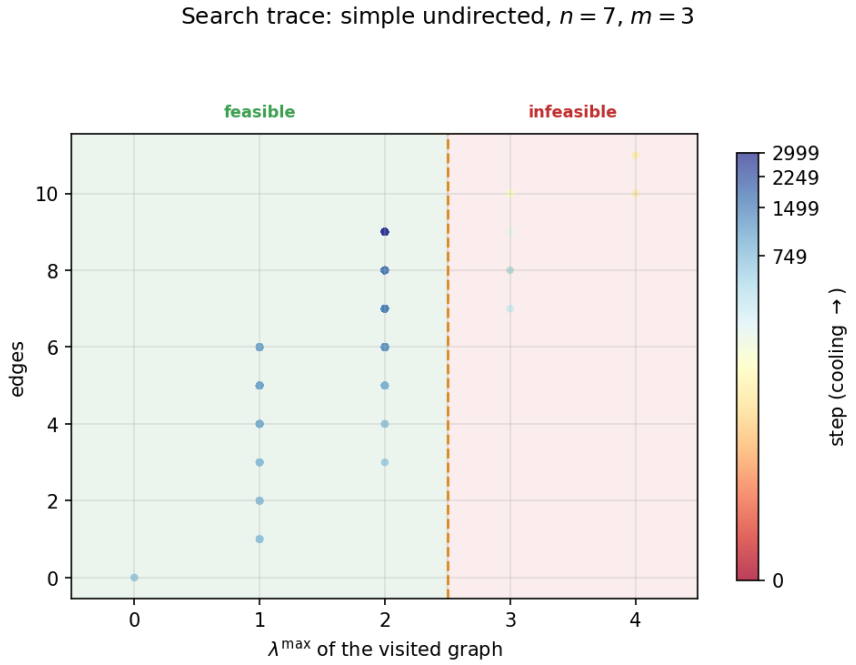


Figure B.2. Search trace for the simple undirected edge problem at $n = 7$, $m = 3$. File `trace_simple_undirected_n7_m3.png`.

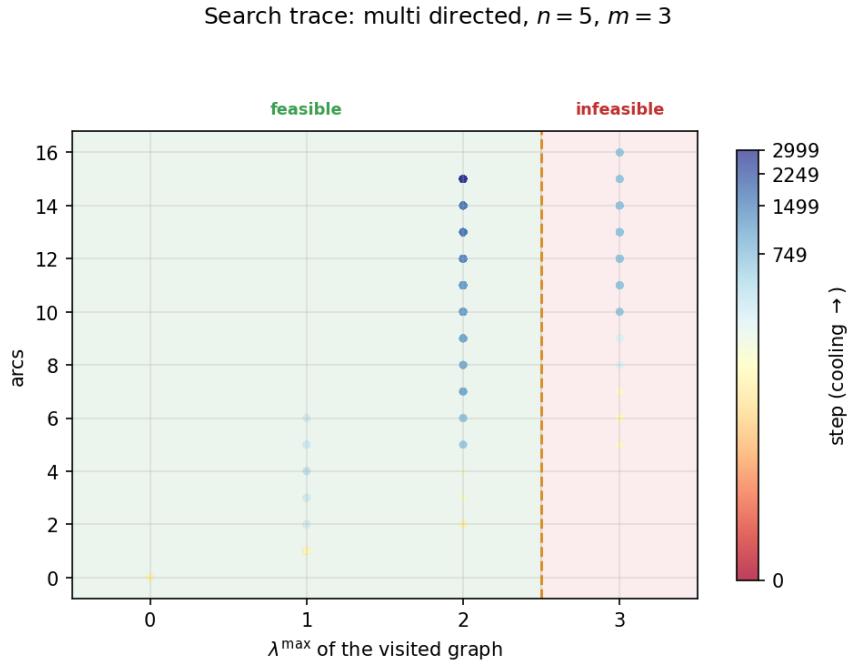


Figure B.3. Search trace for the multigraph directed arc problem at $n = 5$, $m = 3$. File `trace_multi_directed_n5_m3.png`.

qualitative picture. The pooled pair-connectivity and edge-count grids of Chapter 4 (Figures 4.4 and 4.5) already split each variant at its own mid-range connectivity, so they carry no separate $m = 6$ companion.

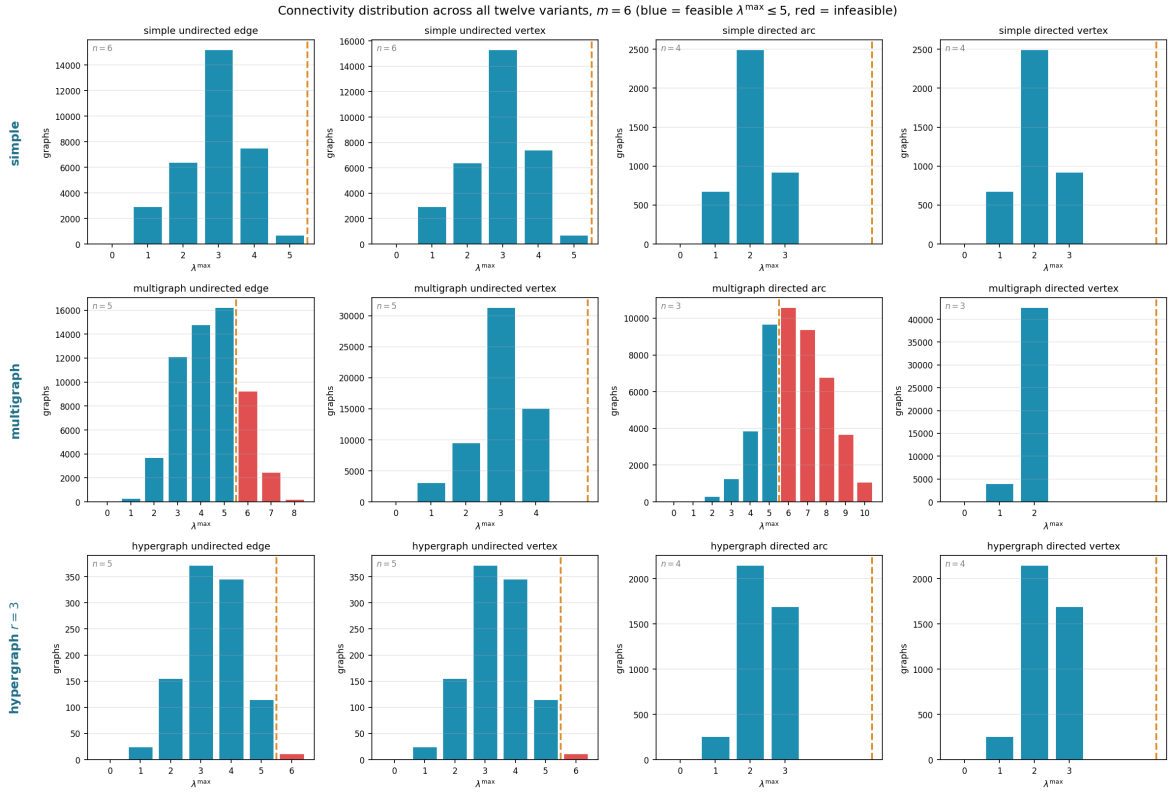


Figure B.4. How the full small-n enumeration distributes by binding connectivity $\lambda^{\max}$ across all twelve variants at the fixed forbidden threshold $m = 6$. Blue bars are feasible graphs ($\lambda^{\max} \leq 5$), red bars infeasible ones, and the dashed line marks the feasibility boundary. At this higher threshold most of the small graphs the enumeration reaches are feasible, so the blue mass dominates: raising m moves the boundary out past the bulk of the distribution. File `conn_dist_m6.png`.

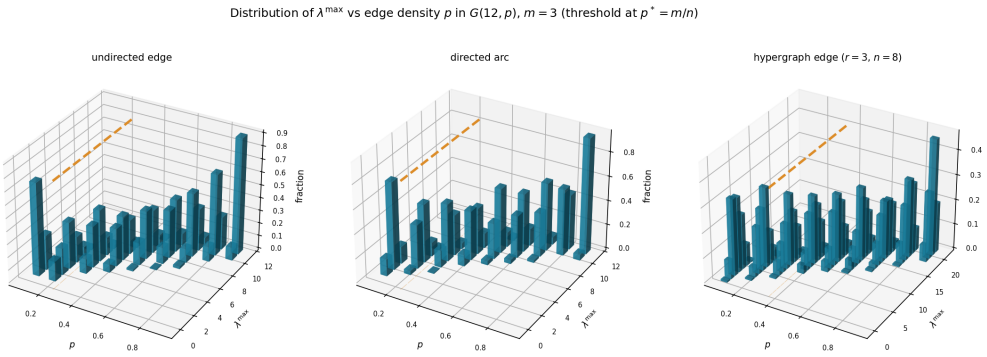


Figure B.5. The appearance threshold $p^* = m/n$ as a three-dimensional bar chart, for three representative variants (simple undirected edge, simple directed arc, three-uniform hypergraph edge) at $n = 12$, $m = 3$. Each panel plots the fraction of $G(n,p)$ samples (height) reaching each binding connectivity $\lambda^{\max}$ (depth) as the density p varies (width), so the mass migrates from low to high connectivity as p crosses the proved threshold $p^* = m/n$ of Theorem 1.15. File `threshold_3d.png`.

Bibliography

[1] Béla Bollobás and Pál Erdős. Extremal problems in graph theory. *Matematikai Lapok*, 13:143–152, 1962. Original statement of Problem 915.

[2] Karl Menger. Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927. The original statement of Menger's theorem.

[3] Pál Erdős. Erdős problems. `https://www.erdosproblems.com/915`. Problem 915.

[4] Wolfgang Mader. Ein Extremalproblem des Zusammenhangs von Graphen. *Mathematische Zeitschrift*, 131:223–231, 1973. Solves Erdős Problem 915 for edge-disjoint paths.

[5] Hassler Whitney. Congruent graphs and the connectivity of graphs. *American Journal of Mathematics*, 54(1):150–168, 1932.

[6] John L. Leonard. On a conjecture of Bollobás and Erdös. *Periodica Mathematica Hungarica*, 2(1–4):191–194, 1972. Proves $\ell_m(n) = k_m(n)$ for $m \leq 4$.

[7] B. A. Sørensen and Carsten Thomassen. On k-rails in graphs. *Journal of Combinatorial Theory, Series B*, 17(2):143–159, 1974. Determines $k_5(n) = \lfloor 8n/3 \rfloor - 3$.

[8] Kristóf Bérczi, Karthekeyan Chandrasekaran, Tamás Király, and Shubhang Kulkarni. Splitting-off in hypergraphs. In *Proceedings of the 51st International Colloquium on Automata, Languages, and Programming (ICALP 2024)*, Leibniz International Proceedings in Informatics (LIPIcs), 2024. arXiv:2307.08555.

[9] Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. Describes the nauty/Traces canonical-labelling tools underlying geng.

[10] Brendan D. McKay and Stanisław P. Radziszowski. $R(4, 5) = 25$. *Journal of Graph Theory*, 19(3):309–322, 1995. Exhaustive-generation determination of a Ramsey number.

[11] Marijn J. H. Heule, Oliver Kullmann, and Victor W. Marek. Solving and verifying the Boolean Pythagorean triples problem via cube-and-conquer. In *Theory and Applications of Satisfiability Testing (SAT 2016)*, volume 9710 of *Lecture Notes in Computer Science*, pages 228–245. Springer, 2016. The certified SAT proof is roughly 200 terabytes in DRAT format.

[12] Marijn J. H. Heule. Schur number five. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, pages 6598–6606. AAAI Press, 2018.

[13] Alexander A. Razborov. Flag algebras. *The Journal of Symbolic Logic*, 72(4):1239–1282, 2007.

[14] Ralph E. Gomory and Tien Chung Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961. Introduces the Gomory-Hu tree structure.

[15] Zsolt Baranyai. On the factorization of the complete uniform hypergraph. In *Infinite and Finite Sets (Colloq., Keszthely, 1973), Vol. I*, volume 10 of *Colloq. Math. Soc. Janos Bolyai*, pages 91–108. North-Holland, 1975.

[16] W. T. Tutte. *Connectivity in Graphs*. University of Toronto Press, Toronto, 1966.

[17] John E. Hopcroft and Robert E. Tarjan. Dividing a graph into triconnected components. *SIAM Journal on Computing*, 2(3):135–158, 1973.

[18] Wolfgang Mulzer. Five proofs of Chernoff's bound with applications. *Bulletin of the EATCS*, 124, 2018. Accessible survey of Chernoff bound variants.

[19] Béla Bollobás. The evolution of random graphs—the giant component. *Transactions of the American Mathematical Society*, 286(1):257–274, 1984. Shows $\kappa(G(n,p)) = \delta(G(n,p))$ with high probability.

[20] Alan Frieze and Michał Karoński. *Introduction to Random Graphs*. Cambridge University Press, 2016. Chapter 7 covers connectivity in $G(n,p)$ and $D(n,p)$; the directed analogue of Bollobás's $\kappa = \delta$ result appears in Section 7.3.

DEPARTMENT OF MATHEMATICS
Celestijnenlaan 200B
3001 Leuven, Belgium
wis.kuleuven.be